

# RECTOR

---

The Power of  
Automated Refactoring



Matthias Noback  
Tomas Votruba

# Rector - The Power of Automated Refactoring

Matthias Noback and Tomas Votruba

This book is for sale at

<http://leanpub.com/rector-the-power-of-automated-refactoring>

This version was published on 2021-06-24



Leanpub

This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2020 - 2021 Matthias Noback and Tomas Votruba

# Contents

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Preface</b> . . . . .                               | <b>i</b> |
| A Trainer's Journey . . . . .                          | i        |
| The Other Side . . . . .                               | ii       |
| Who's Fault is It? . . . . .                           | ii       |
| From Blame to Pain . . . . .                           | ii       |
| From Pain to Idea . . . . .                            | iii      |
| From Idea to First Real Reconstructor . . . . .        | iv       |
| We've Just Started, Now You Get on Board . . . . .     | iv       |
| <b>Introduction</b> . . . . .                          | <b>v</b> |
| What is Rector? . . . . .                              | v        |
| Who Should Read This Book? . . . . .                   | vi       |
| An Overview of the Contents . . . . .                  | vi       |
| About the Code Samples . . . . .                       | vii      |
| About the Authors . . . . .                            | vii      |
| Matthias Noback . . . . .                              | vii      |
| Tomas Votruba . . . . .                                | viii     |
| Acknowledgements . . . . .                             | viii     |
| We Want Your Feedback . . . . .                        | ix       |
| <b>Programmatically Modifying PHP Code</b> . . . . .   | <b>1</b> |
| Introduction . . . . .                                 | 1        |
| Primitive Ways of Modifying Code . . . . .             | 2        |
| Tokenizing PHP Code . . . . .                          | 4        |
| Parsing PHP Tokens: the Abstract Syntax Tree . . . . . | 8        |
| Converting the AST Back to PHP Code . . . . .          | 14       |
| Manipulating the AST . . . . .                         | 16       |

## CONTENTS

|                                                               |           |
|---------------------------------------------------------------|-----------|
| Node Visitors . . . . .                                       | 18        |
| Manipulating the AST with a Node Visitor . . . . .            | 23        |
| Built-in Node Visitors . . . . .                              | 27        |
| Connecting Nodes . . . . .                                    | 27        |
| Resolving Fully-qualified Names . . . . .                     | 31        |
| Finding Nodes . . . . .                                       | 33        |
| Summary . . . . .                                             | 37        |
| <b>PHP Tools in the Game . . . . .</b>                        | <b>38</b> |
| Introduction . . . . .                                        | 38        |
| Working Together with Giants . . . . .                        | 38        |
| 2007 to 2021 Timeline . . . . .                               | 39        |
| The Primary Feature . . . . .                                 | 39        |
| 1. Coding Standard Tools . . . . .                            | 40        |
| PHP_CodeSniffer . . . . .                                     | 40        |
| PHP CS Fixer . . . . .                                        | 41        |
| EasyCodingStandard . . . . .                                  | 42        |
| 2. Static Analyzers . . . . .                                 | 42        |
| PHP-Parser . . . . .                                          | 42        |
| PHPStan . . . . .                                             | 43        |
| Psalm . . . . .                                               | 43        |
| 3. Instant Upgrade Tools . . . . .                            | 44        |
| Symfony-Upgrade-Fixer . . . . .                               | 44        |
| Rector . . . . .                                              | 44        |
| 4. Instant Downgrade Tools . . . . .                          | 45        |
| PHP-Backporter . . . . .                                      | 45        |
| Galapagos . . . . .                                           | 45        |
| Transphpile . . . . .                                         | 45        |
| 7to5 . . . . .                                                | 46        |
| Rector . . . . .                                              | 46        |
| When to Use Which Tool? . . . . .                             | 46        |
| Is Your Project Bare Without Any Tools? . . . . .             | 47        |
| Run Rector First, Then Polish with Coding Standards . . . . . | 47        |
| Recommended Tools . . . . .                                   | 48        |
| Summary . . . . .                                             | 48        |

## CONTENTS

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>Creating Your First Rector Rule</b> . . . . .              | <b>49</b>  |
| Introduction . . . . .                                        | 49         |
| What's a Rector Rule? . . . . .                               | 49         |
| Creating a Custom Rule . . . . .                              | 50         |
| Extending AbstractRector . . . . .                            | 54         |
| Finding the Right Node Class . . . . .                        | 58         |
| Running a Single Rule . . . . .                               | 61         |
| Refactoring the Method Call Node . . . . .                    | 64         |
| What Is the Type of a Variable? . . . . .                     | 67         |
| What if We Run Rector Twice? . . . . .                        | 73         |
| Effectivity Beats Perfection . . . . .                        | 73         |
| Summary . . . . .                                             | 74         |
| <b>Test-driven Rule Development</b> . . . . .                 | <b>75</b>  |
| Introduction . . . . .                                        | 75         |
| Migrating from DateTime to DateTimeImmutable . . . . .        | 76         |
| Creating a Test Class . . . . .                               | 79         |
| Adding First Test Fixture . . . . .                           | 84         |
| Making the First Test Pass . . . . .                          | 86         |
| Narrowing the Scope of the Refactoring . . . . .              | 90         |
| Capturing the Return Value of modify() . . . . .              | 95         |
| Skipping Calls On Classes That Are Not DateTime . . . . .     | 104        |
| Summary . . . . .                                             | 107        |
| <b>More Testing Techniques</b> . . . . .                      | <b>108</b> |
| Testing Multiple Rules Combined . . . . .                     | 108        |
| Rules Should Be Idempotent . . . . .                          | 111        |
| Removing the Clone Step . . . . .                             | 113        |
| Comparing Return Values to Validate Correctness . . . . .     | 123        |
| Shouldn't You Start with a Rule Set Test? . . . . .           | 125        |
| Low-Hanging Fruit First . . . . .                             | 126        |
| Create a Configurable Generic Rule . . . . .                  | 127        |
| Using a Configurable Instead of a Custom Rule . . . . .       | 128        |
| Making Rule Behavior Configurable . . . . .                   | 131        |
| Configurable Rule Makes Hidden Assumptions Explicit . . . . . | 135        |
| Using Supporting Classes in Fixtures . . . . .                | 136        |

## CONTENTS

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| Summary . . . . .                                                        | 140        |
| <b>Continuous Rectifying . . . . .</b>                                   | <b>142</b> |
| Introduction . . . . .                                                   | 142        |
| The Next Member of Your Team . . . . .                                   | 143        |
| Turning a Junior Into a Senior on Day 1 . . . . .                        | 143        |
| Who's to Blame? . . . . .                                                | 145        |
| What Does it Look Like to Work with Rector in CI? . . . . .              | 146        |
| How Rector is Rectifying Itself . . . . .                                | 147        |
| Work First, Enjoy Later . . . . .                                        | 149        |
| Removing Boring Work Opens Your Creativity . . . . .                     | 150        |
| Summary . . . . .                                                        | 151        |
| <b>Rector + GitHub Actions = Next Member of Your Team . . . . .</b>      | <b>152</b> |
| Introduction . . . . .                                                   | 152        |
| What Are The Steps for a Rector Run? . . . . .                           | 153        |
| How to Add Rector to GitHub Actions . . . . .                            | 154        |
| 1. Generic PHP Setup . . . . .                                           | 154        |
| 2. Rector Setup . . . . .                                                | 155        |
| What Feedback Do We Get? . . . . .                                       | 155        |
| How to Work with Rector's Output? . . . . .                              | 156        |
| 1. Manual Changes . . . . .                                              | 157        |
| 2. Run Rector Locally . . . . .                                          | 157        |
| 3. Let Rector Code for Us in Pull Requests . . . . .                     | 157        |
| What is The Work We Want to Automate? . . . . .                          | 158        |
| Make Your Rector Workflow Come Alive . . . . .                           | 158        |
| Make Rector Push Changes . . . . .                                       | 160        |
| Summary . . . . .                                                        | 162        |
| <b>The Future of Instant Upgrades . . . . .</b>                          | <b>163</b> |
| Introduction . . . . .                                                   | 163        |
| Removing Legacy . . . . .                                                | 164        |
| How Can We Be So Confident? . . . . .                                    | 164        |
| No Backward-Compatibility Breaks and No Backward-Compatibility . . . . . | 165        |
| A Bot That Sends Pull-Requests Daily . . . . .                           | 167        |
| Instant Upgrades on Click . . . . .                                      | 168        |
| PHP Stands Out . . . . .                                                 | 169        |

## CONTENTS

|                           |            |
|---------------------------|------------|
| <b>Epilogue . . . . .</b> | <b>170</b> |
|---------------------------|------------|

# Preface

*By Tomas*

Working with old code is painful. In any framework, in any company, for you, and for your employer. Nobody wants to ride a horse on the highway. But why are so many people doing it? Why are there so many legacy PHP projects?

## A Trainer's Journey

I was asking myself this question 6 years ago, almost every day. I was doing trainings on Nette, Symfony, or Doctrine. My usual training looked like this: I came to a company, I gave a presentation about Symfony and its *cool new features*, we did hands-on coding, and I answered curious questions. I had a good feeling about this and feedback from participants was also positive. I gained a lot of experience from trainings in 30+ different PHP companies and used it in every next training. I thought: "That's what trainers do, I finally made it; changing the life of others in just one or two days." Gosh, I was proud of myself. Helping others is my life-long passion, so I was very happy, and also well paid. This feeling was soon to be changed.

I had one training about new features in PHP 7 for a project that was running PHP 5.4. After a year, the same company asked me for a fresh training about new features in the PHP world. I thought: "They must be on PHP 7.1 by now." I asked them, where are they now, so I wouldn't bore them with something they already know. Their answer: "This year we'll definitely get to a new PHP version." I asked: "Oh, you're up to PHP 7.2 already? Wow!" They replied: "No, we're almost at the PHP 7. That's why we need you to help us!"

What? My mind was shocked. Suddenly, I remembered another training I had, where they asked for almost the same topic as a year before. No, that can't be right, can it?



## The Other Side

A trainer's point of view, and one or two days of training, are just one side of the story. A demo is easy to code. Everybody is happy because they work on 200 lines of code and the progress is there. Then the developers have to go through a real code base and make the change happen there as well. They have to evolve the project from 1.000.000 to 5.000.000 lines of code, help the business to grow, and add new features. I realized that my trainings may expand knowledge by showing the hot news in the PHP world or a particular framework. But barely 2% of this knowledge will last for longer than a week after the training.

## Who's Fault is It?

I started to blame myself. My teaching methods were likely not very effective. I tried to improve my presentation and learn more about the project beforehand, but the results didn't change much. Then I blamed the company business leaders. They were nice to hire a trainer, but after he or she is gone, they'd force programmers to work on that legacy code to add more features instead of refactoring it. Then I blamed programmers, who are not willing to spend four hours after work to educate themselves. They should be able to protest against legacy business and stop it, until the code is clean and fun to work with.

Later I've learned, none of these targets are right.

## From Blame to Pain

I stopped giving training so often because I felt like a failure. I was part of a system that didn't make much change for the better. The system repeated the same action twice, with no gain. I couldn't do it anymore. I mean, I physically could not do it. Why not? Let me explain.

When I was 8 years old, I lost a cap while playing outside. In situations like these, my father used to punish me with tasks involving lots of repetition. I always had to write long sentences, a 100 times, like: "I will never go outside while playing and not

returning back without the hat again, or I will have to go out and look for it and won't return until I find it."

At first it was an interesting exercise in writing. After the 30 sentences though, my hand started to hurt. The text was supposed to be clear to read and without any mistake, and there were 10 more sentences left to go. I remember a Saturday, when I spent almost 4 hours on this, unable to do anything else. My mom was helping me sometimes, adding 20 sentences herself.

This was repeated a couple times a month. I became terrified to do anything my father wouldn't like. These punishments came out of the blue. Once my brother broke my father's tool and told it was me. I said I don't know anything about it, so I got the 100-sentences punishment not to lie ever again.

Thanks to these punishments, I've hardcoded in my brain **that any repetition hurts me and I'll do anything to avoid it**. That's my personality trait. It took some time but I've put some healthy boundaries on this and learned to use it to my advantage.

P.S. Don't take me wrong: I love my father and we have a safe and warm relationship now. I'm very grateful for the patience, attention, focus, and resilience he taught me. Thanks to him I'm the man I want to be.

## From Pain to Idea

Do you know when you look at the code, you feel it's wrong, but you don't know what it is? I had this feeling. I knew there must be some better way to do this that is cheaper, faster and re-usable. It won't be easy to find it, but there must be a way to solve this.

I started to share this pain and frustration in chats with my friends and I slowly built up a vision. A vision of a world where all programmers are only thinking about new algorithms, and where terms like "legacy code" and "technical debt" don't exist. Changing frameworks is a matter of a single click. Upgrades are as easy as composer update. And it's all open source, so anyone can use it.

## From Idea to First Real Reconstructor

I had a weekend trip to Brno to visit a friend of mine, Petr Vacha. He encouraged me to just give it a try, when I didn't see the light yet. Thank you Peter. We went to a Happy Tearoom on Freedom Square, I think Saturday Jul 15, 2017 it was, and we spent 4 hours there. Petr doing his coding, and me, struggling with the abstract syntax tree and `nikic/php-parser`, without any IT University education. It was stressful as hell. That's how the [first commits to \*Reconstructor\*](#)<sup>1</sup> (before it was shortened to “Rector”) came to be.

## We've Just Started, Now You Get on Board

That was 2017. Today it's 2021 and legacy code is still a thing. Yes, we have the technology to change the PHP world. Rector is maturing and has over 630 rules that automate boring work. Hundreds of companies already use it to automate their upgrades and improve code quality in their CI. But that's not enough.

That's the reason we're writing this book. To empower you to both join the Rector community, and write rules that help you with your problems. If you're reading this book, you probably have the same feeling I had: “There must be a better way.” There is. We'll show you the way, and we welcome you to get on board with us.

---

<sup>1</sup><https://github.com/rectorphp/rector/commits/v0.1.0?after=0cc75d58bee80233531e5c1d41cd49fba9ca81bb+1084&branch=v0.1.0>

# Introduction

## What is Rector?

Rector is a tool that came into existence because Tomas wanted to help developers everywhere to:

- Prepare their code base for the latest PHP version
- Effortlessly upgrade to the next version of their current framework, or even
- Switch between frameworks if needed
- Improve their code quality without thinking about it
- Define their own automated refactoring procedures

The core abilities of a tool like this would be:

- Being able to understand PHP code
- Being able to manipulate PHP code without breaking it
- Being able to save the upgraded PHP code back to their locations on disk

**Rector**<sup>2</sup> can do all of this, and while it reads and parses your code, it will apply a number of *rules* to it. By doing so, Rector improves and upgrades your code, without human intervention. It won't make the typical mistakes that a human being will when they're typing and copy/pasting code in a text editor.

Some concrete examples of what Rector can do for you:

- Upgrade Symfony from 2.8 to 3.0, to 3.1, all the way to the latest version
- Turn property annotations into actual property types, add return types, etc.
- Change nested `if s` to early `returns`.
- Upgrade classic `switch` statements to PHP 8 `match` statements.
- And the list goes [on and on](https://github.com/rectorphp/rector/blob/main/docs/rector_rules_overview.md)<sup>3</sup>

---

<sup>2</sup><https://github.com/rectorphp/rector/>

<sup>3</sup>[https://github.com/rectorphp/rector/blob/main/docs/rector\\_rules\\_overview.md](https://github.com/rectorphp/rector/blob/main/docs/rector_rules_overview.md)

## Who Should Read This Book?

This is a book for PHP developers who want to modernize their project. Whether the project consists of the worst case of legacy code, or is actually quite modern; knowing how to work with and extend Rector is going to help your project move forward.

We're assuming knowledge of object-oriented PHP programming and several years of experience maintaining PHP projects.

## An Overview of the Contents

In the first chapter, Matthias explores the topic of modifying PHP code *with* PHP code, or [Programmatically modifying PHP code](#). You'll learn about the *Abstract Syntax Tree*, and how to create and modify it with PHP-Parser. Tomas continues with a discussion of the various [PHP tools in the game](#) that can help you with modifying code in an automated fashion. Next, Matthias shows you how to migrate from low-level PHP-Parser node visitors to Rector rules. You'll be [Creating your first Rector rule](#) in this chapter.

Rector makes it easy to create rules, but the process will be challenging anyway. Matthias continues with a demonstration of [test-driven rule development](#). There are several [more testing techniques](#) that need to be discussed, so Matthias adds one more chapter about TDD in the context of Rector.

Now that you know how Rector works, how you can use and extend it, Tomas helps you get your project to the next level. He shows you how you can not only use Rector incidentally, but apply [Continuous Rectifying](#) to your project. In the next chapter Tomas demonstrates how [Rector combined with GitHub Actions can become the Next Member of Your Team](#). The last chapter is connecting some dots: with sophisticated tools Composer, PHP-Parser, and now Rector, we could get rid of wasteful Backward-Compatibility promises, and free the world from legacy code.

We finish this book with the [Epilogue](#), which is in the form of an interview with Tomas. We'll take a look at the vision for Rector and what we might expect from the project in the long run.

## About the Code Samples

The code samples in this book aim to be as realistic as possible. In fact, we run the following tools on them to ensure their correctness now and in the future:

- PHPUnit, for verifying our own assumptions, and the correctness of the code samples.
- Rector, for automatic improvements, migrations to future PHP versions, libraries, etc.
- EasyCodingStandard, for applying a uniform coding style and to make all the lines fit within a single page width.

The latter isn't always possible, e.g. in the case of `use` statements which are sometimes quite long. In those cases we abbreviate the namespace and add a comment (`// (abbreviated)`). When copy/paste-ing code samples into your own project this means you sometimes have to manually expand these namespaces before you can run the code.

When showing the code, we sometimes remove somewhat irrelevant parts so they don't take up too much space on the page. However, you should always be able to figure out the full example based on the previous code samples. That way you can follow along with the tutorial chapters in your own project.

If something doesn't work in your project, please try the following options first:

- Upgrade Rector to its latest version (`composer require --dev rector/rector`).
- Run `composer dump-autoload` to let Composer regenerate its class loader.

If it still doesn't work, please submit an issue to our [feedback repository](https://github.com/rectorphp/the-power-of-automated-refactoring-feedback)<sup>4</sup> on GitHub.

## About the Authors

### Matthias Noback

---

<sup>4</sup><https://github.com/rectorphp/the-power-of-automated-refactoring-feedback>

Matthias Noback has been building web applications since 2003. He is the author of the *Object Design Style Guide* and *Advanced Web Application Architecture*. He's also a regular [blogger](#), [speaker](#) and [trainer](#)<sup>5</sup>.



## Tomas Votruba

Tomas loves to combine open-source and innovations... yet he's super lazy. That's why he always spends dozens of hours to automate problems that would take an hour or two to do manually. Fortunately, he prefers those problems that can help the whole PHP community. Tomas is a [regular blogger](#), [speaker](#) and [trainer](#)<sup>6</sup> as well.



## Acknowledgements

### Tomas

This book and its know-how is a team effort. That's why we would like to thank a few people - without them this book would never exist.

To Kerrial Beckett Newham for the tasty dinner where he made up the working and final title of the book. To Petr Vacha for one day in a Brno teahouse, where he supported my crazy idea of automated refactoring - there the first Rector commits were made, 4 years ago. To Jan Mikes for the fastest framework migration in a week. To Sebastian Schreiber for getting me back on track with a Rector vision and great laughs. To Matthias Noback, my hero blogger who helped me to overcome my scariest dream of writing a book like a walk in the park. Last but not least, to my dad for being my very first supporter on Patreon, although he never paid by card online before. It means the world to me.

Thank you, Tomas.

---

<sup>5</sup><https://matthiasnoback.nl>

<sup>6</sup><https://tomasvotruba.com/>

## Matthias

Publishing a book is a scary business. You put in a lot of time without knowing how your work is going to be received. For *Rector - The Power of Automated Refactoring*, your support, reviews, and suggestions have been very helpful. You have successfully encouraged us to finish this project. Thanks a lot for that!

On a more personal note, thank you Tomas, for trusting me to write a book that would do your project justice. Along the way you have been able to stir up my old-fashioned ways of writing PHP. You've shown me so many new ways to modernize, and automate this project, and for making the tools that keep me on track. Thanks for being my personal Rector.

## We Want Your Feedback

Because Rector is continuously changing, this book will also be a continuous work in progress. We'll keep improving its content, updating it according to future changes in Rector, PHP, and related libraries (in fact, we use Rector itself to refactor this book project!). So we're always looking to receive feedback about this book.

Did you notice any mistakes, or have an improvement in mind? Please let us know in the [Feedback repository on GitHub](https://github.com/rectorphp/the-power-of-automated-refactoring-feedback)<sup>7</sup>.

Now, let's get started with the main content of the book.

---

<sup>7</sup><https://github.com/rectorphp/the-power-of-automated-refactoring-feedback>



# Programmatically Modifying PHP Code

## This Chapter Covers:

- Generating tokens for PHP code
- Generating an Abstract Syntax Tree (AST)
- Visiting AST nodes
- Manipulating AST nodes

## Introduction

In order to automatically refactor PHP code for you, Rector needs a safe way to manipulate code. As a developer, you know it's easy to break existing functionality when you're changing code manually. Rector has an advantage here: given that it's a program, when you give it the right instructions, it will not make the kind of human errors that we tend to make.

How can Rector be so sure it's making the right changes though? Part of this question will be answered in the chapter about [test-driven rule development](#). We have to write tests to prove that Rector makes the right changes in any imaginable situation. Besides an extensive test suite, Rector also relies on low-level language analysis tools like [PHP-Parser](#)<sup>8</sup> and [PHPStan](#)<sup>9</sup>. Using these tools Rector can, for instance, distinguish a property name from a variable name, and derive its type.

This chapter is a discussion of the fundamentals of software that manipulates code. It covers the concepts that are at the core of Rector itself: parsing PHP code, visiting nodes of an abstract syntax tree, and manipulating them while visiting them.

---

<sup>8</sup><https://github.com/nikic/PHP-Parser>

<sup>9</sup><https://phpstan.org/>

## Primitive Ways of Modifying Code

A common change we developers make every day is to rename a method. This change is also known as the *Rename method* refactoring. For example, we have a repository class with a `find($id)` method. We realize that `find()` doesn't communicate the fact that we don't want the computer to find the object, but to *get* it. So we want to rename the method to `getById($id)`.

PHP code is just text, so we can look for occurrences of `find()`, erase the `find` part and type in `getById`. Looking for occurrences in the repository class alone is not enough. If the repository class extends another class, or implements an interface, these files may also contain the now outdated `find()` method. Even if we change those occurrences as well, we won't be done yet. Potentially many clients are still calling the `find()` method, and they should now switch to `getById()`.

Of course, we all have smart IDEs like PhpStorm, which make this kind of refactoring very easy and relatively safe for us. What if we don't want to make this change in an IDE. What if we want to do it with a PHP script? What's the best way to make the change? The first thing to try is some kind of global find and replace action, possibly using a function like `str_replace()`. This most likely results in a number of wrong changes. For example:

- If we're not careful about upper versus lower case characters, a class called `Finder` would be renamed to `getByIder`.
- Even if we're careful about casing, a `refind()` method would be renamed to `regetById()`.
- We might accidentally change a comment: `/* TODO find a better way */` would become `/* TODO getById a better way */`.

Once we realize a plain-text find/replace action isn't going to help, we could try to write a regular expression. The expression would find all the occurrences we're actually looking for, and would rule out a lot of *false positives*. For instance, all the currently existing `find()` methods, in classes and interfaces alike could possibly be found with the expression `/\s(find)\(/`. The space in front of the `find` word is important, because a method definition also has that space (e.g. `public function find()`). Given that the expression expects an opening parenthesis `(` after `find`, this

would leave a comment like `/* TODO find a better way */` intact. What about an old-school function like `function find()` though? If we use the expression `/\s(find)\(/`, this function will also be renamed.

There are several other problems related to whitespaces in PHP code. A method or function can have whitespace between the name and the opening parenthesis: `function find ()` is syntactically correct. You can also start a new line in the middle of a function declaration if you want. And we didn't even address the biggest issue yet: renaming based on text occurrences will likely overshoot the goal. We'll be accidentally renaming methods of other repositories, and we'll be renaming method calls to classes that we don't even own. Okay, it's clear now; both plain-text and regular expression-based find/replace isn't going to work. We need a smarter solution.

What do we need to become more successful at renaming methods? We need the skill that sets aside a plain-text editor from an IDE: the ability to *understand* the code itself. Of course, our brains have this skill. They can look at a piece of code and immediately recognize what needs to be renamed, and what needs to stay the same:

```
<?php
```

```
class Repository
```

```
{
    // should be renamed to getById():
    public function find() {
    }
}
```

```
class AnotherRepository
```

```
{
    // should stay the same:
    public function find() {
    }
}
```

```
// should stay the same:
/* TODO find a better way */
```

```
$repository1 = new Repository();  
// should use getById():  
$repository1->find();  
  
$repository2 = new AnotherRepository();  
// should stay the same:  
$repository2->find();
```

A program can do this too, by analyzing the code, splitting it up into chunks of characters that belong together, and by establishing context, like: is this a method or a function, what type does this variable have? IDEs need to do this in order to give the developer suggestions while typing, also known as code-completion. They also use this ability to help the developer perform some common refactorings, like moving and renaming classes, methods, variables, etc.

Rector uses similar techniques as IDEs do. The difference between Rector and your IDE is that Rector can refactor code from the command-line and can even do it in a [Continuous Integration](#) setup. Its catalog of supported refactorings is also much larger than what any IDE offers today.

In the following sections we'll discover the low-level language tools that Rector uses to support automated refactorings. A basic understanding of them will help you write your own refactoring rules later on.

## Tokenizing PHP Code

All programming languages have some kind of compiler that can read code written in the (more or less) human-readable programming language, and converts it into machine-readable instructions. The process consists of several archetypal steps:

1. Lexing
2. Parsing
3. Compiling

The compiled version of the code can then be executed.

Part of the lexing process is the tokenization of the code. The lexer starts at the beginning of a string of PHP code and splits the string into recognizable chunks. In PHP, you can inspect the result of tokenizing a piece of PHP code by calling the `tokenize()` method of the built-in class `PhpToken`<sup>10</sup> (available since PHP 8.0; if you're on an older version of PHP use the `token_get_all()`<sup>11</sup> function instead). This static method takes a string of PHP code as input and returns an array of *tokens*. As an example we'll *tokenize* the following code snippet:

```
<?php
echo 'Hello, world!';
```

Assuming that this code is loaded into the `$source` variable (e.g. using `file_get_contents()`), we can call `PhpToken::tokenize()` on it and show the result using `print_r()`:

```
$allTokens = PhpToken::tokenize($source);

print_r($allTokens);
```

The output is (abbreviated):

```
Array
(
    [0] => PhpToken Object
        (
            [id] => 390
            [text] => <?php

            [line] => 1
            [pos] => 0
        )
)
```

---

<sup>10</sup><https://www.php.net/manual/en/class.phptoken.php>

<sup>11</sup><https://www.php.net/manual/en/function.token-get-all.php>

```
[1] => PhpToken Object
(
    [id] => 326
    [text] => echo
    [line] => 2
    [pos] => 6
)

[2] => PhpToken Object
(
    [id] => 393
    [text] =>
    [line] => 2
    [pos] => 10
)

[3] => PhpToken Object
```

Each token, or “potentially meaningful chunk of code”, is represented by a `PhpToken` object. It exposes the type of the token (returned by `getTokenName()`), starting with `T_*`. Amongst other things, you can find out on which line of the source code the token was found, by fetching the object’s `line` property. A token also has a textual value which can be retrieved from its `text` property. Looping over the tokens we can create this nice little table:

| Line | Token                                   | Value                        |
|------|-----------------------------------------|------------------------------|
| 1    | <code>T_OPEN_TAG</code>                 | <code>&lt;?php\n</code>      |
| 2    | <code>T_ECHO</code>                     | <code>echo</code>            |
| 2    | <code>T_WHITESPACE</code>               |                              |
| 2    | <code>T_CONSTANT_ENCAPSED_STRING</code> | <code>'Hello, world!'</code> |
| 2    | <code>;</code>                          | <code>;</code>               |
| 2    | <code>T_WHITESPACE</code>               | <code>\n</code>              |

Note that the semi-colon (`;`) doesn’t have a corresponding token type. Tokens like this don’t have a variable value, they are just single characters. In the case of the semi-colon, you can’t use other characters to indicate the end of a statement. The same goes for brackets, parentheses, etc.

Using the data in this list we could recreate the original PHP code. We'd only have to concatenate all the token values:

```
$backToCode = '';  
foreach ($allTokens as $token) {  
    $backToCode .= $token;  
}  
//echo $backToCode;
```

Although tokenization is the first step in the compilation process, other tools may find it useful to do their own type of specialized work. For example, most coding standard tools (e.g. [PHP-CS-fixer](https://github.com/FriendsOfPhp/PHP-CS-Fixer)<sup>12</sup> and [PHP\\_CodeSniffer](https://github.com/squizlabs/PHP_CodeSniffer)<sup>13</sup>) operate at the level of tokens. If a coding standard defines that a PHP open tag should be followed by a blank line, we could make the code follow this standard by tokenizing the original code, then inserting a `T_WHITESPACE` token with a value of `\n` directly after the `T_OPEN_TAG` token:

```
$reformattedTokens = [];  
foreach ($allTokens as $token) {  
    $reformattedTokens[] = $token;  
    if ($token->is(T_OPEN_TAG)) {  
        $reformattedTokens[] = new PhpToken(  
            T_WHITESPACE,  
            "\n"  
        );  
    }  
}
```

If we then concatenate the token values again, just like we did before, the modified code can be copied back to the original file. As expected, the result then looks like this:

---

<sup>12</sup><https://github.com/FriendsOfPhp/PHP-CS-Fixer>

<sup>13</sup>[https://github.com/squizlabs/PHP\\_CodeSniffer](https://github.com/squizlabs/PHP_CodeSniffer)

```
<?php
```

```
echo 'Hello, world!';
```

## Parsing PHP Tokens: the Abstract Syntax Tree

So far we've seen that the lexer cuts a string of PHP code into chunks of characters that belong together. These chunks are called *tokens*. Most of the tokens have a type and a value.

A tokenizer only looks at a string of code and tries to recognize its potentially meaningful substrings. However, it can't decide if the code itself is valid PHP code, or if it can be executed. As an example, the following code can be tokenized successfully:

```
<?php
```

```
'meaningless code' echo;
```

The list of tokens for this is:

| Line | Token                      | Value              |
|------|----------------------------|--------------------|
| 1    | T_OPEN_TAG                 | <?php\n            |
| 2    | T_WHITESPACE               | \n                 |
| 3    | T_CONSTANT_ENCAPSED_STRING | 'meaningless code' |
| 3    | T_WHITESPACE               |                    |
| 3    | T_ECHO                     | echo               |
| 3    | ;                          | ;                  |
| 3    | T_WHITESPACE               | \n                 |

So tokenizing isn't enough. After lexing, the next step in the compilation process is *parsing*. PHP's internal parser iterates over the flat list of tokens and groups them into meaningful, coherent blocks. While doing so it will report any issue in the form of a Parse error. For example, when you try to run the previous example of “meaningless code”, you'll see:



```
PHP Parse error:  syntax error, unexpected token "echo" in  
invalid_code.php on line 3
```

This gives us a clue about how the parser works. While looping over the tokens it will remember the context it's in, and it will look back and forth in the list of tokens to find out if everything is still going as expected. In the example of 'meaningless code' `echo;` the parser is in the context of parsing an *expression* (the string 'meaningless code'), but then suddenly encounters an `echo` token, which represents a *statement*. You can't have an `echo` statement inside an expression; only the other way around: the parser won't have any problem processing the code: `echo 'meaningful code';` In many other situations the PHP parser will warn you when the code does something weird or meaningless, like declaring class properties in the body of a method.

For tokenizing we were able to use PHP's built-in static method `PhpToken::tokenize()`. For parsing, we can rely on two tools:

1. The [PHP extension ast](https://github.com/nikic/php-ast)<sup>14</sup>
2. The [PHP-Parser library](https://github.com/nikic/PHP-Parser)<sup>15</sup>

Since Rector uses the PHP-Parser library we'll use it in this book as well. You could say that Rector is itself an extension of the PHP-Parser library. Rector parses the PHP files in your project using PHP-Parser and then lets you manipulate parts of the *Abstract Syntax Tree* (AST). This means that when you understand PHP-Parser, the AST it produces, and the node types it supports, you'll be able to more effectively write automated refactorings. Which is why this first chapter starts with a demonstration of PHP-Parser.

You can install the library with composer:

```
composer require --dev nikic/php-parser
```

Let's use `vendor/bin/php-parse` to parse our initial code snippet, which looks like this:

---

<sup>14</sup><https://github.com/nikic/php-ast>

<sup>15</sup><https://github.com/nikic/PHP-Parser>

```
<?php
echo 'Hello, world!';
```

You can either instantiate a `PhpParser\Parser` object (using `PhpParser\ParserFactory`) to parse a string of PHP code, or you can use the bundled command-line utility to print the AST of a file:

```
vendor/bin/php-parse hello-world.php
```

The output in this case will be:

```
array(
  0: Stmt_Echo(
    exprs: array(
      0: Scalar_String(
        value: Hello, world!
      )
    )
  )
)
```

The result of parsing the script is an array of *statements*. In this example, there is only one statement in the file; an echo statement (`Stmt_Echo`). Since you could echo multiple things, separated by a comma (e.g. `echo 'a', 'b';`), the echo statement node supports a number of *subnodes* called “exprs”. In our case we only have one expression that we want to echo: the string (`Scalar_String`) `'Hello, world!'`.

Each of the nodes in the “Node dump” output corresponds to a class from the PHP-Parser library. The `Stmt_Echo` node is an object of type `PhpParser\Node\Stmt\Echo_`. `Scalar_String` is of type `PhpParser\Node\Scalar\String_`. When working with Rector it’s going to be more useful to add the `--var-dump` option when running the `vendor/bin/php-parse` script because it prints the exact hierarchy of node objects, including the class names:

```
array(1) {
  [0]=>
  object(PhpParser\Node\Stmt\Echo_)#1157 (2) {
    ["exprs"]=>
    array(1) {
      [0]=>
      object(PhpParser\Node\Scalar\String_)#1156 (2) {
        ["value"]=>
        string(13) "Hello, world!"
        ["attributes":protected]=>
        array(5) {
          ["startLine"]=>
          int(2)
          ["startFilePos"]=>
          int(11)
          ["endLine"]=>
          int(2)
          ["endFilePos"]=>
          int(25)
          ["kind"]=>
          int(1)
        }
      }
    }
    ["attributes":protected]=>
    array(4) {
      ["startLine"]=>
      int(2)
      ["startFilePos"]=>
      int(6)
      ["endLine"]=>
      int(2)
      ["endFilePos"]=>
      int(26)
    }
  }
}
```

```
}
```

The expression nodes for the echo statement are kept as an array of nodes inside its `exprs` property. There is also an `attributes` property. All nodes have them, and they can be used to store metadata about the node. PHP-Parser already adds attributes related to the position of the node in the original code (line and column). We are free to add more attributes to a node using its `setAttribute()` method. In fact, Rector adds many useful attributes itself, as we'll see in the next chapter.

It's often useful to look up the actual node classes in your IDE. As an example, here is what the `Echo_` class looks like:

```
<?php

declare(strict_types=1);

namespace PhpParser\Node\Stmt;

use PhpParser\Node;

class Echo_ extends Node\Stmt
{
    /**
     * @var Node\Expr[] Expressions
     */
    public $exprs;

    /**
     * Constructs an echo node.
     *
     * @param Node\Expr[] $exprs      Expressions
     * @param array        $attributes Additional attributes
     */
    public function __construct(array $exprs, array $attributes = [])
    {
        $this->attributes = $attributes;
    }
}
```

```
        $this->exprs = $exprs;
    }

    public function getSubNodeNames(): array
    {
        return ['exprs'];
    }

    public function getType(): string
    {
        return 'Stmt_Echo';
    }
}
```

There's nothing really special about these node classes; we don't need the parser to create them. Instead, we can recreate an AST manually by instantiating these objects ourselves. For instance, this is how we can create the AST that the parser created for our `hello-world.php` script:

```
use PhpParser\Node\Stmt\Echo_;
use PhpParser\Node\Scalar\String_;

return new Echo_([new String_('Hello, world!')]);
```



If you often use `vendor/bin/php-parse --var-dump` to find out how to recreate part of an AST manually, you will be better off using the [php-parser-instantiation-printer](#)<sup>16</sup>. This tool uses PHP-Parser to create the AST of a PHP file and then produces the node instantiation code for you. Install it with `composer` and run it on the command-line:

```
composer require --dev matthiasnoback/php-parser-instantiation-printer

vendor/bin/print-node-instantiation-code hello-world.php
```

This will produce the following output:

```
<?php

return new PhpParser\Node\Stmt\Echo_( [
    new PhpParser\Node\Scalar\String_( 'Hello, world!' )
]);
```

Paste the result in your project, import the classes, and you're done.

## Converting the AST Back to PHP Code

Summarizing what we've seen so far: you can parse PHP code using the PHP-Parser library and the result is a tree of node objects: the AST.

PHP itself would traverse the AST nodes and generate machine code based on it. In the context of automated refactorings, we don't want to do that. Instead, we'd like to make changes to the code by manipulating the AST and then copy those changes back to the original PHP file. PHP-Parser offers a so-called “pretty printer” for this. It takes an array of nodes and returns a string which contains the PHP code for these nodes. The following should be true about the pretty printer. For any snippet of PHP code:

*Given you've parsed this snippet, which results in an AST  
And you haven't modified this AST*

---

<sup>16</sup><https://github.com/matthiasnoback/php-parser-instantiation-printer>

When you use the pretty printer to print PHP code for this AST  
Then the resulting PHP code should be the same as the original snippet of  
PHP code you parsed.

The following is a *contract test* to prove that the previous statements are true:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\ConvertingBackToPhpCode;

use PhpParser\ParserFactory;
use PhpParser\PrettyPrinter\Standard;
use PHPUnit\Framework\TestCase;
use function Safe\file_get_contents;

final class ContractTest extends TestCase
{
    public function testNodesCanBePrintedAsCode(): void
    {
        $parserFactory = new ParserFactory();
        $parser = $parserFactory->create(ParserFactory::PREFER_PHP7);

        $originalCode = rtrim(
            file_get_contents(__DIR__ . '/hello-world.php'),
            "\n"
        );

        $statements = $parser->parse($originalCode);
        assert(is_array($statements));

        $printer = new Standard();
        $printedCode = "<?php\n" . $printer->prettyPrint($statements);

        self::assertSame($originalCode, $printedCode);
    }
}
```

```
}  
}
```

Note that we have to do some extra work to make the Standard printer follow the described contract. Out-of-the-box, the printer doesn't include the PHP open tag `<?php`, so we'll have to add it manually. We also trim the trailing newline from the original file because the printer doesn't add it either. When you use Rector you don't have to worry about these issues, because Rector ships with its own improved version of the printer that takes care of details like this.

## Manipulating the AST

The true power of parsing code and printing the nodes back to code gets unleashed once we start *modifying* the AST between parsing and printing. If we do this, we'll end up with modified PHP code. Since we no longer use a naive text-based find/replace mechanism, but are manipulating the AST according to its own rules, we can't produce meaningless or invalid code.

Out of the box, PHP-Parser already has everything we need for modifying an AST. The tree itself is *mutable*, and all of its nodes are *mutable* too. This means we can modify and replace any node in the tree, and we can even add new nodes to it. This turns out to be a very powerful ability.

As an example, this is how we can change `echo 'Hello, world!'` into `echo 'I am a book'`. We only have to replace the first expression node inside the first statement (which is the echo statement). With the pretty printer we can print the nodes back to a string of PHP code and store it in the original file:



```
$parserFactory = new ParserFactory();
$parser = $parserFactory->create(
    ParserFactory::PREFER_PHP7
);

// Load and parse the original file
$originalFile = __DIR__ . '/hello-world.php';
$originalCode = file_get_contents($originalFile);

$statements = $parser->parse($originalCode);
assert(is_array($statements));

/*
 * $statements = [
 *     new Echo_(
 *         [
 *             new String_('Hello, world!')
 *         ]
 *     )
 * ];
 */

// Replace the first expression of the first statement:
$echoStatement = $statements[0];
assert($echoStatement instanceof Echo_);

$echoStatement->exprs[0] = new String_('I am a book');

// Print the modified AST back to the original file
$printer = new Standard();
file_put_contents(
    $originalFile,
    "<?php\n" . $printer->prettyPrint($statements)
);
```

## Node Visitors

Using numerical array indices isn't the most convenient way to target a specific node in the AST. It also isn't very flexible. If we'd add a `declare(strict_types=1);` statement to the original PHP file (like we should) the previous code would already stop working, since the statement with index 0 would now be the `declare` statement. Once we start creating our own automated refactorings for Rector we want them to work with any PHP file, not just files for which we already know the exact structure of the AST.

Instead of targeting nodes at specific *places* in the tree, we should be targeting nodes of specific *types*, regardless of their place in the tree. This can be accomplished using so-called *node visitors*. PHP-Parser comes with a node traverser which traverses the AST and while doing so will invoke all of the registered node visitors. The traverser starts with the first statement, then iterates over all the subnodes of that statement, and all the subnodes of those subnodes recursively. A node visitor can be notified upon “entering” a node or upon “leaving” the node, that is, when all of the subnodes of a node have been visited.

We can create our own node visitors by implementing the `NodeVisitor` interface, but in most cases we can extend from the `NodeVisitorAbstract` so we don't have to implement all its methods. Let's start with a simple node visitor that demonstrates the order in which the traverser calls its various methods:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\NodeVisitors;

use PhpParser\Node;
use PhpParser\NodeVisitorAbstract;

final class EchoCallsToNodeVisitor extends NodeVisitorAbstract
{
    public function beforeTraverse(array $nodes)
```

```
{
    echo "Before traverse\n";

    return null;
}

public function enterNode(Node $node)
{
    echo 'Enter node: ' . $node->getType() . "\n";

    return null;
}

public function leaveNode(Node $node)
{
    echo 'Leave node: ' . $node->getType() . "\n";

    return null;
}

public function afterTraverse(array $nodes)
{
    echo "After traverse\n";

    return null;
}
}
```

After using the parser to get the list of statements from our `hello-world.php` script, we have to instantiate a `NodeTraverser` and add the `EchoCallsToNodeVisitor` as a visitor. Then we call `traverse()`:

```
$traverser = new NodeTraverser();

$nodeVisitor = new EchoCallsToNodeVisitor();
$traverser->addVisitor($nodeVisitor);

$traverser->traverse($statements);
```

The output shows the following:

```
Before traverse
Enter node: Stmt_Echo
Enter node: Scalar_String
Leave node: Scalar_String
Leave node: Stmt_Echo
After traverse
```

The node traverser enters the Echo node, then enters the String node, then leaves it, and since it has processed all the subnodes of the Echo node, it also leaves the Echo node.

A node visitor can be *stateful*, like in the following example where we count the number of echo statements in the given PHP code. We use the `beforeTraverse()` method to reset the counter to 0 in case the traverser is reused:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\NodeVisitors;

use PhpParser\Node;
use PhpParser\Node\Stmt\Echo_;
use PhpParser\NodeVisitorAbstract;

final class EchoCounterNodeVisitor extends NodeVisitorAbstract
{
```

```
private int $numberOfEchos;

public function beforeTraverse(array $nodes)
{
    $this->numberOfEchos = 0;

    return null;
}

/**
 * @return Node|int|null
 */
public function enterNode(Node $node)
{
    if ($node instanceof Echo_) {
        $this->numberOfEchos++;
    }

    return null;
}

public function getNumberOfEchos(): int
{
    return $this->numberOfEchos;
}
}
```

This is how you use the `EchoCounterNodeVisitor` to count the echos in a given piece of code.

```
$parserFactory = new ParserFactory();
$parser = $parserFactory->create(ParserFactory::PREFER_PHP7);

$originalCode = <<<'CODE_SAMPLE'
<?php

echo 'Hello, world!';
echo 'Foo, bar!';
CODE_SAMPLE;

// Parse the original code
$statements = $parser->parse($originalCode);
assert(is_array($statements));

// Traverse all the nodes with EchoCounterNodeVisitor enabled
$traverser = new NodeTraverser();

$echoCounterNodeVisitor = new EchoCounterNodeVisitor();
$traverser->addVisitor($echoCounterNodeVisitor);

$traverser->traverse($statements);

$numberOfEchoes = $echoCounterNodeVisitor->getNumberOfEchos();

// Result: 2
```

Node visitors like the `EchoCounterNodeVisitor` can be very useful for the purpose of *static analysis*, i.e. collecting statistics or code problems without actually executing the code. Popular projects that rely on PHP-Parser, its AST, and node visitors are:

- [PHPStan](https://phpstan.org/)<sup>17</sup>, which can produce a list of code issues mostly by inspecting the nodes in the AST (e.g. “Method [...] has no return type”).
- The [sebastianbergmann/complexity](https://github.com/sebastianbergmann/complexity)<sup>18</sup> package which is part of PHPUnit’s code coverage tool. It collects code metrics using node visitors (e.g. cyclomatic

---

<sup>17</sup><https://phpstan.org/>

<sup>18</sup><https://github.com/sebastianbergmann/complexity>

complexity, number of executable lines, etc.).

## Manipulating the AST with a Node Visitor

In the context of automated refactorings we can use node visitors to target specific node types and replace or modify them on-the-fly. You can tell the node traverser to replace a node by returning a new node from the `enterNode()` method. First, you need to verify that you're targeting the correct node. This can be accomplished by adding a few *early return* clauses to the `enterNode()` method. For example, if we only want to modify echo statements that echo the string 'Hello, world!' we should start our `enterNode()` method as follows:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\ManipulatingTheAst\Version1;

use PhpParser\Node;
use PhpParser\Node\Scalar\String_;
use PhpParser\Node\Stmt\Echo_;
use PhpParser\NodeVisitorAbstract;

final class ReplaceHelloWorldNodeVisitor extends NodeVisitorAbstract
{
    /**
     * @return Node|int|null
     */
    public function enterNode(Node $node)
    {
        if (! $node instanceof Echo_) {
            // Ignore nodes that are not `echo` statements
            return null;
        }
    }
}
```

```

    if (count($node->exprs) !== 1) {
        /*
         * Ignore `echo` statements that echo nothing,
         * or more than one thing
         */
        return null;
    }

    if (! $node->exprs[0] instanceof String_) {
        // Ignore `echo` statements that don't echo a string
        return null;
    }

    if ($node->exprs[0]->value !== 'Hello, world!') {
        // This `echo` statement doesn't echo 'Hello, world!'
        return null;
    }

    // We know that this `echo` statement does echo `Hello, world!`

    // TODO replace the string

    return null;
}
}

```

When we've reached the end of the `enterNode()` method we can be certain: we're dealing with an `Echo_` node that contains a `String_` expression, the value of which is `Hello, world!`.



When working with Rector to create your own automated refactorings, filtering out irrelevant nodes is about half of the work. In the chapter about [test-driven rule development](#) we'll see how you can add test cases to prove that your node visitor doesn't accidentally target the wrong nodes.

Now it's time to replace the string with something else, e.g. I am a book:



```

        // We know that this `echo` statement does echo `Hello, world!`

        return new Echo_([new String_('I am a book')]);
    }
}

```

We can finally use the visitor to modify code. Just like before we need to register it as a visitor on the NodeTraverser object. Since we're modifying the AST, we need to capture the return value of the `traverse()` method. When we print the modified AST back to PHP code, the result should be `echo 'I am a book';`.

```

$originalCode = <<<'CODE_SAMPLE'
<?php

echo 'Hello, world!';
CODE_SAMPLE;

$statements = $parser->parse($originalCode);
assert(is_array($statements));

$traverser = new NodeTraverser();
$traverser->addVisitor(new ReplaceHelloWorldNodeVisitor());

$modifiedStatements = $traverser->traverse($statements);

$expectedCode = <<<'CODE_SAMPLE'
echo 'I am a book';
CODE_SAMPLE;

self::assertSame(
    $expectedCode,
    (new Standard())->prettyPrint($modifiedStatements)
);

```

When creating your own node visitors you'll find that there are often several possible options for the implementation. For our simple example of replacing `echo 'Hello, world!'` with `echo 'I am a book'` we have made some decisions already:

- We're replacing the entire `Echo_` node, which causes its attributes to be dropped (e.g. the position of the node in the original code).
- We ignore `echo 'Hello, world!', 'And something else';` even though we could easily replace the string here too.
- Our visitor isn't smart enough to deal with `$message = 'Hello, world!';` followed by `echo $message;`.

When writing your own node visitor it'll be useful to create a list of all these edge cases or unsupported cases. You can then decide if you want to modify your visitor to deal with those cases too. Either way, it'll be useful to provide test cases for all of them, so it's clear what is and isn't supported by your visitor. When you write node visitors for use with Rector you'll often need to at least generalize them a bit. As an example of a generalization: in the case of our `ReplaceHelloWorldNodeVisitor` the least we can do is make the replacement strings configurable via constructor arguments:

```
<?php
```

```
declare(strict_types=1);
```

```
namespace ProgrammaticallyModifyingPhpCode\ManipulatingTheAst\Version3;
```

```
use PhpParser\Node;
```

```
use PhpParser\Node\Scalar\String_;
```

```
use PhpParser\Node\Stmt\Echo_;
```

```
use PhpParser\NodeVisitorAbstract;
```

```
final class ReplaceEchoedStringNodeVisitor extends NodeVisitorAbstract
{
```

```
    public function __construct(
        private string $originalString,
        private string $replacementString
    ) {
    }
}
```

```
/**
```

```

    * @return Node|int|null
    */
    public function enterNode(Node $node)
    {
        if (! $node instanceof Echo_) {
            return null;
        }

        // ...

        $echoedExpr = $node->exprs[0];

        /** @var String_ $echoedExpr */
        if ($echoedExpr->value !== $this->originalString) {
            // This `echo` statement doesn't echo 'Hello, world!'
            return null;
        }

        // ...

        return new Echo_([new String_($this->replacementString)]);
    }
}

```

## Built-in Node Visitors

PHP-Parser comes with a number of node visitors that solve some common issues with node traversing. We'll only look at the visitors that are also used by Rector.

## Connecting Nodes

When building up the AST, PHP-Parser creates nodes, adds subnodes to them, and so on. When a visitor enters a node, it has access to the current node and any of the subnodes. If you need to know more about the context of the node, you'd want

to go up the tree to find the parent node of the current node, and so on, until you find the node you're looking for. Normally there is no two-way connection between parent and child nodes, but if you need this connection, you can add [the NodeConnectingVisitor](#)<sup>19</sup> to the NodeTraverser. This will add 'parent' and 'next' attributes to each node, which enables you to find the parent node, as well as the previous and next nodes (the siblings).

Let's use the NodeConnectingVisitor on a piece of code that contains a class and a method:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\BuiltInNodeVisitors;

final class TheClass
{
    public function aMethod(): void
    {
    }
}
```

Given that we've loaded this code into the \$code variable, we then:

- Parse the code
- Set up a node traverser with the NodeConnectingVisitor enabled
- Let it traverse the nodes

Once the nodes have been connected, we can traverse the nodes again and echo the class of each method node. We use a custom node visitor for this:

---

<sup>19</sup><https://github.com/nikic/PHP-Parser/blob/master/lib/PhpParser/NodeVisitor/NodeConnectingVisitor.php>

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\BuiltInNodeVisitors;

use PhpParser\Node;
use PhpParser\Node\Stmt\Class_;
use PhpParser\Node\Stmt\ClassMethod;
use PhpParser\NodeVisitorAbstract;

final class EchoMethodClassNodeVisitor extends NodeVisitorAbstract
{
    /**
     * @return Node|int|null
     */
    public function enterNode(Node $node)
    {
        if (! $node instanceof ClassMethod) {
            return null;
        }

        $parentNode = $node->getAttribute('parent');
        if (! $parentNode instanceof Class_) {
            return null;
        }

        echo sprintf(
            "Method %s() belongs to class %s\n",
            $node->name,
            $parentNode->name
        );

        return null;
    }
}
```

We start with a fresh node traverser:

```
$statements = $parser->parse($code);
assert(is_array($statements));

// First round: connect the nodes
$traverser = new NodeTraverser();
$traverser->addVisitor(new NodeConnectingVisitor());
$traverser->traverse($statements);

// Second round: show some information
$traverser = new NodeTraverser();
$traverser->addVisitor(new EchoMethodClassNodeVisitor());
$traverser->traverse($statements);
```

As expected, the output is:

Method aMethod() belongs to class TheClass



## Verify Your Assumptions

In the previous example we expect the parent node of a `ClassMethod` node to always be a `Class_` node. In everyday programming situations we'd verify such an assumption using an assertion (e.g. `assert($parent instanceof Class_);`) or by throwing an exception. When working with node visitors, and later with Rector rules, it's better if your code doesn't throw exceptions or trigger errors, because that will stop the execution of the node traverser, and Rector itself. A better way (as shown in this example) is to codify your assumption using an *early return*.

One day you'll try to use the `EchoMethodClassNodeVisitor` on a trait or an interface and when you don't see any output, you'll learn that a `ClassMethod` node can also be the child of a `Class_` or `Interface_` node. Then you can either let go of the early return, or accommodate your code to deal with these formerly unknown situations.

## Resolving Fully-qualified Names

Consider the `Bar` class that lives in the `Foo` namespace:

```
<?php

namespace Foo;

class Bar
{
}
```

If we run `vendor/bin/php-parse --var-dump` on this, we'll get the following nodes:

```
array(1) {
  [0] => object(PhpParser\Node\Stmt\Namespace_) {
    ["name"] => object(PhpParser\Node\Name) {
      ["parts"] => array(1) {
        [0] => "Foo"
      }
    }
    ["stmts"] => array(1) {
      [0] => object(PhpParser\Node\Stmt\Class_) {
        ["name"] => object(PhpParser\Node\Identifier) {
          ["name"] => "Bar"
        }
        ["stmts"] => array(0)
      }
    }
  }
}
```

We see that the name of the `Class_` node only mentions `Bar`, not `Foo\Bar`. When working with `Class_` nodes, it will be useful to know the name of the class itself (`Bar`), but also to know the *fully-qualified class name*, that includes the namespace (`Foo\Bar`).

PHP-Parser comes with a `NameResolver` [visitor](#)<sup>20</sup> that can figure out the fully-qualified class names (FQCNs) for you. The following example shows how to use it:

```
$statements = $parser->parse($source);
assert(is_array($statements));

$traverser = new NodeTraverser();
$traverser->addVisitor(new NameResolver());
$traverser->traverse($statements);
```

The `NameResolver` adds a `namespacedName` property to several node types, including class, interface, and trait declaration nodes. As an example, you can find out the FQCN of a class node by accessing its `namespacedName` property:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\BuiltInNodeVisitors;

use PhpParser\Node;
use PhpParser\Node\Stmt\Class_;
use PhpParser\NodeVisitorAbstract;

final class EchoNamespacedNameNodeVisitor extends NodeVisitorAbstract
{
    /**
     * @return Node|int|null
     */
    public function enterNode(Node $node)
    {
        if (! $node instanceof Class_) {
            return null;
        }
    }
}
```

---

<sup>20</sup>[https://github.com/nikic/PHP-Parser/blob/master/doc/component/Name\\_resolution.markdown](https://github.com/nikic/PHP-Parser/blob/master/doc/component/Name_resolution.markdown)



```
    }

    echo sprintf(
        "The namespaced name of class %s is %s\n",
        $node->name,
        $node->namespacedName
    );

    return null;
}
}
```



Note: Rector adds these 2 node visitor out-of-the-box. When we work with AST the nodes are already connected and with fully qualified names available.

## Finding Nodes

When you're analyzing or manipulating nodes in an AST you'll often need to find nearby nodes. For instance, if you'd like to find out if an object method could be turned into a static method, one thing you should check for is so-called `PropertyFetch` nodes. If any of those nodes tries to fetch an object property, e.g. `$this->property`, then it can't become a static method. As an example, in the following class `anObjectMethod()` couldn't be made a static method, because it relies on the instance property `$foo`. The `methodPotentiallyAStaticMethod()` could potentially be made static, because it doesn't rely on instance properties:

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\BuiltInNodeVisitors;

final class SomeClass
{
    private string $foo = 'bar';

    public function anObjectMethod(): string
    {
        return $this->foo;
    }

    public function potentiallyAStaticMethod(): string
    {
        return 'baz';
    }
}
```

Let's create a visitor that inspects every `ClassMethod` node and looks for `PropertyFetch` nodes inside the method.

In the `enterNode()` method we should look at each of the `ClassMethod`'s statements and find out if they are of type `PropertyFetch`. But each node might have subnodes, and those may also be `PropertyFetch` nodes. So we have to do a recursive search. Recursively traversing a set of nodes is something every `NodeVisitor` can do. If you want to filter specific nodes while traversing, you can use PHP-Parser's `NodeFinder` utility class. You can provide it with your own callback function.

Upon entering a node we check if it's a `ClassMethod` node. If it is, we'll use `NodeFinder::find()` to select only the nodes that access an object property, and print a message on the screen

```
<?php

declare(strict_types=1);

namespace ProgrammaticallyModifyingPhpCode\BuiltInNodeVisitors;

use PhpParser\Node;
use PhpParser\NodeFinder;
use PhpParser\Node\Expr\PropertyFetch;
use PhpParser\Node\Expr\Variable;
use PhpParser\Node\Stmt\ClassMethod;
use PhpParser\NodeVisitorAbstract;

final class CanMethodBeStaticNodeVisitor extends NodeVisitorAbstract
{
    public function __construct(
        private NodeFinder $nodeFinder
    ) {
    }

    /**
     * @return Node|int|null
     */
    public function enterNode(Node $node)
    {
        if (! $node instanceof ClassMethod) {
            return null;
        }

        $propertyFetchesFromThis = $this->nodeFinder->find(
            $node,
            fn (Node $candidate) => $candidate instanceof PropertyFetch
                && $candidate->var instanceof Variable
                && $candidate->var->name === 'this'
        );
    }
}
```

```

        if (count($propertyFetchesFromThis) === 0) {
            echo sprintf(
                "Method %s() is a candidate for becoming static\n",
                $node->name
            );
        } else {
            echo sprintf(
                "Method %s() needs access to a class instance\n",
                $node->name
            );
        }

        return null;
    }
}

```

The following example shows how to use the `CanMethodBeStaticNodeVisitor`. We first parse the PHP code containing the class we just saw. Then we traverse it using our own visitor:

```

$statements = $parser->parse($originalCode);
assert(is_array($statements));

$traverser = new NodeTraverser();
$traverser->addVisitor(
    new CanMethodBeStaticNodeVisitor(new NodeFinder())
);

$traverser->traverse($statements);

```

As expected, the output looks like this:

```

Method anObjectMethod() needs access to a class instance
Method potentiallyAStaticMethod() is a candidate for becoming static

```

Note that echo-ing things from a visitor isn't something you'd normally do. In practice, you'll either collect statistics and only later print them on the screen, or manipulate nodes. We'll do the latter in the next chapter when we create our first Rector rule.

## Summary

When you want to programmatically modify PHP code, the safest way to do this is to:

1. Parse the PHP code using PHP-Parser, which produces a set of nodes called the Abstract Syntax Tree.
2. Traverse the nodes in the tree with a node traverser and a node visitor, which will be notified about every node in the tree.

A node visitor can change existing nodes by returning a new or modified node from its `enterNode()` method.

PHP-Parser ships with a number of reusable node visitors, like the `NameResolver` which resolves fully-qualified/namespaced names, and the `NodeConnectingVisitor` which establishes links between parent and child nodes, as well as sibling nodes.

# PHP Tools in the Game

## This Chapter Covers:

- A history of the PHP tools that enable Rector
- The different levels at which PHP tools operate
- How to combine different tools in your development workflow

## Introduction

Before we start to explore Rector internals and its powerful features, we'll take a broader look at the wider range of tools that can analyze and change your code. Why do we talk about other tools in a book about Rector? Well, have you heard about the [Unix philosophy](#)<sup>21</sup>? It's nothing about free software or independence, but rather about co-dependence:

Make each program do one thing well. To do a new job, build afresh rather than complicate old programs by adding new “features”.

Rector is just one of the tools in the chain. It's the most powerful one, but without its friends it would be an utterly useless empty shell, like our body would be without lungs.

## Working Together with Giants

Rector builds on static analysis. It needs to understand types to decide if it's the class that should be renamed or just an unknown mixed variable and skip it.

---

<sup>21</sup>[https://en.wikipedia.org/wiki/Unix\\_philosophy](https://en.wikipedia.org/wiki/Unix_philosophy)

Coding standards tidy up after Rector refactors it, so the code looks exactly like all the other code in your codebase.

In the previous chapter, you've learned the difference between PHP tokens and Abstract Syntax Tree. Now we'll use this knowledge to understand how other tools cooperate with Rector. We'll also tell you what the most effective order to plug them into your project is.

## 2007 to 2021 Timeline

In 2021 it feels like we have all the tools available since the time of Composer. In reality, it took 13 years to get where we are. It's like raising children from birth to puberty. To give some kind of perspective, here is how the evolution went year by year:

| Year | Tools                                |
|------|--------------------------------------|
| 2007 | PHP_CodeSniffer                      |
| 2008 |                                      |
| 2009 |                                      |
| 2010 | PHP-Parser                           |
| 2011 |                                      |
| 2012 |                                      |
| 2013 |                                      |
| 2014 | PHP CS Fixer                         |
| 2015 |                                      |
| 2016 | Easy Coding Standard, PHPStan, Psalm |
| 2017 | Rector                               |
| 2018 |                                      |
| 2019 |                                      |
| 2020 |                                      |
| 2021 |                                      |

## The Primary Feature

The features of these tools slightly overlap. Several tools primarily focus on coding standards; how the code looks, and where the comma is located. Some of them also

have rules that help you with adding `void` return types to methods. It works the other way round too. The primary purpose of some tools is to upgrade your code from PHP 7 to 8, but they also help with formatting, so the code does not look bad after the upgrade.



Why do you buy a car? Probably to drive somewhere faster than with public transfer, when you need to get shopping done and ride out to beautiful nature. Does it mean it's missing a radio? Not at all, but we don't get cars only to play music. Considering this, let's focus on the primary purpose of these tools.

The following table provides a quick overview:

|                        | <b>Reports issues</b> | <b>Fixes issues</b> |
|------------------------|-----------------------|---------------------|
| <b>Coding standard</b> | PHP_CodeSniffer       | PHP-CS-Fixer, ECS   |
| <b>Logic</b>           | PHPStan, Psalm        | Rector              |

We'll go through the tools one by one to understand where exactly they fit in the process of refactoring, and we'll see how some of these tools combine reporting about code issues and automatically modifying the code to fix those issues.

In the end, we can separate these PHP tools into 4 categories:

1. Coding Standards
2. Static Analyzers
3. Instant Upgrades
4. Instant Downgrades

Let's look at one by one in detail and what tools do they include.

## 1. Coding Standard Tools

### PHP\_CodeSniffer

- Modifies code



- Is based on tokens

The first tool that made it to the mainstream of coding standard tools is [PHP\\_CodeSniffer](#)<sup>22</sup>, created around 2007 by Greg Sherwood (Australia), based on the [date of the oldest post I could find](#)<sup>23</sup>. Greg has been maintaining the repository for the last 11 years (at least). I don't know anyone else who would take care of their project for such a long time without stepping back, *much respect*.

## PHP CS Fixer

- Modifies code
- Is based on tokens

Reporting errors is helpful, but fixing them for you is even more helpful. More people got a similar need, and around [Symfony 2.0 times and 2011](#)<sup>24</sup> the next tool was born: [PHP CS Fixer](#)<sup>25</sup>.

[The first version](#)<sup>26</sup> was created by Fabien Potencier, author and founder of Symfony, and it used mostly regular expressions.

The decision to *fix everything* by default was a huge jump in the history of these tools. It was the first case of a tool that would be so bold to change your code for you. You had to believe it. You had to overcome the fear that it might modify code in the wrong way or even delete it. I mean, now we're used to it like we're used to cars, but at some point in history they were just riding bombs.

With [PHP CS Fixer 1.0 release in 2014](#)<sup>27</sup> the rising popularity of *automated fixes* was a big motivation for PHP\_CodeSniffer to add a similar feature to version 2: [code beautifier](#)<sup>28</sup>.

It also helped with another issue. The development of PHP\_CodeSniffer 2.8 and later almost froze to zero. I remember because I started working on [EasyCodingStandard](#)<sup>29</sup>

---

<sup>22</sup>[https://github.com/squizlabs/PHP\\_CodeSniffer](https://github.com/squizlabs/PHP_CodeSniffer)

<sup>23</sup><http://gregsherwood.blogspot.com/2006/12/if-not-test-first-then-test-really-soon.html>

<sup>24</sup><https://gist.github.com/fabpot/3f2555dce956accd4dd>

<sup>25</sup><https://github.com/FriendsOfPHP/PHP-CS-Fixer>

<sup>26</sup><https://gist.github.com/fabpot/3f2555dce956accd4dd>

<sup>27</sup><http://fabien.potencier.org/php-cs-fixer-finally-reaches-version-1-0.html>

<sup>28</sup>[https://github.com/squizlabs/PHP\\_CodeSniffer/wiki/Fixing-Errors-Automatically](https://github.com/squizlabs/PHP_CodeSniffer/wiki/Fixing-Errors-Automatically)

<sup>29</sup><https://github.com/symplify/easy-coding-standard>

right between PHP\_CodeSniffer 2.8 and 3.0 (depending on 3.0-dev), which took 14 uncertain months.

Both PHP CS Fixer and PHP\_CodeSniffer use `PhpToken::tokenize()`, which we used in the previous chapter as well. We won't go deeper in coding standards in here, but if you'd like to learn more about the internals of these tools see [“How PHP Coding Standard Tools Actually Work”](#)<sup>30</sup> post.

## EasyCodingStandard

- Modifies code
- Is based on tokens

I saw many projects that use both PHP CS Fixer and PHP\_CodeSniffer, yet very poorly because split attention divides the focus in the same ratio. Which is why I created [EasyCodingStandard](#)<sup>31</sup> with this mission: *to help new generations adopt coding standards with almost no effort*. EasyCodingStandard was born in 2016 in the Czech Republic.

## 2. Static Analyzers

### PHP-Parser

- Is based on the AST
- Can be extended to analyse and modify code

This package is barely known for code analysis, but it provides the technology that all the other tool build on: it can create an *Abstract Syntax Tree* (AST). It all started with a question on [StackOverflow](#)<sup>32</sup>: “Any decent PHP parser written in PHP?” Nikita Popov (nikic) answered himself with a [php-parser](#)<sup>33</sup> less than six months later.

All of the tools that remain to be discussed in this chapter use an *AST analyzer* on top of AST provided by PHP-Parser. An AST analyzer knows what type of object is in the `$object` variable:

---

<sup>30</sup><https://tomasvotruba.com/blog/2017/07/31/how-php-coding-standard-tools-actually-work/>

<sup>31</sup><https://github.com/symplify/easy-coding-standard>

<sup>32</sup><https://stackoverflow.com/questions/5586358/any-decent-php-parser-written-in-php>

<sup>33</sup><https://github.com/nikic/PHP-Parser>

```
class SomeObject
{
    public function exist()
    {
    }
}

// The analyzer remembers that `$object` is of type `SomeObject`
$object = new SomeObject;

// An analyzer warns you that the method `missing()`
// does not exist on `SomeObject`:
$object->missing();
```

## PHPStan

- Analyses code based on the AST

If you don't use any static analyzer, give [PHPStan](#)<sup>34</sup> by Ondřej Mirtes a try (created in 2016). I've made a [minimalist intro that will help you with the first steps](#)<sup>35</sup>. It's worth investing even a day or two to set it up because a tool like PHPStan will be in every developer's toolkit eventually, just like Composer or PHPUnit are.

## Psalm

- Analyses code based on the AST
- Modifies code

Created by Matthew Brown in 2016, [Psalm](#)<sup>36</sup> is a very interesting tool that was born to fight Vimeo code complexity. It was the first tool from the pack that started [fixing the code](#)<sup>37</sup> for you, even though only in unused variables, private properties and return and param types.

---

<sup>34</sup><https://github.com/phpstan/phpstan>

<sup>35</sup><https://tomasvotruba.com/blog/2017/01/28/why-i-switched-scrutinizer-for-phpstan-and-you-should-too/>

<sup>36</sup><https://github.com/vimeo/psalm>

<sup>37</sup>[https://psalm.dev/docs/manipulating\\_code/fixing/](https://psalm.dev/docs/manipulating_code/fixing/)

## 3. Instant Upgrade Tools

### Symfony-Upgrade-Fixer

- Is based on tokens
- Modifies code
- Is now deprecated

[Symfony-Upgrade-Fixer](#)<sup>38</sup> was created in 2015 and it's revolutionary. It wisely connects the token analysis of PHP CS Fixer with deprecations in Symfony 2 to 3. It's like PHP CS Fixer and [sensiolabs-de/deprecation-detector](#) are working together for you. However, its features are limited due to it relying on tokens.

### Rector

- Modifies code based on the AST

It's 2017 and Rector still had to wait many months to be born. What was missing? Well, Rector is built on top of `php-parser`, and as it modifies the code and prints it back, it needs to preserve whitespace. That's one of AST's drawbacks: it doesn't care about all the new lines and spaces that coding standard tools are concerned with.

They say "history repeats", but I never trusted that. Until I saw that Fabien Potencier had a similar need for preserving whitespace while manipulating ASTs when he created an issue for this in 2012 while making PHP CS Fixer: [Optionally add nodes for whitespace](#)<sup>39</sup>. More people wanted AST-based coding standards. `PHP-Parser 4.0-dev` already had this feature, so Rector ran on it from the very start, although we had to wait until [February 2018](#)<sup>40</sup> when it was finally released.

The last giant Rector builds on is the one for type analysis - `PHPStan`. Thanks to that, Rector doesn't have to reinvent the wheel and can focus on the refactoring part. And this concludes a brief history of tools that watch your code or modify it for you.

---

<sup>38</sup><https://github.com/umpirsky/Symfony-Upgrade-Fixer>

<sup>39</sup><https://github.com/nikic/PHP-Parser/issues/41>

<sup>40</sup><https://github.com/nikic/PHP-Parser/releases/tag/v4.0.0>

## 4. Instant Downgrade Tools



How can downgrades be useful in modern PHP? The downgrade concept is nothing new. Igor Wiedler coined [an idea of backporting syntax down from PHP 5.4 to PHP 5.3](#)<sup>41</sup> back in 2013. Downgrades are getting more attention since 2020 and will improve PHP packages release ecosystem by light years.

### PHP-Backporter

- Uses AST
- Is now deprecated
- Written by Nikita Popov, authors of php-parser

[PHP-Backporter](#)<sup>42</sup> was a first tool that automatically downgraded PHP code. Which version? 5.3 to 5.2

### Galapagos

- Uses AST
- Is now deprecated

The [galapagos](#)<sup>43</sup> packages was downgrading PHP 5.4 to PHP 5.3.

It was actually first package that used `nikic/php-parser` as a dependency to manipulate PHP code in formal direction.

### Transphpilea

- Uses AST
- Is now deprecated

[Transphpilea](#)<sup>44</sup> is a simple package that downgrades PHP 7 code to PHP 5.6.

---

<sup>41</sup><https://igor.io/2013/07/26/evolving-syntax.html>

<sup>42</sup><https://github.com/nikic/PHP-Backporter>

<sup>43</sup><https://github.com/endel/galapagos>

<sup>44</sup><https://github.com/jaytaph/Transphpilea>

## 7to5

- Uses AST
- Is now deprecated

[7to5](#)<sup>45</sup> a package by [Freek Van der Herten](#)<sup>46</sup> from Spatie. It's job was to downgrade PHP 7.0 to 5.6.

## Rector

- Uses AST

Rector is handling both upgrades and downgrades. Rector downgrades itself from [PHP 8 to 7.1](#)<sup>47</sup> and at time of writing it supports path from PHP 8 to PHP 5.6.

That's 4 categories we can put any PHP CLI tools in.

## When to Use Which Tool?

In the beginning we talked about co-dependency and an ideal order for these tools to run. We'll talk about it in more detail in the chapter about [Continuous Rectifying](#), but here's a brief summary. The perfect recipe is these three tools together:

- A coding standard tool
- A static analyzer
- Rector

---

<sup>45</sup><https://github.com/spatie/7to5>

<sup>46</sup><https://twitter.com/freemurze>

<sup>47</sup><https://getrector.org/blog/2021/03/22/rector-010-released-with-php71-support>

## Is Your Project Bare Without Any Tools?

Some projects don't have neither coding standards nor static analyzer. If we would run Rector on such a project, we might get burned very quickly and give up with "this legacy is way too hard for Rector" disappointment. Luckily, there is a safe step by step process to make Rector run even there.

If you don't have any tools yet, start with low hanging fruit. The easiest to setup is coding standard, because it only changes code design spots like spaces or brackets positions. Add your coding standard config, run it locally, fix found errors and create a pull-request. Once you get the code to at least the [PSR-12<sup>48</sup>](https://www.php-fig.org/psr/psr-12/) set we can move to a next tool.

The 2nd low hanging tool to setup is a static analyzer. Why do we run the static analyzer before we run Rector? That's because Rector works best if all type inference issues have been solved. In fact, Rector rules heavily rely on PHPStan's analyzer, so we first want to make sure that PHPStan is happy about our code. If your project contains a lot of legacy code, then this may be daunting. But the code base doesn't have to comply with PHPStan level max everywhere. Generally speaking, PHPStan level 3 or 4 should be sufficient to safely use Rector.

Do we have coding standards and static analyzer running our CI and passing? Now it's time to add Rector.

## Run Rector First, Then Polish with Coding Standards

Rector uses nikic/php-parser that leverages an abstract syntax tree (AST). An AST doesn't know about spaces and when written to a file it can produce poorly formatted code. That's why it's important to run coding standard after every Rector run, the same way you run coding standards after your every manual change of your code.

---

<sup>48</sup><https://www.php-fig.org/psr/psr-12/>

## Recommended Tools

At the time of writing you can choose between PHPStan or Psalm as your static analyzer. To avoid a discussion about which tool is best, use the tool that does the job right, and that you or your team knows well.

These are the tool we're the most familiar with:

1. A coding standard tool - [ECS](#)<sup>49</sup>
2. A static analyzer tool - [PHPStan](#)<sup>50</sup>
3. Rector

## Summary

In this chapter we've seen a number of tools that have been created over the years to help developers find issues with their code without even running that code. The two major areas these tools are concerned with: coding standards and logic issues. We saw that some of these tools only report issues, while others can fix issues for you.

There are 4 groups we divide these tools in - coding standard tools, static analysers, instant upgrade tools and instant downgrades tools.

Rector is a 4 years old, but the most recent addition to the tool chain, and it's built on top of PHP-Parser and PHPStan. This makes Rector quite unique: it modifies code, just like token-based coding standard tools do, but it's more powerful because it directly manipulates the AST and uses static analyzer to make better decisions while doing so.

---

<sup>49</sup><https://github.com/symplify/easy-coding-standard>

<sup>50</sup><https://github.com/phpstan/phpstan>



# Creating Your First Rector Rule

## This Chapter Covers:

- Manipulating an AST with Rector
- Creating a Rector rule for refactoring existing code
- Running a single rule

## Introduction

In the chapter [Programmatically modifying PHP code](#) we used PHP-Parser to load a single PHP file, parse it, and manipulate some of its nodes if they matched certain criteria. In this chapter we'll take the next step: using Rector to load *all* PHP files in a given project directory and modify the code according to certain *rules*.

## What's a Rector Rule?

A rule is an automated refactoring. It represents one particular thing that Rector could change for all the PHP files in your project at once. Here are several examples of rules that are already included in the Rector package:

- Rename method; e.g. call `newMethod()` instead of `oldMethod()` on `SomeExampleClass`
- Encapsed strings to `sprintf`; e.g. replace `return "Unsupported format { $format}";` with `return sprintf('Unsupported format %s', $format);`
- Finalize classes without children; if a class has no subclasses, make it `final`.

It's always smart to scan the [list of existing rules](#)<sup>51</sup> before you start implementing your own rule. Chances are that what you're looking for already exists, and in that case it's likely that the existing rule can deal with many edge cases you didn't think of yet.

## Creating a Custom Rule

In this chapter we'll create a rule that doesn't exist yet, and it's based on the following scenario:

In Symfony, the `Request` object has a `get($key)` method which is rather ambiguous. It will try to find a value by the given key in the request *attributes* (e.g. dynamic route parameters), the query string, or in case the request is a POST request: the request body. I know that in my project all the requests where we use `Request::get($key)` are POST requests, and I think it's better to explicitly use `$request->request->get($key)` in that case. I don't want to manually change all these calls, so I'd like to create a Rector rule that can do the refactoring automatically for me.

As with any Rector rule you'll create, the best way to start is to write an *example* which shows a piece of code before and after applying the rule. This gets you in the right mindset for implementing the rule itself. This is the *before* code:

```
<?php
```

```
declare(strict_types=1);
```

```
use Symfony\Component\HttpFoundation\JsonResponse;
```

```
use Symfony\Component\HttpFoundation\Request;
```

```
use Symfony\Component\HttpFoundation\Response;
```

```
final class UserController
```

```
{
```

```
    public function registerUser(Request $request): Response
```

```
    {
```

```
        $username = $request->get('username');
```

---

<sup>51</sup>[https://github.com/rectorphp/rector/blob/main/docs/rector\\_rules\\_overview.md](https://github.com/rectorphp/rector/blob/main/docs/rector_rules_overview.md)

```
        return new JsonResponse('...');
    }
}
```

And this is the *after* code:

```
<?php

declare(strict_types=1);

use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;

final class UserController
{
    public function registerUser(Request $request): Response
    {
        $username = $request->request->get('username');

        return new JsonResponse('...');
    }
}
```

In essence, a *Rector rule* is a *PHP-Parser node visitor*, as you can see in the `PhpRectorInterface` which extends from `PhpParser\NodeVisitor`. It defines several methods that we have to implement ourselves:

```
<?php

declare(strict_types=1);

namespace Rector\Core\Contract\Rector;

use PhpParser\Node;

interface PhpRectorInterface extends \PhpParser\NodeVisitor,
\Rector\Core\Contract\Rector\RectorInterface
{
    /**
     * List of nodes this class checks, classes that implements
     * \PhpParser\Node See beautiful map of all nodes
     * https://github.com/rectorphp/php-parser-nodes-docs#node-overview
     *
     * @return array<class-string<Node>>
     */
    public function getNodeTypes(): array;

    /**
     * Process Node of matched type
     *
     * @return Node|Node[]|null
     */
    public function refactor(\PhpParser\Node $node);
}
```

There is one other interface method that RectorInterface inherits from its parent interface DocumentedRuleInterface:

```
<?php

declare(strict_types=1);

namespace Symplify\RuleDocGenerator\Contract;

use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

interface DocumentedRuleInterface
{
    public function getRuleDefinition(): RuleDefinition;
}
```

The `DocumentedRuleInterface` interface helps other developers understand what the rules does. We can also generate documentation from it.



## Generating Rector Rule Documentation

If you are interested in generating Markdown documentation for all the Rector rules in your project, install the [Rule Doc Generator](https://github.com/symplify/rule-doc-generator)<sup>52</sup> in your project:

```
composer require symplify/rule-doc-generator --dev
```

Then run:

```
vendor/bin/rule-doc-generator generate utils/ \
    --output-file docs/rules_overview.md
```

A nice example of documentation generated in this way is the [list of available Rector rules for Typo3](https://github.com/sabbelasichon/typo3-rector/blob/main/docs/all_rectors_overview.md)<sup>53</sup>.

We shouldn't implement just the `PhpRectorInterface`, because most of the work is done by `AbstractRector`. The `enterNode()` method of this class has all the logic needed to determine if the rule applies to the current AST node, and it knows what

<sup>52</sup><https://github.com/symplify/rule-doc-generator>

<sup>53</sup>[https://github.com/sabbelasichon/typo3-rector/blob/main/docs/all\\_rectors\\_overview.md](https://github.com/sabbelasichon/typo3-rector/blob/main/docs/all_rectors_overview.md)

to do in case the rule actually manipulates the current node. `AbstractRector` only needs three things from us:

1. A description of the rule.
2. A list of one or more node types that should be considered candidates for this refactoring.
3. How to actually perform the refactoring.



## Requirements

If you want to follow along with the next steps, you'll need the following things:

- An empty project
- PHP
- Composer
- Rector

Rector is installed as a Composer dependency:

```
composer require rector/rector --dev
```

See the project's [README](https://github.com/rectorphp/rector)<sup>54</sup> for more information.

## Extending AbstractRector

Let's create an implementation of `AbstractRector` and start building our first rule. A good name for the rule class would be `UseRequestRequestGetRector`. We'll create this class in our project in a directory called `utils/rector/`.

---

<sup>54</sup><https://github.com/rectorphp/rector>



## Where to Store Your Rule Classes?

You're actually free to place the rule classes anywhere in your project, in any namespace/directory. In this book we follow the naming convention suggested in [Rector's online documentation](https://github.com/rectorphp/rector/blob/main/docs/create_own_rule.md)<sup>55</sup>. Make sure that the rule classes are auto-loadable via Composer. When you use the namespace `Utils\Rector\Rector` for your rule classes and store them in `utils/rector/src/Rector`, then the "autoload-dev" section in your `composer.json` file should look like this:

```
{
    "require": {
        "rector/rector": "^0.11"
    },
    "autoload-dev": {
        "psr-4": {
            "Utils\\Rector\\": "utils/rector/src",
            "Utils\\Rector\\Tests\\": "utils/rector/tests"
        }
    }
}
```

After changing settings for auto-loading in `composer.json` you should always run:

```
composer dump-autoload
```

This regenerates Composer's class auto-loader so that it will be able to find all the classes in `vendor/` as well as in our own project, including the rule and helper classes we're going to create in the `Utils\Rector` namespace.

After creating the new `UseRequestRequestGetRector` class, the first thing we do is provide a default implementation for the required methods. It's the least we can do to make "the compiler" happy.

---

<sup>55</sup>[https://github.com/rectorphp/rector/blob/main/docs/create\\_own\\_rule.md](https://github.com/rectorphp/rector/blob/main/docs/create_own_rule.md)

utils/rector/src/Rector/UseRequestRequestGetRector.php

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Rector\Rector;
```

```
use PhpParser\Node;
```

```
use Rector\Core\Rector\AbstractRector;
```

```
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;
```

```
final class UseRequestRequestGetRector extends AbstractRector
```

```
{
```

```
    /**
```

```
     * @return array<class-string<Node>>
```

```
     */
```

```
    public function getNodeTypes(): array
```

```
    {
```

```
        return [];
```

```
    }
```

```
    public function refactor(Node $node): ?Node
```

```
    {
```

```
        return null;
```

```
    }
```

```
    public function getRuleDefinition(): RuleDefinition
```

```
    {
```

```
        return new RuleDefinition(
```

```
            'Use $request->request->get() instead of $request->get()',
```

```
            []
```

```
        );
```

```
    }
```

```
}
```

---



Let's fill in the details of the definition of this rule by providing a short description, and a code sample:

```
use Simplify\RuleDocGenerator\ValueObject\CodeSample\CodeSample;
use Simplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class UseRequestRequestGetRefactor extends AbstractRefactor
{
    public function getRuleDefinition(): RuleDefinition
    {
        return new RuleDefinition(
            'Use $request->request->get() instead of $request->get()',
            [
                new CodeSample(
                    <<<'CODE_SAMPLE'
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;

final class UserController
{
    public function registerUser(Request $request): Response
    {
        $username = $request->get('username');
    }
}
CODE_SAMPLE
,
                    <<<'CODE_SAMPLE'
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;

final class UserController
{
    public function registerUser(Request $request): Response
    {
        $username = $request->request->get('username');
```

```

    }
}
CODE_SAMPLE
    ),
    ]
    );
}

```

Although we only provide one before and after code sample here, `RuleDefinition` can deal with multiple code samples as well. In practice, multiple code samples are useful for rules that are configurable. They can show how code changes differently based on different configuration. We'll also add multiple code samples when creating tests for a rule (we'll discuss tests in the chapter [Test-driven rule development](#)).

## Finding the Right Node Class

The next method we have to implement is `getNodeTypes()`. This method works in a similar way as an event subscriber announcing the event types it wants to subscribe to. A Rector rule acts like a subscriber, but instead of events, Rector will “dispatch” AST nodes to the rule. Unlike a node visitor, a Rector rule isn't interested in any node type, so the rule declares a list of node types that it wants to subscribe to. These node types are the AST node classes that this rule considers to be candidates for refactoring.

In our case we're looking for calls to `$request->get()` because we want to change them to `$request->request->get()`. To find out the right node type that we should return here, we use the PHP-Parser command-line utility, just like we did before. First, create a minimal example and store it in a temporary file (be sure not to commit this file):

`before.php`

---

```
<?php
```

```
$request->get('username');
```

---

```
vendor/bin/php-parse before.php --var-dump
```



## Install PHP-Parser

If `php-parse` doesn't exist in your project, you have to install PHP-Parser first:

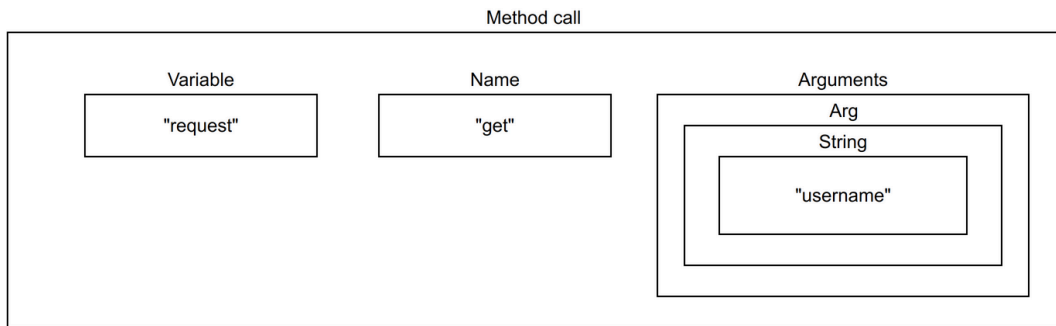
```
composer require nikic/php-parser --dev
```

Now try to run the previous command again.

The output (abbreviated) will be:

```
array(1) {
  [0] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\MethodCall) {
      ["var"] => object(PhpParser\Node\Expr\Variable) {
        ["name"] => "request"
      }
      ["name"] => object(PhpParser\Node\Identifier) {
        ["name"] => "get"
      }
      ["args"] => array(1) {
        [0] => object(PhpParser\Node\Arg) {
          ["name"] => null
          ["value"] => object(PhpParser\Node\Scalar\String_) {
            ["value"] => "username"
          }
        }
      }
    }
  }
}
```

This diagram depicts the nesting of the nodes more accurately:



If you're not used to working with the AST and node types, it may be hard to pick the right node type from this "mess". To make it a bit easier we prepared a [table of PHP code and its AST transcription examples](#)<sup>56</sup>. It takes a couple of weeks to start thinking in nodes, so don't be shy to use it everytime you create a new Rector rule. In general you can stick to the following advice: pick the type of the node you want to modify. The reason for this is that you can't modify parent nodes.

In our case, we want to refactor a method call, and replace it with a different method call, so intuitively it makes sense to choose the `MethodCall` node as a candidate for refactoring. We'll return the class name of this node type from the `getNodeTypes()` method:

```

use PhpParser\Node\Expr\MethodCall;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class UseRequestRequestGetRector extends AbstractRector
{
    /**
     * @return array<class-string<Node>>
     */
    public function getNodeTypes(): array
    {
        return [MethodCall::class];
    }
}
  
```

The last method we need to implement before we can run the rule is the `refactor()` method. This method receives a node that is an instance of one of the types returned

<sup>56</sup><https://github.com/rectorphp/php-parser-nodes-docs#node-overview>

by `getNodeTypes()`. Since the only type of node we listen for is `MethodCall`, we can be sure that the node we receive is actually an instance of `MethodCall`. To help your IDE with completion and PHPStan with static analysis, we'll document it with `@param MethodCall $node`.

We don't know exactly what we have to do inside the `refactor()` method in order to change `$request->get()` into `$request->request->get()` yet. For now, we'll only make a simple change to the node instead. This will prove to us that the `refactor()` method will in fact be invoked, and that it will be invoked for the correct node types:

```
use PhpParser\Node\Identifier;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class UseRequestRequestGetRector extends AbstractRector
{
    /**
     * @param MethodCall $node
     */
    public function refactor(Node $node): ?Node
    {
        $node->name = new Identifier('modified');

        return $node;
    }
}
```

The current version of the rule will change *all* method calls to call a different method, namely `modified()`. Let's find out what the result of running this rule will be.

## Running a Single Rule

You can configure which Rector rules to run on your project by creating a `rector.php` configuration file. It's a PHP file that allows you to manipulate the configuration of the Symfony service container that Rector uses internally. Any class implementing `PhpRectorInterface` that you register on the container will be used by Rector to refactor the code in your project.

If you don't have a `rector.php` configuration file in your project yet, you can run `vendor/bin/rector init` to let Rector generate one. You can also manually create a minimal version of this file and register only the rule you're currently working on. Make sure to configure a list of directories that Rector should process. Assuming that your project files are in `src/`, `rector.php` should look like this:

`rector.php`

---

```
<?php
```

```
declare(strict_types=1);
```

```
use Utils\Rector\Rector\UseRequestRequestGetRector;
```

```
use Rector\Core\Configuration\Option;
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $parameters = $containerConfigurator->parameters();
```

```
    $parameters->set(Option::PATHS, [__DIR__ . '/src']);
```

```
    $services = $containerConfigurator->services();
```

```
    $services->set(UseRequestRequestGetRector::class);
```

```
};
```

---

Once you've set up the `rector.php` configuration file, it's time to run our first rule on an existing project:

```
vendor/bin/rector process --dry-run
```

When you use the `process` command, Rector will:

1. load all the `.php` files in `src/`
2. parse the code to AST

3. give all the configured rules a chance to refactor the code (in our case, only the `UseRequestRequestGetRector` rule)

The `--dry-run` option ensures that Rector doesn't copy the modified code back to the original files. Rector will show us a “diff” of the changes it was going to make, like this:

1 file with changes

=====

1) app/Controller/UserController.php

```

----- begin diff -----
--- Original
+++ New
@@ @@
{
    public function registerUser(Request $request): Response
    {
-        $username = $request->get('username');
+        $username = $request->modified('username');

-        $hasher = $this->container->get('password_hasher');
+        $hasher = $this->container->modified('password_hasher');

        // ...

-        return $this->render('register.html.twig');
+        return $this->modified('register.html.twig');
    }
}
----- end diff -----

```

Applied rules:

- \* `UseRequestRequestGetRector`

```
[OK] 1 file would have changed (dry-run) by Rector
```



If you've set your new rule as a service, it could still happen that Rector says:

```
[ERROR] We could not find any Rector rules to run
```

This probably means that the rule class could not be auto-loaded by Composer. Fix the `autoload(-dev)` configuration in `composer.json` and run `composer dump-autoload` afterwards.

## Refactoring the Method Call Node

The output of the dry-run proves that the rule is invoked, and that it targets the right type of nodes. We still need to:

1. Ignore method calls that shouldn't be modified (like `$this->container->get()` and `$this->render()`).
2. Make the correct change (i.e. change `$request->get()` to `$request->request->get()`).

Starting with the first issue, our `refactor()` method should not change calls to methods that aren't called `get()`. The name of the method is inside the `name` property of the `MethodCall` node. But the `name` property can contain either an `Identifier` or an `Expr` node.





## How Do We Know That?

We look at the `MethodCall` node in `php-parser`<sup>57</sup>:

```
class MethodCall extends Expr
{
    /**
     * @var Expr Variable holding object
     */
    public $var;

    /**
     * @var Identifier|Expr Method name
     */
    public $name;
```

It's a good habit to look up node classes in the PHP-Parser library yourself to find out more about the internals.

We could deal manually with this ambiguity and extract a string from either of these node types, but it's better to use the utility function `isName()`. It's part of the `AbstractRector` base class, together with many other helpful functions.

If the method name isn't `get()`, we should return `null` from the `refactor()` method. This is telling Rector that we didn't modify the node.

```
/**
 * @param MethodCall $node
 */
public function refactor(Node $node): ?Node
{
    if (! $this->isName($node->name, 'get')) {
        // We don't want to modify this node
        return null;
    }
}
```

---

<sup>57</sup><https://github.com/nikic/PHP-Parser/blob/b67560d38854dc072a17f5b0af0df6e094690661/lib/PhpParser/Node/Expr/MethodCall.php#L13-L14>

```

    $node->name = new Identifier('modified');

    return $node;
}

```

Running Rector again, we get a much better result:

1 file with changes

=====

1) app/Controller/UserController.php

```

----- begin diff -----
--- Original
+++ New
@@ @@
{
    public function registerUser(Request $request): Response
    {
-        $username = $request->get('username');
+        $username = $request->modified('username');

-        $hasher = $this->container->get('password_hasher');
+        $hasher = $this->container->modified('password_hasher');

        // ...
    }
}
----- end diff -----

```

Applied rules:

- \* UseRequestRequestGetRector

[OK] 1 file would have changed (dry-run) by Rector

The diff is smaller this time: the call to `$this->render()` will no longer be modified because the method name is `render()`, which will be ignored. We still have one other

*false-positive* though: the call to `$this->container->get()`. Its method name is also `get()`. We know that this call shouldn't be modified. How can we skip this node? A simple approach would be to look at the name of the variable: it's supposed to be `$request`, right? However, the name of a variable alone is not a guarantee that the variable contains a Symfony Request object.

## What Is the Type of a Variable?

Finding out the type of a variable is quite a messy process. You'd have to trace the origins of the variable. Is it a method argument, then you could look at the parameter type for that argument. If it's the return value of some other function, you could look at the declared return type of that function. Luckily we don't have to write all this logic ourselves. Rector uses PHPStan to derive the types of variables and offers a simple but extremely powerful shortcut for this process as the `isObjectType()` utility function. We can use it to exclude method calls on variables that are not instances of Request:

```
/**
 * @param MethodCall $node
 */
public function refactor(Node $node): ?Node
{
    if (! $this->isName($node->name, 'get')) {
        return null;
    }

    if (! $this->isObjectType(
        $node->var,
        new ObjectType('Symfony\Component\HttpFoundation\Request')
    )) {
        // We don't want to modify `get()` calls on other objects
        return null;
    }

    $node->name = new Identifier('modified');
```

```

    return $node;
}

```

Running Rector again, the resulting diff is:

```
1 file with changes
```

```
=====
```

```
1) app/Controller/UserController.php
```

```

----- begin diff -----
--- Original
+++ New
@@ @@
{
    public function registerUser(Request $request): Response
    {
-        $username = $request->get('username');
+        $username = $request->modified('username');

        $hasher = $this->container->get('password_hasher');
----- end diff -----

```

Applied rules:

```
* UseRequestRequestGetRector
```

```
[OK] 1 file would have changed (dry-run) by Rector
```

It seems we have successfully narrowed down the search for node candidates to actual calls to `Request::get()`.

Now we can write something more accurate than `$node->name = 'modified'`; to modify the method call.

Let's use `vendor/bin/php-parse` once more to find out what the modified `MethodCall`

should look like. Create an `after.php` file containing a minimal example of the resulting code.

**after.php**

---

```
<?php
```

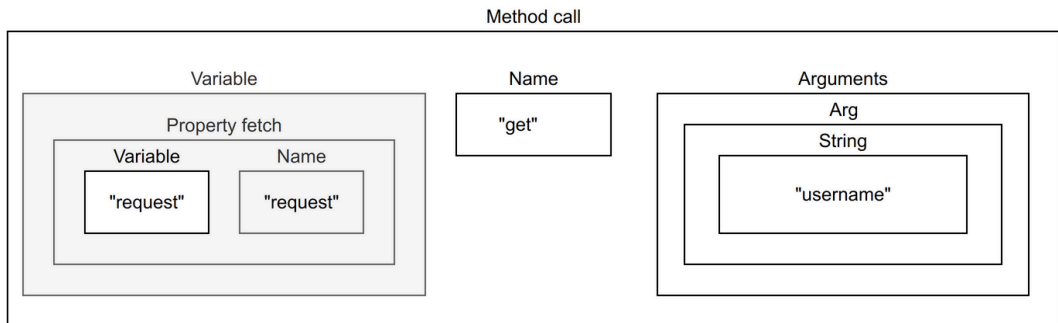
```
$request->request->get('username');
```

---

Then run `vendor/bin/php-parse --var-dump after.php`. It will look something like this:

```
array(1) {
  [0] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\MethodCall) {
      ["var"] => object(PhpParser\Node\Expr\PropertyFetch) {
        ["var"] => object(PhpParser\Node\Expr\Variable) {
          ["name"] => "request"
        }
        ["name"] => object(PhpParser\Node\Identifier) {
          ["name"] => "request"
        }
      }
      ["name"] => object(PhpParser\Node\Identifier) {
        ["name"] => "get"
      }
      ["args"] => array(1) {
        [0] => object(PhpParser\Node\Arg) {
          ["name"] => null
          ["value"] => object(PhpParser\Node\Scalar\String_) {
            ["value"] => "username"
          }
        }
      }
    }
  }
}
```

Here is a visual representation of the method call node:



The grey background color indicates what nodes are different, compared to the `--var-dump` output of `$request->get()` that we've looked at earlier. What has changed is that the “variable” of the method call is now a *property fetch* (`$request->request`). The property fetch itself consists of two parts:

1. The name of the property being fetched, which is `request`,
2. The variable from which we fetch the property, which is the original variable: `$request`.

This means we should keep the original variable, but wrap it in a `PropertyFetch` node. We should then return the modified `MethodCall` node to notify Rector that we changed it. This is what it looks like in code:

```

/**
 * @param MethodCall $node
 */
public function refactor(Node $node): ?Node
{
    if (! $this->isName($node->name, 'get')) {
        return null;
    }

    if (! $this->isObjectTypes(
        $node->var,

```

```

        new ObjectType('Symfony\Component\HttpFoundation\Request')
    )) {
        return null;
    }

    /*
     * Make the original method call on the fetched `request`
     * property instead:
     */
    $node->var = new PropertyFetch($node->var, 'request');

    return $node;
}

```

Running Rector again, we now get a very promising diff:

1 file with changes

=====

1) app/Controller/UserController.php

```

----- begin diff -----
--- Original
+++ New
@@ @@
{
    public function registerUser(Request $request): Response
    {
-        $username = $request->get('username');
+        $username = $request->request->get('username');

        $hasher = $this->container->get('password_hasher');
----- end diff -----

```

Applied rules:

- \* UseRequestRequestGetRector

```
[OK] 1 file would have changed (dry-run) by Rector
```

Great, this in fact produces the change we envisioned at the beginning of this chapter. It's time to run Rector without the `--dry-run` option, so it can actually update the PHP files in our project following the new rule:

```
vendor/bin/rector process
```

When Rector is done, it will show the number of files that have been modified, e.g.

```
[OK] Rector is done! 1 file has been changed.
```

It still shows the diff, but you can also use `git diff` to inspect the changes before committing them.



## Commit Your Code Before Running Rector

Developers, myself included, usually prefer to make all code changes *manually*, so that nothing happens to our code without reviewing it first. That is until we trust a tool to make those changes safely for us. An example is PhpStorm, which we trust when renaming a class or a method. Since Rector may still feel like a new tool, not all of us will be able to trust it immediately.

A good way to gain trust is to use the `--dry-run` option, as we've just seen. Another way is to use traditional version control tactics. Before running Rector you should do a `git commit`. Then after running Rector, you'll be able to inspect the changes it made using `git diff` and `git status`. You still have the chance to run `git restore -- .` at any time to revert all the changes that Rector has made. If you need more fine-grained control, you can also use `git add -p` or `git add -i` to interactively stage only the changes you want to keep.

One final check before we consider our rule “done”: let's run Rector again and see if it didn't try to refactor the code again, resulting in `$request->request->request->get()` and `$request->request->request->request->get()` on subsequent runs. We're lucky, the output is:



[OK] Rector is done!

## What if We Run Rector Twice?

Apparently, processing the same rule again doesn't change the existing code that already looks good according to our rule. The reason is that the second time we run Rector, the resolved type for `$request->request` isn't `Request`. You can confirm this by `var_dump`-ing the return value of `$this->getObjectType($node->var)`. It shows that the type of `$request->request` is now `InputBag`:

```
object(Rector\...\FullyQualifiedObjectType) {  
    ["className"] => "Symfony\Component\HttpFoundation\InputBag"  
}
```

```
object(Rector\...\FullyQualifiedObjectType) {  
    ["className"] => "Psr\Container\ContainerInterface"  
}
```

[OK] Rector is done!

So running the rule again and again will not cause any problems by making the refactoring. It's safe to add it to the standard ruleset for the project. Whenever somebody commits an old-style usage of `$request->get()`, running Rector again will only update that new case and leave existing code untouched.

## Effectivity Beats Perfection

Although we have created a rather nice and clean rule, it's not a rule that could be contributed back to the Rector project. It works in our project, because we made the assumption that whenever a controller made a call to `$request->get()`, it should have actually called `$request->request->get()`. Since `$request->get($key)` also looks in dynamic route attributes and the query parameters for a value with that key, this assumption won't be true in all the projects of the world. Changing `$request->get()` to `$request->request->get()` would break those projects.

In practice, custom rules are often based on assumptions. Sometimes you can verify those assumptions while running the rule, but in many cases it's impossible. This means that some rules that you create will only be useful in your project, or in similar projects that have similar characteristics. Don't hesitate to write those single-purpose rules though. They could be throw-away rules, but they have a positive impact on your project and help keep it maintainable in the long run.

## Summary

A Rector rule is a node visitor that extends from `AbstractRector`. It defines itself using a short description, and one or more before-and-after code snippets. It declares a list of AST node types that the rule is interested in. When Rector processes the files (`vendor/bin/rector process`) in a project, it will parse each PHP file and provide the opportunity to each registered rule to refactor its nodes. Rector then copies the modified code back to the original files. If you run `process` with the `--dry-run` option, Rector won't touch the original files, but only show you the diffs.

# Test-driven Rule Development

## This Chapter Covers:

- Creating a test class for a single Rector rule
- Adding test cases in the form of fixtures
- Taking small steps when creating a rule
- Creating idempotent rules

## Introduction

In the previous chapter we've created our first Rector rule. We concluded that it was a throw-away rule because it wasn't generalized enough to use it in other situations. It was also a single-use rule because we didn't write tests for it. When you want to maintain a rule for more than a couple of days, it won't be enough to rely on *exploratory testing* only. Running your rule on the code base of a real project gives you great feedback about the quality of the rule; it gives you ideas for edge cases you didn't think of. But verifying the correctness of your rule manually, using only visual verification, will be time-consuming and unreliable as well. Automated testing is the easiest way to solve this problem.

Rector ships with some testing tools based on PHPUnit. There's an `AbstractRectorTestCase` class you can extend your test class from. In general each Rector rule you create will have its own test class. This class should extend `AbstractRectorTestCase` and it will be called `[SomeName]RectorTest`. When creating a rule you add several before and after code snippets to demonstrate that your rule makes the correct transformations. You also add examples of code snippets that should be left untouched by your rule.

By adding many examples you can shape your rule gradually to match those examples. Any edge case you encounter later on can be added as yet another example. Doing so will save you a lot of stress: once an example is added as an automated test, it will keep working forever. If not, you'll know it because the test fails.

Although you can write tests for a rule that has already been created, the best time to write tests is *before* you write your first line of code in the rule class. Doing so brings you into the right mindset. You'll think about the problem, while not worrying about the possible complexity of the solution. Just describe what you want, and have faith in your ability to *eventually* come up with the solution.

## Migrating from DateTime to DateTimeImmutable

In this chapter we'll *test-drive* the development of a Rector rule. We start with a problem, describe in code what we want, and then we develop a rule (and eventually a set of rules) that provide the solution to the problem. The problem I'm thinking about is this: at some point PHP added the `DateTimeImmutable` class, which is an [immutable variant](https://derickrethans.nl/immutable-datetime.html)<sup>58</sup> of the already existing `DateTime` class. First, let's take a look at the differences between `DateTime` and `DateTimeImmutable`.

```
use DateTime;

function nextMonday(DateTime $dt): DateTime
{
    $modified = $dt->modify('next monday');

    // The return value of `modify()` could be `false`
    assert($modified instanceof DateTime);

    return $modified;
}

$today = new DateTime();
```

---

<sup>58</sup><https://derickrethans.nl/immutable-datetime.html>

```
$nextMonday = nextMonday($today);  
  
echo $today->format('Y-m-d'), "\n";  
echo $nextMonday->format('Y-m-d'), "\n";
```

Although the function `nextMonday()` seems to return a new instance of `DateTime`, but modified to represent the next Monday, in reality this function also updates the *original* `DateTime` object passed to it. So running this script will just output the same date twice. In code bases that use `DateTime`, this is a common problem that often goes unnoticed, until one day it leads to hard-to-debug problems. The proposed solution has always been to `clone` the provided instance before modifying it:

```
use DateTime;  
  
function nextMonday(DateTime $dt): DateTime  
{  
    $dt = clone $dt;  
  
    $dt->modify('next monday');  
  
    return $dt;  
}
```

Doing so keeps the original object from being modified.

If we had used the “new” `DateTimeImmutable` class, we wouldn’t have had this problem, and we wouldn’t have to `clone` the object:

```
use DateTimeImmutable;

function nextMonday(DateTimeImmutable $dt): DateTimeImmutable
{
    $modified = $dt->modify('next monday');

    assert($modified instanceof DateTimeImmutable);

    return $modified;
}
```

If `nextMonday()` is the only function you'd need to adapt, it wouldn't be a big deal. But what if your code base has thousands of usages of `DateTime` and you want to migrate to `DateTimeImmutable`? Rector is the perfect tool for this job. Let's create a Rector rule that can do at least part of the migration for us.

What applies to other programming tasks, applies to creating Rector rules as well: a problem that looks quite complex from the start should be subdivided into smaller and simpler problems first. For instance, the seemingly complex migration from `DateTime` to `DateTimeImmutable` consists of several smaller (and simpler) tasks:

1. Where the code uses `DateTime` as a type (e.g. as a parameter, return, or property type) we should change it to `DateTimeImmutable`.
2. Where the code has a `use`-statement for `DateTime`, we should also change it to `DateTimeImmutable`.
3. Where the code calls `modify()` on a `DateTime` instance we should capture the result in a variable, e.g. `$dt = $dt->modify('next monday');`
4. Where the code clones a `DateTime` instance, we should no longer clone it.

It's likely that we'll come up with other tasks, but at least this seems like a list of things we can solve with the knowledge acquired from previous chapters. This attitude is what makes test-driven development so great: we can start small, and cycle between adding another test and writing a bit of code to make that test pass. Eventually we'll be able to work ourselves up from the simple, somewhat uninteresting tasks, accumulating smaller bits of behavior into bigger chunks of more complex and powerful behavior. In order to be successful, don't try to build too much at once.

## Creating a Test Class

The first problem to solve will be to switch all `DateTime` types in the code to `DateTimeImmutable` types. A Rector rule for this change could be called `UseDateTimeImmutableRector`. Let's create this class which extends from `AbstractRector` inside the directory `utils/rector/src/Rector`:

```
utils/rector/src/Rector/UseDateTimeImmutableRector.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Rector\Rector;
```

```
use Rector\Core\Rector\AbstractRector;
```

```
final class UseDateTimeImmutableRector extends AbstractRector
{
    // TODO
}
```

---

Next, we create a test class for this rule: `UseDateTimeImmutableRectorTest`. Since we're going to create some extra files next to the test class itself, we'll put the test class in its own directory named after the rule, inside `utils/rector/tests`.

The file structure should look like this:

```

/Utils
  /rector
    /tests
      /Rector
        /SomeSpecificRector
          /config # directory for configs
          /Fixture # the tested fixture files go here
          SomeSpecificRectorTest.php # test file itself

```

The test class needs to extend `AbstractRectorTestCase`. We also need a few basic methods before we can add examples of before and after code:

```
utils/rector/tests/Rector/UseDateTimeImmutableRector/UseDateTimeImmutableRectorTest.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Rector\Tests\Rector\UseDateTimeImmutableRector;
```

```
use Rector\Testing\PHPUnit\AbstractRectorTestCase;
```

```
use Iterator;
```

```
use Symplify\SmartFileSystem\SmartFileInfo;
```

```
final class UseDateTimeImmutableRectorTest extends
```

```
AbstractRectorTestCase
```

```
{
```

```
    /**
```

```
     * @dataProvider provideData()
```

```
    */
```

```
    public function test(SmartFileInfo $fileInfo): void
```

```
    {
```

```
        $this->doTestFileInfo($fileInfo);
```

```
    }
```

```
    /**
```



```

    * @return Iterator<SmartFileInfo>
    */
    public function provideData(): Iterator
    {
        return $this->yieldFilesFromDirectory(__DIR__ . '/Fixture');
    }

    public function provideConfigFilePath(): string
    {
        return __DIR__ . '/config/config.php';
    }
}

```

---

The `provideConfigFilePath()` method returns the path to a Rector configuration file. It uses the same format as the `rector.php` in the root of your project. This file should set up the rule you're testing as a service using the `ContainerConfigurator`:

```
utils/rector/tests/Rector/UseDateTimeImmutableRector/config/config.php
```

---

```
<?php
```

```

declare(strict_types=1);

use Symfony\...\ContainerConfigurator; // (abbreviated)
use Utils\Rector\Rector\UseDateTimeImmutableRector;

return static function (
    ContainerConfigurator $containerConfigurator
): void {
    $services = $containerConfigurator->services();
    $services->set(UseDateTimeImmutableRector::class);
};

```

---

The `test()` method is the actual test. It iterates over all the files in the `Fixture/` directory and runs a few checks based on them. So we also have to create this `Fixture/` directory which should be inside `utils/rector/tests/Rector/UseDateTimeImmutableRector`.



## Add @see Annotations to Jump Between the Rule and its Test

A nice trick to establish a relation between the rule class and its test is to add an @see annotation above the Rector rule class:

```
/**
 * @see \Tests\SomeRectorTest
 */
use Rector\Core\Rector\AbstractRector;

final class SomeRector extends AbstractRector
{
}
```

IDEs like PhpStorm will turn this into a clickable link so you can easily jump back and forth between test and the subject-under-test.

Before we can attempt to run the “empty” test, we should first provide a minimum implementation of the rule, so we don’t get any PHP parse errors:

```
/**
 * @see UseDateTimeImmutableRectorTest
 */
final class UseDateTimeImmutableRector extends AbstractRector
{
    /**
     * @return array<class-string<Node>>
     */
    public function getNodeTypes(): array
    {
        return [];
    }

    public function refactor(Node $node): ?Node
    {
        return null;
    }
}
```

```
}

public function getRuleDefinition(): RuleDefinition
{
    return new RuleDefinition(
        'Replace usages of DateTime with DateTimeImmutable',
        []
    );
}
```

Since we don't know yet which node types we should target with this new rule, we'll return an empty array from `getNodeTypes()`. We also don't know how to make the actual refactoring, so we'll leave all nodes untouched by returning `null` in the `refactor()` method. The rule definition for this rule describes what the job of this rule is. Ideally it also shows some before and after code samples, but we skip them for now since we'll soon start adding these code samples as test fixtures instead.

Now that we've implemented the basics of the Rector rule class, we can now run the test class. Generally speaking you'll have several options for running the test:

- Configure your IDE so you can right-click on the test and run it inside your IDE.
- Add `utils/rector/tests` to your existing `testsuite` element in `phpunit.xml`.
- Provide this directory as a command-line argument when running `phpunit`.

To keep things simple, we'll use the last option now:

```
vendor/bin/phpunit utils/rector/tests
```



## Install PHPUnit

If the `phpunit` command isn't available in your project, you should first install PHPUnit:

```
composer require --dev phpunit/phpunit
```

The output should be something like this:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

S 1 / 1 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK, but incomplete, skipped, or risky tests!

Tests: 1, Assertions: 0, Skipped: 1.

That's because the data provider `provideData()` doesn't return anything. After all, the `Fixture` directory is empty, so there are no examples to process.

## Adding First Test Fixture

Let's start by adding our first test fixture now. A test fixture is a `.php.inc` file which contains two PHP scripts. The first script is the original PHP script. Rector will run your rule on this script and compare the outcome to the second script. The test is successful if your rule has modified the first script in such a way that it now looks like the second script. Even though the file is PHP content, the suffix is not `*.php` to hide from processing by PHPStan, ECS, Rector or PHPUnit.

In our case, a good example would be:

```
utils/rector/tests/Rector/UseDateTimeImmutableRector/Fixture/class-with-datetime-parameter-type.php.inc
```

---

```
<?php
```

```
namespace Utils\Rector\Tests\Rector\UseDateTimeImmutableRector\Fixture;
```

```
use DateTime;
```

```
final class ClassWithDateTimeParameter
```

```
{
    public function __construct(DateTime $parameter)
    {
```

```
    }  
}  
?>  
-----  
<?php  
  
namespace Utils\Rector\Tests\Rector\UseDateTimeImmutableRector\Fixture;  
  
use DateTimeImmutable;  
  
final class ClassWithDateTimeParameter  
{  
    public function __construct(DateTimeImmutable $parameter)  
    {  
    }  
}  
?>
```

---

As you can see, test fixture contains the smallest change possible. This example shows what should happen to a parameter of type `DateTime` and a `use` statement that imports `DateTime`: both of these names should be replaced with `DateTimeImmutable`.

Although your IDE might not be happy about this `.php.inc` file and show you all kinds of errors, both of these scripts are completely valid PHP on their own. The only thing special about this file is the separator between the two scripts (`-----`). This file is called `class-with-datetime-parameter-type.php.inc` and it lives in the `Fixture` directory. In this case the file name describes what we're testing. If the script file name doesn't speak for itself you can always add plain old PHP comments to its content.

Now that we have at least a single fixture, let's run the test again. The output should be something like this:

There was 1 failure:

```
1) UseDateTimeImmutableRectorTest::test with data set #0
class-with-datetime-parameter-type.php.inc
Failed asserting that string matches format description.
```

```
--- Expected
```

```
+++ Actual
```

```
@@ @@
```

```
namespace Utils\Rector\Tests\Rector\UseDateTimeImmutableRector\Fixture;
```

```
-use DateTimeImmutable;
```

```
+use DateTime;
```

```
final class ClassWithDateTimeParameter
{
-    public function __construct(DateTimeImmutable $parameter)
+    public function __construct(DateTime $parameter)
    {
    }
}
```

```
AbstractRectorTestCase.php:114
```

```
AbstractRectorTestCase.php:91
```

```
UseDateTimeImmutableRectorTest.php:21
```

FAILURES!

Tests: 1, Assertions: 3, Failures: 1.

## Making the First Test Pass

Of course, we didn't write any refactoring code for the rule yet, so this test was expected to fail. It would be a miracle, and somewhat disturbing, if Rector had magically replaced `DateTime` with `DateTimeImmutable`. Still, *seeing* it fail is a mandatory step when doing test-driven rule development. From a failing test you

can work your way to a passing test. That's when you know you're done.

The first thing we have to do is determine what node types we should refactor. Let's find out by copying our "before" script to a temporary file, e.g. temp.php:

temp.php

---

```
<?php
```

```
namespace TestDrivenRuleDevelopment\FindingNodeCandidates;
```

```
use DateTime;
```

```
final class ClassWithDateTimeParameter
```

```
{
    public function __construct(DateTime $parameter)
    {
    }
}
```

---

When parsing it with `vendor/bin/php-parse --var-dump`, the result looks something like this:

```
array(1) {
  [0] => object(PhpParser\Node\Stmt\Namespace_) {
    ["name"] => object(PhpParser\Node\Name) {
      ["parts"] => array(1) {
        [0] => "FindingNodeCandidates"
      }
    }
  }
  ["stmts"] => array(2) {
    [0] => object(PhpParser\Node\Stmt\Use_) {
      ["type"] => 1
    }
    [1] => object(PhpParser\Node\Stmt\Class_) {
      ["name"] => object(PhpParser\Node\Identifier) {
        ["name"] => "ClassWithDateTimeParameter"
      }
    }
  }
}
```





nodes with a `DateTimeImmutable` Name node. The `refactor()` method then looks like this:

```
use PhpParser\Node\Name;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class UseDateTimeImmutableRector extends AbstractRector
{
    /**
     * @return array<class-string<Node>>
     */
    public function getNodeTypes(): array
    {
        return [Name::class];
    }

    /**
     * @param Name $node
     */
    public function refactor(Node $node): ?Node
    {
        return new Name('DateTimeImmutable');
    }
}
```

Let's run the test and find out what works and what doesn't work yet:

There was 1 failure:

```
1) UseDateTimeImmutableRectorTest::test with data set #0
class-with-datetime-parameter-type.php.inc
Failed asserting that string matches format description.
--- Expected
+++ Actual
@@ @@
<?php
```

```
-namespace Utils\Rector\Tests\Rector\UseDateTimeImmutableRector\Fixture;  
+namespace DateTimeImmutable;
```

```
use DateTimeImmutable;
```

AbstractRectorTestCase.php:114

AbstractRectorTestCase.php:91

UseDateTimeImmutableRectorTest.php:21

FAILURES!

Tests: 1, Assertions: 3, Failures: 1.

The diff points out that:

1. The constructor parameter has in fact been correctly renamed (it's not shown in the diff, so our rule was successful at making this change).
2. The use statement has been renamed (same here: it's not shown in the diff, which is good).
3. An “unexpected” difference has popped up: the namespace has been modified as well.

That last change is certainly not desirable. Even if the namespace would literally be `DateTime`, we shouldn't change it, ever. The namespace of a class is in no way related to an out-dated use of PHP's `DateTime` class. So what we have to do is modify our `refactor()` method:

1. We should find out if the `Name` node we're going to replace is the name of a namespace.
2. If that's the case, we shouldn't replace it, that is, we should return `null`.

## Narrowing the Scope of the Refactoring

To implement the first step, we need to be able to learn more about the context of the `Name` node we're refactoring. In particular, we need to know if its parent node

is a `Namespace_` node. As we've seen before in the section [Connecting nodes](#), by default PHP-Parser nodes contain references to their subnodes, but not to their parent nodes. If you want to establish such a relation, you have to use the `NodeTraverser` together with the `NodeConnectingVisitor`. When using `Rector`, we don't have to do this manually, because `Rector` already does it for us. The visitor modifies all nodes in the tree to contain a reference to their parent node, which can be accessed using `$node->getAttribute()`.

The 'parent' attribute can be an actual `Node` object, or `null` in case `$node` is a top-level statement node. This allows us to do a simple `instanceof` check and return `null` in case the parent node is a `Namespace_` node:

```
use Rector\NodeTypeResolver\Node\AttributeKey;
use PhpParser\Node\Stmt\Namespace_;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class UseDateTimeImmutableRector extends AbstractRector
{
    /**
     * @param Name $node
     */
    public function refactor(Node $node): ?Node
    {
        $parent = $node->getAttribute(AttributeKey::PARENT_NODE);
        if ($parent instanceof Namespace_) {
            return null;
        }

        return new Name('DateTimeImmutable');
    }
}
```



To prevent typos in the attribute name and enhance discoverability, use the `AttributeKey` class which has public constants for all supported attribute keys.

After making this change we can run the test again, and we should see it pass: there

are no differences between the expected result of applying our rule compared to the actual result:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

. 1 / 1 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK (1 test, 2 assertions)

Now, our single example in `Fixtures/` is pretty minimal, so there will be many issues when we run this code on a real project. One thing we don't know at this point is: would `Name` nodes occur in other places than parameter, property, or return types? Should we also excluded those cases?

There's another, more pressing issue. At this point, we do exclude namespace names, but we still change *all* other names. Of course, we shouldn't do that. Only if the current name refers to the `DateTime` class we should change it to use `DateTimeImmutable` instead. If the name isn't `DateTime`, we shouldn't touch it.

Again, we have several options. We could modify the existing fixture and add a second parameter with a type that's different than `DateTime` and let it be the same in both the before and after code samples, proving that this parameter remains untouched. However, a best practice for test-driven development is *not to modify an existing test* or example, but instead to *add a new one* that helps you prove another point.

For normal PHPUnit-based tests this means you would add another method, e.g. `testSkipOtherNames()`. For a Rector rule test it means you add another fixture, e.g. `skip-other-names.php.inc`, that describes the behavior you're looking for.

A minimal example is a class that has a parameter of a different type. This type should itself be a class. Let's call this class `SomeOtherClass`, because that would reveal the intention behind its presence in the test fixture:

```
utils/rector/tests/Reactor/UseDateTimeImmutableReactor/Fixture/skip-other-names.php.inc
```

---

```
<?php
```

```
namespace Utils\Reactor\Tests\Reactor\UseDateTimeImmutableReactor\Fixture;
```

```
final class ClassUsingSomeOtherClass
{
    public function __construct(SomeOtherClass $parameter)
    {
    }
}
```

---

This fixture is different from the other fixture we have, because it doesn't have distinct before and after code samples separated by -----. If you want to prove that a rule doesn't make any change to the original code, you don't have to provide two versions. In fact, the test fixture *should* only contain one code sample. The Rector test runner will interpret this as: there should be no changes to this code after running the rule.



## Start Single-script Fixtures with “skip-”

Fixtures that contain only a single script are examples of code that your rule should “skip”, so the Rector project itself uses the convention to start these fixtures with `skip-` as well.



## Is it Okay if SomeOtherClass Doesn't Actually Exist?

It's okay if `SomeOtherClass` isn't a real class. PHP can parse and even load this code without any issues. It only becomes a problem when you try to instantiate a class that doesn't exist, or when you use a static analysis tool like PHPStan on your fixtures.

After adding the new fixture, let's run the tests again. We should see it fail:

There was 1 failure:

1) UseDateTimeImmutableRectorTest::test with data set #0

skip-other-names.php.inc

Failed asserting that string matches format description.

--- Expected

+++ Actual

@@ @@

```
final class ClassUsingSomeOtherClass
{
-   public function __construct(SomeOtherClass $parameter)
+   public function __construct(DateTimeImmutable $parameter)
    {
    }
}
```

AbstractRectorTestCase.php:114

AbstractRectorTestCase.php:91

UseDateTimeImmutableRectorTest.php:21

FAILURES!

Tests: 2, Assertions: 5, Failures: 1.

The diff shows what we already expected: our rule has changed `SomeOtherClass` into `DateTimeImmutable`. This shouldn't have happened. To make the tests pass we have to change the `refactor()` method of our rule and add another early return: if the name of the node is something else than `DateTime`, we should not replace it. The `AbstractRector` class comes with a reusable method called `isName()` that can help you find out the name of a number of node types, like class method nodes, interface nodes, and so on. So let's use it here as well:

```

/**
 * @param Name $node
 */
public function refactor(Node $node): ?Node
{
    $parent = $node->getAttribute(AttributeKey::PARENT_NODE);
    if ($parent instanceof Namespace_) {
        return null;
    }

    if (!$this->isName($node, DateTime::class)) {
        return null;
    }

    return new Name('DateTimeImmutable');
}

```

Running the tests again, they should now pass:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

.. 2 / 2 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK (2 tests, 4 assertions)

Great! We have implemented the first automated refactoring that we need to migrate from `DateTime` to `DateTimeImmutable`. We have included tests, so whatever we do in subsequent steps, the existing behavior will be preserved.

## Capturing the Return Value of `modify()`

As we saw in the beginning of this chapter, migrating from `DateTime` to `DateTimeImmutable` isn't just a matter of replacing the class name everywhere. The behavior changes

too. For example, when we call `modify()` on a `DateTime` instance, the instance itself will be modified. When calling `modify()` on a `DateTimeImmutable`, `modify()` will not change the instance, but return a new instance. So to automate the migration we also have to refactor `$instance->modify('next monday')` to `$instance = $instance->modify('next monday')`. We're lucky, because this is a safe refactoring, even if we would keep using `DateTime`. That's because `DateTime::modify()` also returns the modified instance, so `$instance = $instance->modify('next monday')` is valid in both the before and after situation.

Should we implement this new task in the rule class we already have? In some cases that certainly is an option you can try out. Generally speaking, it's better to create a new rule. That's because a rule preferably targets *only a single node type*. Once it starts to target several node types, the code in `refactor()` quickly becomes too complex to manage. Given the refactoring task we're looking at now isn't going to target `Name_` nodes, but `MethodCall` nodes, I would say it's smart to implement this behavior in a new rule.

Let's call the rule `CaptureReturnValueRector` and provide some basic implementations for its required methods:

`utils/rector/src/Rector/CaptureReturnValueRector.php`

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Rector\Rector;
```

```
use PhpParser\Node;
```

```
use Rector\Core\Rector\AbstractRector;
```

```
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;
```

```
final class CaptureReturnValueRector extends AbstractRector
{
```

```
    /**
```

```
     * @return array<class-string<Node>>
```

```
    */
```

```
    public function getNodeTypes(): array
```



```

    {
        return [];
    }

    public function refactor(Node $node): ?Node
    {
        return null;
    }

    public function getRuleDefinition(): RuleDefinition
    {
        return new RuleDefinition(
            'Capture the return value of DateTime::modify()',
            []
        );
    }
}

```

---

We'll go over the TDD process for this rule with slightly less ceremony now. We create a test class for this rule, extending from `AbstractRectorTestCase`, and we make sure the required methods are there:

`utils/rector/tests/Rector/CaptureReturnValueRector/CaptureReturnValueRectorTest.php`

---

```
<?php
```

```

declare(strict_types=1);

namespace Utils\Rector\Tests\Rector\CaptureReturnValueRector;

use Iterator;
use Rector\Testing\PHPUnit\AbstractRectorTestCase;
use Symplify\SmartFileSystem\SmartFileInfo;

final class CaptureReturnValueRectorTest extends AbstractRectorTestCase
{
    /**

```

```

    * @dataProvider provideData()
    */
    public function test(SmartFileInfo $fileInfo): void
    {
        $this->doTestFileInfo($fileInfo);
    }

    /**
     * @return Iterator<SmartFileInfo>
     */
    public function provideData(): Iterator
    {
        return $this->yieldFilesFromDirectory(__DIR__ . '/Fixture');
    }

    public function provideConfigFilePath(): string
    {
        return __DIR__ . '/config/config.php';
    }
}

```

---

This time the config.php file should only load the new CaptureReturnValueRector rule:

```
utils/rector/tests/Rector/CaptureReturnValueRector/config/config.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
use Utils\Rector\Rector\CaptureReturnValueRector;
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $containerConfigurator->services()
```

```

        ->set(CaptureReturnValueRector::class);
    };

```

---

We also create a `Fixture` directory for the test, and a first `.php.inc` fixture file to demonstrate our intentions with this rule:

```
utils/rector/tests/Rector/CaptureReturnValueRector/Fixture/capture-return-value-of-modify-call.php.inc
```

---

```
<?php
```

```

$dt = new DateTime();
$dt->modify('next monday');
?>

```

```
-----
```

```
<?php
```

```

$dt = new DateTime();
$dt = $dt->modify('next monday');
?>

```

---

Running the test for the first time, we'll see it fail:

There was 1 failure:

```

1) CaptureReturnValueRectorTest::test with data set #0
capture-return-value-of-modify-call.php.inc
Failed asserting that string matches format description.

```

```
--- Expected
```

```
+++ Actual
```

```
@@ @@
```

```
<?php
```

```

$dt = new DateTime();
-$dt = $dt->modify('next monday');
+$dt->modify('next monday');
?>

```

```
AbstractRectorTestCase.php:114
AbstractRectorTestCase.php:91
CaptureReturnValueRectorTest.php:18
```

FAILURES!

Tests: 1, Assertions: 3, Failures: 1.

The first thing we should do is target a specific node type. To find out what node type specifically, we'll use `vendor/bin/php-parse` again, on a temporary file that contains the “before” code of the fixture:

**before.php**

---

```
<?php
```

```
$dt = new DateTime();
$dt->modify('next monday');
```

---

```
vendor/bin/php-parse before.php --var-dump
```

The result looks something like this:

```
array(2) {
  [0] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\Assign) {
      ["var"] => object(PhpParser\Node\Expr\Variable) {
        ["name"] => "dt"
      }
    }
    ["expr"] => object(PhpParser\Node\Expr\New_) {
      ["class"] => object(PhpParser\Node\Name) {
        ["parts"] => array(1) {
          [0] => "DateTime"
        }
      }
    }
  }
}
```

```

        ["args"] => array(0)
    }
}
}
[1] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\MethodCall) {
        ["var"] => object(PhpParser\Node\Expr\Variable) {
            ["name"] => "dt"
        }
        ["name"] => object(PhpParser\Node\Identifier) {
            ["name"] => "modify"
        }
        ["args"] => array(1) {
            [0] => object(PhpParser\Node\Arg) {
                ["name"] => null
                ["value"] => object(PhpParser\Node\Scalar\String_) {
                    ["value"] => "next monday"
                }
            }
        }
    }
}
}
}
}

```

The first node is the assignment expression (`$dt = new DateTime()`). We can see the var subnode, which contains the name of the `$dt` variable. We also see the expr subnode, which contains the value that will be assigned to the variable. That value is a new object of type `DateTime`.

The second node is the method call expression (`$dt->modify('next monday')`). The specific node we should be looking for is that node of type `MethodCall`. When the name part of this expression equals `modify` then we should wrap this method call in an assignment node:

```

use PhpParser\Node\Expr\Assign;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class CaptureReturnValueRector extends AbstractRector
{
    /**
     * @return array<class-string<Node>>
     */
    public function getNodeTypes(): array
    {
        return [MethodCall::class];
    }

    /**
     * @param MethodCall $node
     */
    public function refactor(Node $node): ?Node
    {
        if (! $this->isName($node->name, 'modify')) {
            return null;
        }

        return new Assign($node->var, $node);
    }
}

```

Running the test, we unfortunately get an exception:

InfiniteLoopTraversingException

What happened? After changing `$dt->modify()` into `$dt = $dt->modify()`, our rule would repeat the same process with nested method call over and over again:

```
// first run:
$dt->modify();

// second run:
$dt = $dt->modify();

// third run:
$dt = ($dt = $dt->modify());
```

To prevent this infinite loop we need to ensure we're not doing the same refactoring twice. We need to do two things to accomplish that:

1. We need to establish a parent-subnode relationship between the original `MethodCall` node and the new `Assign` node.
2. We have to add another early return to the `refactor()` method: if the parent node of the `MethodCall` node is already an `Assign` node, we shouldn't wrap it in an `Assign` node again.

In code, this looks as follows:

```
use Rector\NodeTypeResolver\Node\AttributeKey;
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class CaptureReturnValueRector extends AbstractRector
{
    /**
     * @param MethodCall $node
     */
    public function refactor(Node $node): ?Node
    {
        if (! $this->isName($node->name, 'modify')) {
            return null;
        }

        // If the return value is already captured, do nothing:
```

```
$parent = $node->getAttribute(AttributeKey::PARENT_NODE);  
if ($parent instanceof Assign) {  
    return null;  
}  
  
return new Assign($node->var, $node);  
}
```

After making this change we can run the tests again, and this time the process doesn't end up in an infinite loop:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

.

1 / 1 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK (1 test, 2 assertions)

Great, so this means we can automatically refactor existing code to capture the return value of calls to `modify()`.

## Skipping Calls On Classes That Are Not DateTime

We're not done yet. Our rule is still too generic: it applies itself to any call to `modify()` while it should only change calls to `DateTime::modify()`. Let's turn this requirement into a second test fixture:



```
utils/rector/tests/Reactor/CaptureReturnValueReactor/Fixture/skip-modify-on-non-datetime.php.inc
```

---

```
<?php
```

```
$object = new ClassWithModifyMethod();
$object->modify();
```

---

This fixture introduces another class which has a `modify()` method. This class is an invented class, which doesn't really exist. As explained earlier, this often isn't a problem. It only becomes a problem when you need to analyze the class and its method to find out more about the parameter or return types it uses. In that case the parser will need to load the actual class. We'll encounter a situation like this in the next chapter, which is when we'll introduce a `Source` class.

Running the test, we see it fail. That was to be expected, because the current version of the rule changes *every* call to `modify()` without looking for the type of the object it's called on:

There was 1 failure:

```
1) CaptureReturnValueReactorTest::test with data set #0
skip-modify-on-non-datetime.php.inc
Failed asserting that string matches format description.
```

```
--- Expected
```

```
+++ Actual
```

```
@@ @@
```

```
<?php
```

```
$object = new ClassWithModifyMethod();
-$object->modify();
+$object = $object->modify();
```

```
AbstractReactorTestCase.php:114
```

```
AbstractReactorTestCase.php:91
```

```
CaptureReturnValueReactorTest.php:21
```

FAILURES!

Tests: 2, Assertions: 5, Failures: 1.

How can we improve our rule to only change calls to `DateTime::modify()`? In the rule class we can use the helper function `isObjectType()` from the `AbstractRector` class to find out if the type of the node matches the provided type. We should check if the class name is `DateTime`. If it isn't, we should not change anything about it, and return `null`:

```
use PHPStan\Type\ObjectType;

final class CaptureReturnValueRector extends AbstractRector
{
    /**
     * @param MethodCall $node
     */
    public function refactor(Node $node): ?Node
    {
        if (! $this->isName($node->name, 'modify')) {
            return null;
        }

        if (! $this->isObjectType(
            $node->var,
            new ObjectType('DateTime')
        )) {
            return null;
        }
    }
}
```

Running the tests again, we find that this indeed does the trick:

OK (2 tests, 4 assertions)

The question remains: is this the full solution? Did we cover every possible scenario? With test-driven development this doesn't really matter. We just keep adding

examples (fixtures) until we feel safe enough that our code holds up in most common scenarios. That said, there is more to discuss when it comes to testing, but we'll do that in the next chapter.

## Summary

Tests for a Rector rule are based on PHPUnit. Each rule gets its own `[SomeName]RectorTest` class which extends from `AbstractRectorTestCase`. The test class contains a single `test()` method which loads all the `.php.inc` files in the `Fixture` directory. It then runs the rule against the scripts and tests whether processing the rule has the desired effect. If the fixture contains two scripts separated by `-----`, the test runs the rule against the code above the break and compares it to the expected outcome below the break. If the fixture contains only a single script, the test verifies that the rule leaves this code untouched.

Creating a Rector rule is an incremental process. You describe each aspect of the intended behavior as a fixture with a descriptive name. When you run the test, it will fail the first time. You then modify the rule class to make the test pass. After seeing the test pass, you shouldn't change the fixture anymore. Instead you add new fixtures that describe new behaviors.

Try to limit the number of nodes your rule can deal with. Ideally your rule should only manipulate one node type. You'll probably need to write several rules to implement all the automated refactorings you need in order to finish a migration project like migrating from `DateTime` to `DateTimeImmutable`. In fact, we'll continue this project in the next chapter.

# More Testing Techniques

## This Chapter Covers:

- Specifying how a combination of rules should work together
- Proving that a refactoring didn't change existing behavior
- Providing specific configuration values for a rule
- Defining support classes for fixtures

## Testing Multiple Rules Combined

In the previous chapter we've talked about the need to keep rules small and focused on (preferably) a single node type. However, at some point it will be useful to find out if multiple rules work well together. Or, even better, to specify how they should work together and then add a test for this expectation. Now that we have a rule that renames `DateTime` to `DateTimeImmutable` and another rule that captures the return value of `modify()`, we should find out if these rules don't get in each other's way, and will eventually enable us to migrate an entire project from using `DateTime` to using `DateTimeImmutable`.

Multiple rules that work together to achieve a bigger refactoring or migration are called a *rule set*. A rule set is defined by a configuration file in which a number of rules will be registered, combined with some specific settings for those rules (we'll talk about configuring rules in the section [Making rule behavior configurable](#)). We've seen the syntax of this configuration file before, when we created a `rector.php` configuration file for the entire project, and when we created `config.php` files for the separate rule tests. When you're testing a single rule you'll enable just that rule. When testing multiple rules you can call `set()` multiple times to enable them all at the same time.

Let's do this in a configuration file called `date_time_immutable.php`:

`utils/rector/config/sets/date_time_immutable.php`

---

```
<?php
```

```
declare(strict_types=1);
```

```
use Utils\Rector\Rector\CaptureReturnValueRector;
```

```
use Utils\Rector\Rector\UseDateTimeImmutableRector;
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $services = $containerConfigurator->services();
```

```
    $services->set(CaptureReturnValueRector::class);
```

```
    $services->set(UseDateTimeImmutableRector::class);
```

```
};
```

---

The advantage of having a single configuration file that enables all rules together like this, is that you can easily include this file in the project's main `rector.php` configuration file:

`rector.php`

---

```
<?php
```

```
declare(strict_types=1);
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $containerConfigurator->import(
```

```
        __DIR__ . '/utils/rector/config/sets/date_time_immutable.php'
```

```
);
};
```

---

The next thing we have to do is create a test class for the rule set. Let's call it `MigrateToDateTimeImmutableRectorTest`. We already have a configuration file that enables the rules, so we can return its path in `provideConfigFileInfo()` as well:

```
utils/rector/tests/Rector/MigrateToDateTimeImmutableRector/MigrateToDateTimeImmutableRectorTest.php
final class MigrateToDateTimeImmutableRectorTest extends
AbstractRectorTestCase
{
    /**
     * @dataProvider provideData()
     */
    public function test(SmartFileInfo $fileInfo): void
    {
        $this->doTestFileInfo($fileInfo);
    }

    /**
     * @return Iterator<SmartFileInfo>
     */
    public function provideData(): Iterator
    {
        return $this->yieldFilesFromDirectory(__DIR__ . '/Fixture');
    }

    public function provideConfigFilePath(): string
    {
        return __DIR__
            . '/../../config/sets/date_time_immutable.php';
    }
}
```

---

The test needs at least one fixture file and the contents should be the before and after code of the previous “happy path” fixtures combined:

```
utils/rector/tests/Reactor/MigrateToDateTimeImmutableReactor/Fixture/successful-migration.php.inc
```

```
<?php
```

```
$dt = new DateTime();  
$dt->modify('next monday');  
?>
```

```
-----
```

```
<?php
```

```
$dt = new DateTimeImmutable();  
$dt = $dt->modify('next monday');  
?>
```

When running the test, the output is:

```
PHPUnit 9.5.5 by Sebastian Bergmann and contributors.
```

```
.
```

```
1 / 1 (100%)
```

```
Time: 00:00.782, Memory: 64.50 MB
```

```
OK (1 test, 2 assertions)
```

Normally when the tests pass the first time you run them, you should take it as a warning sign. It often means that the test didn't even run. However, at this point we're testing rules that have already been specified and tested on their own. Running them together shouldn't give us any surprising results. However, it's still a good idea to add at least one rule set test that proves that the rules, when used together, don't get in each other's way.

## Rules Should Be Idempotent

By adding fixtures you can prove that a rule or rule set can deal with several situations. One situation that every rule has to deal with is when the user runs the rule more than once. The code should only be modified the first time the user runs

the rule. The second time, and any other time, the code should be left untouched, because it has already been refactored to the standards proposed by the rule.

In several other areas of programming, the quality of a function to only make the change the first time you call it is called *idempotency*. In the case of a Rector rule we can prove idempotency by adding an example where the before code is already what we expect it to be. Although the Rector test case itself doesn't enforce it, you should always add a fixture to your test that proves idempotency for your rule to prevent surprises when you run the rule again on your code base.

Let's add such an idempotency test fixture to the rule set test now:

```
utils/rector/tests/Rector/MigrateToDateTimeImmutableRector/Fixture/skip-already-migrated.php.inc
```

---

```
<?php
```

```
$dt = new DateTimeImmutable();  
$dt = $dt->modify('next monday');
```

---

Running the tests, we see that the rules in our rule set are already idempotent:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

```
..                                                    2 / 2 (100%)
```

```
Time: 00:00.782, Memory: 64.50 MB
```

```
OK (2 tests, 4 assertions)
```

That's because `CaptureReturnValueRector` has an early return in case the return value of `modify()` gets already assigned to a variable. For `UseDateTimeImmutableRector` we verify that the name matches `DateTime`, which won't be the case on the second run. However, it would still be smart to add idempotency fixtures to the tests for these rules as well. In fact, stick to this as a general requirement for rule tests: *always add a fixture to prove idempotency*.



## Removing the Clone Step

As discussed in the introduction of the previous chapter, you have to use `clone` on `DateTime` instances before you call `modify()` on them. This prevents the original instance from being modified. When using `DateTimeImmutable` this is no longer required. Cloning a `DateTimeImmutable` instance isn't harmful, but it will be a surprise to anyone who reads the code. So it will be smart to write a new Rector rule for removing the superfluous `clone` statements.

We'll do the initiate-Rector-rule dance once more:

1. Write a fixture that demonstrates the desired result of the automated refactoring.
2. Pick a name for the rule class, let it extend `AbstractRector`, add a short description.
3. Create a test class, let it extend `AbstractRectorTestCase`, and create a `Fixture` directory.
4. Run the test and see it fail.
5. Find out what node type the new rule is going to modify by dumping the nodes of the “before” part of the fixture.
6. Declare this type in `getNodeTypes()`.
7. Implement the `refactor()` method in the simplest way possible to make the test pass.

The first step requires a bit of thought. A first try:

```
<?php
```

```
$dt = new DateTime();  
$dt = clone $dt;  
$dt->modify('next monday');  
?>
```

```
-----
```

```
<?php
```

```
$dt = new DateTime();  
$dt->modify('next monday');  
?>
```

We're just removing the whole line that contains `clone`. In this example, the result will still be correct. Consider a slightly modified version though:

```
<?php  
  
$now = new DateTime();  
$nextMonday = clone $now;  
$nextMonday->modify('next monday');  
?>  
-----  
<?php  
  
$now = new DateTime();  
$nextMonday->modify('next monday');  
?>
```

Removing the entire line would be a mistake in this case, because it breaks the code: the `$nextMonday` variable will be undefined. A safer refactoring would be to remove only the `clone` part, and preserve the variable (re)assignment:

```
<?php  
  
$now = new DateTime();  
$nextMonday = clone $now;  
$nextMonday->modify('next monday');  
?>  
-----  
<?php  
  
$now = new DateTime();  
$nextMonday = $now;
```

```
$nextMonday->modify('next monday');  
?>
```

Technically we can make our rule deal with both cases. If the left and right variable name are the same we could remove the entire line, if they aren't, we just remove the `clone` part.

Another option is to just remove the `clone` part in all cases, and create *another* rule that fixes the unnecessary reassignment situation. In fact, that would be a nice idea anyway, since it isn't strictly related to `DateTime` and `DateTimeImmutable`-related code. Any statement that looks like `$b = $a` can be removed, if all subsequent occurrences of `$b` will be renamed to `$a`.

When creating Rector rules, it's always good to look for boundaries like this. What behavior belongs to the rule I'm creating now? What behavior should be implemented by another rule? A guiding question can be: is the issue I'm solving here unique to the problem I'm working on right now? If it isn't, it's better to write another generic rule that makes your solution also useful in a different situation.

Following the same reasoning, we should let our new rule only remove the `clone` part, and we shouldn't try to fix the variable rename situation at the same time.

The next question is: do we remove `clone` when it's used on instances of `DateTime` or `DateTimeImmutable`? In our recent attempts we've removed `clone` when used on a `DateTime` instance. However, doing so results in incorrect code. Unless we only use this rule as part of the rule set, which also replaces `DateTime` with `DateTimeImmutable` everywhere. The dilemma can be solved by leaving the judgement to the user. If they want to remove `clone` statements, they should be aware that it influences the behavior of the code. So they should really only use this rule as part of the `DateTime` to `DateTimeImmutable` migration process. Having decided on this, it means we should use `DateTime` in our fixtures, and the `UseDateTimeImmutableRector` rule will take care of the rest.

The test for this rule looks the same as the other tests we've created:

utils/rector/tests/Reactor/RemoveCloneOnDateTimeReactor/RemoveCloneOnDateTimeReactorTest.php

---

<?php

**declare**(strict\_types=1);

**namespace** Utils\Reactor\Tests\Reactor\RemoveCloneOnDateTimeReactor;

**use** Reactor\Testing\PHPUnit\AbstractReactorTestCase;

**use** Iterator;

**use** Symplify\SmartFileSystem\SmartFileInfo;

**final class** RemoveCloneOnDateTimeReactorTest **extends**

AbstractReactorTestCase

{

/\*\*

\* @dataProvider provideData()

\*/

**public function** test(SmartFileInfo \$fileInfo): void

{

\$this->doTestFileInfo(\$fileInfo);

}

/\*\*

\* @return Iterator<SmartFileInfo>

\*/

**public function** provideData(): Iterator

{

**return** \$this->yieldFilesFromDirectory(\_\_DIR\_\_ . '/Fixture');

}

**public function** provideConfigFilePath(): string

{

**return** \_\_DIR\_\_ . '/config/config.php';

}

}

---

Let's add a single fixture that shows the result when `clone` is used on a `DateTime` object:

`utils/rector/tests/Reactor/RemoveCloneOnDateTimeReactor/Fixture/remove-clone-before-datetime-instance.php`

---

```
<?php
```

```
$now = new DateTime();
```

```
$copy = clone $now;
```

```
?>
```

```
-----
```

```
<?php
```

```
$now = new DateTime();
```

```
$copy = $now;
```

```
?>
```

---

We don't need a fixture to prove idempotency, because you can't remove `clone` twice anyway.

Next we initiate the new rule class, keeping it mostly empty for now:

`utils/rector/src/Reactor/RemoveCloneOnDateTimeReactor.php`

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Reactor\Reactor;
```

```
use PhpParser\Node;
```

```
use Reactor\Core\Reactor\AbstractReactor;
```

```
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;
```

```
final class RemoveCloneOnDateTimeReactor extends AbstractReactor
{
```

```
    /**
```

```
     * @return array<class-string<Node>>
```

```

    */
    public function getNodeTypes(): array
    {
        return [];
    }

    public function refactor(Node $node): ?Node
    {
        return null;
    }

    public function getRuleDefinition(): RuleDefinition
    {
        return new RuleDefinition(
            'Remove clone on DateTime instances',
            []
        );
    }
}

```

---

When running the test, we see it fail:

There was 1 failure:

1) RemoveCloneOnDateTimeReactorTest::test with data set #0  
 remove-clone-before-datetime-instance.php.inc  
 Failed asserting that string matches format description.

--- Expected

+++ Actual

@@ @@

<?php

\$now = new DateTime();

-\$copy = \$now;

+\$copy = clone \$now;

?>

AbstractRectorTestCase.php:114

AbstractRectorTestCase.php:91

RemoveCloneOnDateTimeRectorTest.php:18

FAILURES!

Tests: 1, Assertions: 3, Failures: 1.

That's to be expected, because we didn't implement the `refactor()` method yet. Now it's time to analyze the nodes of the before script of the first fixture. The result looks like this:

```
array(2) {
  [0] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\Assign) {
      ["var"] => object(PhpParser\Node\Expr\Variable) {
        ["name"] => "now"
      }
      ["expr"] => object(PhpParser\Node\Expr\New_) {
        ["class"] => object(PhpParser\Node\Name) {
          ["parts"] => array(1) {
            [0] => "DateTime"
          }
        }
      }
    }
  }
  [1] => object(PhpParser\Node\Stmt\Expression) {
    ["expr"] => object(PhpParser\Node\Expr\Assign) {
      ["var"] => object(PhpParser\Node\Expr\Variable) {
        ["name"] => "nextMonday"
      }
      ["expr"] => object(PhpParser\Node\Expr\Clone_) {
        ["expr"] => object(PhpParser\Node\Expr\Variable) {
          ["name"] => "now"
        }
      }
    }
  }
}
```

```

        }
    }
}
}
}

```

The `Clone_` node seems like the perfect candidate. In the `refactor()` method we should replace each `Clone_` node with whatever gets “cloned”, which is contained in the `expr` subnode:

`utils/rector/src/Rector/RemoveCloneOnDateTimeRector.php`

---

```

/**
 * @return array<class-string<Node>>
 */
public function getNodeTypes(): array
{
    return [Clone_::class];
}

/**
 * @param Clone_ $node
 */
public function refactor(Node $node): ?Node
{
    return $node->expr;
}

```

---

The result should be that `clone` gets removed from the code. Indeed, running the test again, we see it pass:



PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

.

1 / 1 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK (1 test, 2 assertions)

Of course, we know that removing `clone` everywhere, without even considering what's being cloned, is a bad idea. However, this really was the simplest thing we could do to make the test pass. It's a good rule to keep in mind when creating your own rules: do the least amount of work to make each example work. This means you have to keep your examples very small too.

To show that other cases of `clone` will be left untouched, let's add another fixture:

utils/rector/tests/Reactor/RemoveCloneOnDateTimeReactor/Fixture/skip-clone-on-other-instances.php.inc

---

<?php

```
$object = new stdClass();
$copy = clone $object;
```

---

After adding this fixture, the test fails. The fix is easy: we can reuse the `isObjectType()` method we've used before, to see if the object being cloned is a `DateTime` instance:

utils/rector/src/Reactor/RemoveCloneOnDateTimeReactor.php

---

```
use Symplify\RuleDocGenerator\ValueObject\RuleDefinition;

final class RemoveCloneOnDateTimeReactor extends AbstractReactor
{
    /**
     * @param Clone_ $node
     */
    public function refactor(Node $node): ?Node
    {
        if (! $this->isObjectType(
            $node->expr,
```

```
        new ObjectType('DateTime')
    )) {
        return null;
    }

    return $node->expr;
}
```

---

Running the tests again, we see that this additional check fixed it:

PHPUnit 9.5.5 by Sebastian Bergmann and contributors.

.. 2 / 2 (100%)

Time: 00:00.782, Memory: 64.50 MB

OK (2 tests, 4 assertions)

We also have to update the rule set configuration file to include the new rule:

utils/rector/config/sets/date\_time\_immutable.php

---

<?php

```
declare(strict_types=1);
```

```
use Utils\Rector\Rector\CaptureReturnValueRector;
```

```
use Utils\Rector\Rector\RemoveCloneOnDateTimeRector;
```

```
use Utils\Rector\Rector\UseDateTimeImmutableRector;
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $services = $containerConfigurator->services();
```

```
    $services->set(CaptureReturnValueRector::class);
```

```
$services->set(UseDateTimeImmutableRector::class);

// The new rule:
$services->set(RemoveCloneOnDateTimeRector::class);
};
```

---

The tests pass, great!

## Comparing Return Values to Validate Correctness

So far we've been developing several Rector rules in a test-driven way, proving that our rules make the desired transformations in various situations. What we didn't prove though is that those transformations are behavior-preserving. This is actually quite an important aspect of any refactoring work that we do as developers, whether we do it manually or in an automated fashion.

In order to test preservation of behavior, we need to run a bit of code that's written in the “before” style of our transformation, then run a bit of code in the “after” style, and finally verify that they produce the same result. A code fragment that we could use to test the behavior preservation of our `migrate_to_date_time_immutable` ruleset is the `nextMonday()` function we started the previous chapter with:

```
use DateTime;

function nextMonday(DateTime $dt): DateTime
{
    $dt = clone $dt;

    $dt->modify('next monday');

    return $dt;
}
```

The “after” version of this code looks like this:

```
use DateTimeImmutable;

function nextMonday(DateTimeImmutable $dt): DateTimeImmutable
{
    $modified = $dt->modify('next monday');

    assert($modified instanceof DateTimeImmutable);

    return $modified;
}
```

It uses `DateTimeImmutable` instead of `DateTime`, it captures the return value of `modify()`, and it doesn't clone the original variable.

To demonstrate that these functions are equivalent in a test, we should run both these functions and then normalize the returned `DateTime` or `DateTimeImmutable` instance to a string value that we can compare. We can't use a regular `.php.inc` fixture for such a test. Instead we have to add an extra test method to the existing `MigrateToDateTimeImmutableRectorTest` class, for instance:

```
public function testItPreservesBehavior(): void
{
    $old = function (DateTime $dt): DateTime {
        $dt = clone $dt;

        $dt->modify('next monday');

        return $dt;
    };

    $new = function (DateTimeImmutable $dt): DateTimeImmutable {
        $modified = $dt->modify('next monday');

        assert($modified instanceof DateTimeImmutable);

        return $modified;
    };
}
```

```

};

$dt = new DateTime();
self::assertEquals(
    $old($dt)
    ->format('Y-m-d'),
    $new(DateTimeImmutable::createFromMutable($dt))
    ->format('Y-m-d'),
);
}

```

There are many ways to write such a test. Instead of using closures we could have also created private methods in the test class (e.g. `nextMondayOld()` and `nextMondayNew()`), or we could have extracted the functions to their own classes and put them in `Source/`, and so on.

Since the before and after functions accept different argument types and have different return types, we have to do some type juggling when calling them. At the end of the test we prove that their effective result is the same by converting the `DateTime(Immutable)` object to a string first, which can be more easily compared. Running the tests again, we see that the assertion count has increased, so we know the test is actually executed:

```
... 3 / 3 (100%)
```

```
Time: 00:00.782, Memory: 64.50 MB
OK (3 tests, 5 assertions)
```

We have now proven that the transformations our Rector ruleset makes are behavior-preserving.

## Shouldn't You Start with a Rule Set Test?

When implementing a migration process as a set of Rector rules, you usually have a clear vision of the end result. In the case of the `DateTime` migration we

also had this vision. In the beginning of the previous chapter we defined what the `nextMonday()` function would look like when migrated to `DateTimeImmutable`. Recognizing different aspects of the migration we worked on several smaller rules that took care of each of these aspects. In this chapter we started adding a test for the rule set as a whole. This test immediately passed, and our latest change to the rule set test also passed without any extra effort. The same goes for the behavior-preserving test. It also passed immediately. Normally you'd call this a *testing smell*. A test should help you discover the code you have to write to make the test pass. If the test passes, it doesn't seem to add anything.

What if we had changed the order? Imagine that we started with a rule set test, then created tests for each rule that we needed. The rule set test initially fails. Looking at the diff for the failing test, the easiest thing we can do is to rename `DateTime` to `DateTimeImmutable`. Once the test for this rule passes the rule set test still fails, but in a different way: `DateTime` will be renamed to `DateTimeImmutable`, but we don't capture the return value of `modify()` yet. We create another rule for this functionality. Once the rule works correctly, we run the rule set test again and notice the unnecessary `clone` step. Once again, we dive in and work on a rule that tackles this problem. We know that we're done when this rule passes, because at that point the rule set test also passes.

## Low-Hanging Fruit First

In summary, we “unlock” an entire high-level migration process by finishing the lower level rules that make up the complete migration. This is not an uncommon technique in software development. It's called top-down or behavior-driven development. As a developer we write a high-level test that describes the feature as a whole. This test fails, and we start implementing the low-level building blocks needed to eventually make the high-level test pass.

This is a different approach than bottom-up development, where we create smaller building blocks and finally combine them into a single feature. I actually prefer top-down development because bottom-up development comes with certain risks:

- You may be creating building blocks that you don't actually need. With top-down development all building blocks will be justified because they are created in support of the feature itself.

- You may be creating building blocks that are incompatible. Even if you test each building block in isolation, when using them together they may not produce the desired result. With top-down development you'll find out during development if any of the building blocks get in each other's way.



Discussing top-down versus bottom-up rule development with Tomas, he told me that Rector's own rules are thoroughly tested in isolation. When combined in a rule set, Rector generally doesn't have tests that show these rules work well together. Given the rules are sufficiently generic, a rule set test only confirms what you already know; they don't provide new insights, they don't increase the level of trust in the rule set. I think this means that bottom-up or top-down rule development are *styles* of test-driven development; you can experiment with each approach and decide which one works better for you.

## Create a Configurable Generic Rule

Having used the word “generic” several times now, it would be interesting to look into this topic some more. As you may know, test-driven development is iterative, and it moves between different stages continually. They are often described as:

1. Red
2. Green
3. Refactor

You start with a failing test, which shows a red bar on your screen. You then write the least amount of code possible to make the test pass. It's likely that you wrote code that doesn't look great, is hard to read, or complicated to change later on. So the last step is “refactor”. Between step 1 and step 2 the tests tell us when we're done, that is, when we have successfully implemented the behavior specified by the test. Between step 2 and 3 the tests switch to a different role. From that moment on the tests are there to save us from accidentally breaking existing behavior while we're cleaning up.

A common thing you'll do in the refactor stage is making a rule more generic, that is, making it useful in more situations than just the one you imagined. In some cases this means you modify your rule so it will be able to deal with more node types. Instead of dealing with method names you may also add support for function names. Instead of dealing only with traits you add support for parent classes, and so on.

## Using a Configurable Instead of a Custom Rule

Another common generalization is where you make certain values configurable. For example, we have created a rule that changes `DateTime` to `DateTimeImmutable`. If we allow the more generalized version of changing `ClassX` to `ClassY`, then this rule becomes useful in many more situations and projects. The rule would be called `RenameClassRector`, and... it already exists.

This may not be such a big surprise, and you may have already been screaming at this book when we started working on our own `UseDateTimeImmutableRector`. In fact, a very smart thing to do before you start creating your own rules is to look for existing rules on the [Rules overview](#)<sup>59</sup> page of the documentation. If we would've done this at the start of the previous chapter, we could have just skipped that first rule we wrote. The generic `RenameClassRector` rule has been tested extensively and used in real-world projects to automatically migrate to newer framework and library versions, so it's safe to use in your own projects. It also deals with many situations that we didn't think of when creating our own rule for this.

Let's use the existing `RenameClassRector` in our rule set, instead of the custom rule that we created. If you only set it as a service, it won't do anything. You have to give it a list of old class to new class pairs:

---

<sup>59</sup>[https://github.com/rectorphp/rector/blob/main/docs/rector\\_rules\\_overview.md](https://github.com/rectorphp/rector/blob/main/docs/rector_rules_overview.md)



```
utils/rector/config/sets/date_time_immutable.php
```

---

```
use Rector\Renaming\Rector\Name\RenameClassRector;

return static function (
    ContainerConfigurator $containerConfigurator
): void {
    $services = $containerConfigurator->services();

    $services
        ->set(RenameClassRector::class)
        ->call(
            'configure',
            [
                [
                    RenameClassRector::OLD_TO_NEW_CLASSES => [
                        DateTime::class => DateTimeImmutable::class,
                    ],
                ],
            ],
        );

    $services->set(CaptureReturnValueRector::class);

    $services->set(RemoveCloneOnDateTimeRector::class);
};
```

---

The convention for a Rector rule is to get its configuration values injected via a call to its public `configure()` method (which is defined in the `ConfigurableRectorInterface`). The configuration file follows the standard [PHP format of Symfony's service container configuration](https://symfony.com/doc/current/service_container/configuration.html)<sup>60</sup>. It might seem a bit confusing at first. Instead of directly calling `configure()` you have to instruct the service container to call `configure()` on the instantiated service object. The argument to this `call()` function is an array of the arguments that the container has to pass to `configure()`. `configure()` has a single parameter of type array. This explains the nested array.

---

<sup>60</sup>[https://symfony.com/doc/current/service\\_container/calls.html](https://symfony.com/doc/current/service_container/calls.html)

Before we continue, let's run the tests to find out if using `RenameClassRector` like this is in fact equivalent to the way we did it ourselves. Surprisingly, it doesn't do the exact same thing:

There was 1 failure:

```
1) MigrateToDateTimeImmutableRectorTest::test with data set #1
successful-migration.php.inc
Failed asserting that string matches format description.
--- Expected
+++ Actual
@@ @@
<?php

-$dt = new DateTimeImmutable();
+$dt = new \DateTimeImmutable();
 $dt = $dt->modify('next monday');
?>
```

The difference is related to how Rector deals with name nodes. Rector always uses the fully qualified class name (FQCN), which in this case means that a leading `\` will be included in the class name. It then relies on the global setting called “auto-import names” to either leave the name as it is, or import it with a `use` statement. This is how you

Given that we already have a configuration file for the rule set, we should actually set this configuration option there. It ensures that our test produces consistent output regardless of the project's own `rector.php` configuration file:

```
utils/rector/config/sets/date_time_immutable.php
```

---

```
use Rector\Core\Configuration\Option;

return static function (
    ContainerConfigurator $containerConfigurator
): void {
    $parameters = $containerConfigurator->parameters();
    $parameters->set(Option::AUTO_IMPORT_NAMES, true);

    $services = $containerConfigurator->services();
```

---

When running the tests again, we see them pass.

## Making Rule Behavior Configurable

Instead of generalizing `UseDateTimeImmutableRector` we ended up replacing it with `RenameClassRector`. Let's take a look at our second rule: `CaptureReturnValueRector`. The fact that the class itself contains a class name *and* a method name is a strong sign that we can make this rule more generic. We could make `DateTime` and `modify` configurable strings that can be provided to the rule via a call to `configure()`.

The best way to upgrade the rule to a configurable rule is to define one or more private properties that can hold the configuration values. The hard-coded values then become the default values of those properties:

```
utils/rector/src/Rector/CaptureReturnValueRector.php
```

---

```
use DateTime;

final class CaptureReturnValueRector extends AbstractRector
{
    /**
     * @var array<array{class-string, string}>
     */
    private array $classMethodPairs = [[DateTime::class, 'modify']];
```

```

/**
 * @param MethodCall $node
 */
public function refactor(Node $node): ?Node
{
    $parent = $node->getAttribute(AttributeKey::PARENT_NODE);
    if ($parent instanceof Assign) {
        return null;
    }

    foreach ($this->classMethodPairs as [$class, $method]) {
        if (! $this->isName($node->name, $method)) {
            continue;
        }
        if (! $this->isObjectType(
            $node->var,
            new ObjectType($class)
        )) {
            continue;
        }

        return new Assign($node->var, $node);
    }

    return null;
}

```

---

At this point we run the tests again to prove that the rule still works. It means that in theory the rule is ready to be configured from the outside. For this purpose we let the rule implement the `ConfigurableRectorInterface`, provide an implementation of its public `configure()` method in `CaptureReturnValueRector`, and add a constant that can be used in a configuration file:

```
utils/rector/src/Reactor/CaptureReturnValueReactor.php
```

---

```
use Reactor\Core\Contract\Reactor\ConfigurableReactorInterface;

final class CaptureReturnValueReactor extends AbstractReactor implements
ConfigurableReactorInterface
{
    public const CLASS_METHOD_PAIRS = 'class_method_pairs';

    /**
     * @var array<array{class-string, string}>
     */
    private array $classMethodPairs = [];

    /**
     * @param array<mixed> $configuration
     */
    public function configure(array $configuration): void
    {
        $this->classMethodPairs =
            $configuration[self::CLASS_METHOD_PAIRS] ?? [];
    }
}
```

---

I also gave the class a more generic name, because it no longer deals with `DateTime::modify()` specifically. At this point the name is too generic though, but we'll look at this later.

First, we should run the tests again. Unfortunately, they fail. That's because we don't call `configure()` yet, so the list of classes and method names is empty, and the rule will effectively do nothing. We have to call `configure()` and pass the previously hard-coded list containing `DateTime::class` and `modify`. This can be done in a test-specific service container configuration file which sets the rule service and adds a call to `configure()`:

```
utils/rector/tests/Reactor/CaptureReturnValueReactor/config/config.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
use Utils\Reactor\Reactor\CaptureReturnValueReactor;
```

```
use Symfony\...\ContainerConfigurator; // (abbreviated)
```

```
return static function (
```

```
    ContainerConfigurator $containerConfigurator
```

```
): void {
```

```
    $services = $containerConfigurator->services();
```

```
    $services
```

```
        ->set(CaptureReturnValueReactor::class)
```

```
        ->call(
```

```
            'configure',
```

```
            [
```

```
                [
```

```
                    CaptureReturnValueReactor::CLASS_METHOD_PAIRS => [
```

```
                        [DateTime::class, 'modify'],
```

```
                    ],
```

```
                ],
```

```
            ]
```

```
        );
```

```
};
```

---

We've used this kind of file before to configure a rule set. This time, config.php doesn't really define a reusable rule set, so it can live in the same directory as the test itself. This makes it very easy to refer to it in the `provideConfigFileInfo()` method of the test class:

```
utils/rector/tests/Reactor/CaptureReturnValueReactor/CaptureReturnValueReactorTest.php
```

---

```
public function provideConfigFilePath(): string
{
    return __DIR__ . '/config/config.php';
}
```

---

Running the tests again, they pass. We should now also update the `date_time_immutable.php` configuration file and provide the same configuration values there.

We have now successfully made the rule configurable. It can be used in other situations where we'd like to capture the return value of a method call. The same steps can be followed to make `RemoveCloneOnDateTimeReactor` work with other types of objects as well.

## Configurable Rule Makes Hidden Assumptions Explicit

Making a specific rule configurable often exposes some implicit assumptions. These assumptions were correct when the rule was still specific, but are potentially incorrect now that the rule is generic. Having renamed the class to `CaptureReturnValueReactor` exposes the issue. What we do is capture the return value of a method, but that return value is supposed to have the same class as the object on which the method is called. That is, it only works for methods that are part of a so-called *fluent interface*.

There are two options:

1. We can leave the rule as it is, and rely on the user to determine if the rule will work in their situation.
2. We can modify the rule to only perform the refactoring if everything looks right.

Let's explore the second option now, because it's the most interesting one. What we have to check is that the return type of the method is the same as the variable on which the method is called. As an example, before we change `$dt->modify()` into

`$dt = $dt->modify()` we should make sure that `modify()` returns an object that has the same type as `$dt` has at this moment.

Let's add a fixture that shows what we mean:

```
utils/rector/tests/Reactor/CaptureReturnValueReactor/Fixture/skip-non-fluent-method-call.php.inc
```

---

```
<?php
```

```
use Utils\...\ClassWithNonFluentMethodCall; // (abbreviated)
```

```
$object = new ClassWithNonFluentMethodCall();  
$object->modify();
```

---

## Using Supporting Classes in Fixtures

Instead of using invented classes again, ones that don't really exist, it'll be better to use real classes here, with actually defined methods, and actual return types that Rector can analyze. That's why the previous fixture imports the `ClassWithNonFluentMethodCall` from the `Source` namespace. This is another convention for Rector rule tests: if you need additional classes for your fixtures which aren't needed by the production code of your application, only for tests, then you put them in the `Source` directory which lives next to the `Fixture` directory. Classes located there will be auto-loadable if you give them a namespace that matches your auto-load configuration for Rector rule tests in the "autoload-dev" section of your `composer.json`.

This is what `ClassWithNonFluentMethodCall` looks like:



```
utils/rector/tests/Reactor/CaptureReturnValueReactor/Source/ClassWithNonFluentMethodCall.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Reactor\Tests\Reactor\CaptureReturnValueReactor\Source;
```

```
final class ClassWithNonFluentMethodCall
```

```
{
    public function modify(): SomeOtherClass
    {
        return new SomeOtherClass();
    }
}
```

---

And this is the `SomeOtherClass` that the return type of `modify()` refers to:

```
utils/rector/tests/Reactor/CaptureReturnValueReactor/Source/SomeOtherClass.php
```

---

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Utils\Reactor\Tests\Reactor\CaptureReturnValueReactor\Source;
```

```
final class SomeOtherClass
```

```
{
}
```

---

First we need to see the new test fixture fail. Running the test now makes it pass, because the rule configuration only contains `DateTime::modify()`, so the rule doesn't even attempt to change the `modify()` call on `ClassWithNonFluentMethodCall`. We should first add `ClassWithNonFluentMethodCall::modify()` as a candidate for refactoring in `config.php`:

```
utils/rector/tests/Reactor/CaptureReturnValueReactor/config/config.php
```

---

```
$services
->set(CaptureReturnValueReactor::class)
->call(
    'configure',
    [
        [
            CaptureReturnValueReactor::CLASS_METHOD_PAIRS => [
                [DateTime::class, 'modify'],
                [ClassWithNonFluentMethodCall::class, 'modify'],
            ],
        ],
    ],
);
```

---

Running the tests for this rule, we see it fail for the new fixture we added:

There was 1 failure:

```
1) CaptureReturnValueReactorTest::test with data set #1
skip-non-fluent-method-call.php.inc
Failed asserting that string matches format description.
--- Expected
+++ Actual
@@ @@
 use Utils\...\ClassWithNonFluentMethodCall; // (abbreviated)

 $object = new ClassWithNonFluentMethodCall();
-$object->modify();
+$object = $object->modify();

AbstractReactorTestCase.php:114
```

We can now start to modify the `refactor()` method to only refactor method call nodes where the return type looks alright. Again, we can use the built-in

`isObjectType()` method to verify that the return type of the called method matches any of the configured classes. To do this, we can conveniently call the `isObjectType()` method providing the `MethodCall` node itself:

`utils/rector/src/Reactor/CaptureReturnValueReactor.php`

---

```
/**
 * @param MethodCall $node
 */
public function refactor(Node $node): ?Node
{
    $parent = $node->getAttribute(AttributeKey::PARENT_NODE);
    if ($parent instanceof Assign) {
        return null;
    }

    foreach ($this->classMethodPairs as [$class, $method]) {
        if (!$this->isName($node->name, $method)) {
            continue;
        }
        if (!$this->isObjectType(
            $node->var,
            new ObjectType($class)
        )) {
            continue;
        }
        if (!$this->isObjectType(
            $node,
            new ObjectType($class)
        )) {
            continue;
        }
        return new Assign($node->var, $node);
    }

    return null;
}
```

---

Only if the current type of the variable matches the return type of the called method should we do the refactoring. Otherwise, we'll just ignore this class and method. At this point, all the tests pass, but that's not because our code works.



The problem with ignoring a class and method if they don't conform to our rules for refactoring the node, is that the user has provided the class and method name as input. If we can't perform the refactoring based on their input, shouldn't we provide some kind of error or warning? Discussing this with Tomas, he said mentioned two things about input validation:

1. A Rector rule should definitely validate the configuration values provided via a call to its `configure()` method. Inside this method you should throw an exception if anything is wrong, e.g. when you expect an array of strings, but you get an integer instead. In the Rector project itself the validation of configuration values is done in most places using *value objects*.
2. Once the configuration values have been validated, and Rector starts to run your rule on the parsed code, you should make sure that your Rector rule never throws an exception or triggers an error. This would stop the execution of Rector itself and prevent other rules from manipulating the code. Rector should be considered at all times a non-interactive command that should always be allowed to finish its job.

If a rule can't perform the refactoring because it doesn't know how to deal with the nodes provided to it, or because it encounters an edge case that it doesn't have a solution for, it should just do an *early return* instead.

Finally, we might give our rule class a better name, to better communicate its purpose. Maybe `CaptureModifiedInstanceReturnedByMethodCallRector`?

## Summary

Rules have to be *idempotent*: it should be possible to run them again and again, but they shouldn't perform the same refactoring again and again. Instead, they should refrain from updating the code if the code already looks good, according to the rule.

A Rector test visually compares the expected code with the actual code after running the rule.

A number of rules combined to implement some kind of migration process is called a rule set. You can test these rule sets just like normal rules.

Once the tests pass, rules can be refactored. One of the ways to improve a rule is by generalizing it, building in support for other situations than the one you built it for. One way to generalize a rule is by making it configurable.

If test fixtures need helper classes in order to make the code analyzable, you shouldn't store them in the `Fixture` directory but in the `Source` directory next to it. Make sure the classes in this directory are auto-loadable by giving them the correct namespace.

# Continuous Rectifying

## This Chapter Covers:

- Automating code reviews with Rector
- Effortlessly enforcing new programming rules
- Removing boring work, making way for more interesting work

## Introduction

PHPUnit is a tool that runs test on your code base; it makes sure the code still works after you change it. PHPStan is a tool that detects the types of your variables, method calls, arguments and so on; it makes sure your code does not make obvious mistakes like calling a method on `null`. ECS is a tool that makes your code look standardized; all of the spaces and brackets in every class and method will look the same.

It's very nice to run these tools on the command-line and get instant feedback about what is wrong and where. But their biggest added value is going to be somewhere else in 2021...

Imagine a new developer will join the team. Every time it happens you need to give them training about company standards: "No, we use tabs, not spaces. No, we don't use static service calls" etc. With all the right tools in place, instead of training we can give the person access to our repository and just let them make a pull request. Then continuous integration (CI) will become their trainer. They will get instant feedback from a failed CI job. PHPUnit failed? They should fix the logic. PHPStan failed? They should improve the types. ECS failed? There is a difference in code style. Just fix it and push again.

So how does all this CI feedback relate to Rector? We're glad you asked.

rectify *verb*

**rectify something** to put right something that is wrong

– Oxford Advanced Learner's Dictionary<sup>61</sup>

We already know how to use Rector to change our code, how to write our own rule and how to test it. You can also use Rector to upgrade to a new PHP version in just a fraction of the time it would take when doing this work manually. You can switch between frameworks over a weekend, or refactor from static service calls to constructor dependency injection. That sounds exciting, right? We bet you have ideas how Rector can help you get your project in shape again.

Certainly, some teams are using Rector this way: install it, handle a PHPUnit upgrade and a Symfony upgrade, then remove it again. That's quite sad, really. It's like using PhpStorm to open your project once and then switch back to Notepad again. The real power of Rector is not in quick jobs, even though it's very good at it. Its full potential can only be unlocked if you make it part of your project in the long run. When you work with Rector every day, it will continue helping you out, shares its wisdom, and taking care of the project.

## The Next Member of Your Team

Now let's consider what happens when you're not only using Rector for migrating legacy code bases or doing PHP upgrades. Imagine it's a member of your team. One that never sleeps, and helps you with all the code reviews. If it finds something to improve in the code you've just pushed, it won't annoy you with smart comments; it will handle the change for you, create a commit, create a pull request, and show you the result, only waiting for you to accept it. You can trust it to suggest only the changes you think are important, since you've told it to do so; you set up Rector in continuous integration, and told it what to look for.

## Turning a Junior Into a Senior on Day 1

If we add Rector to our CI, it will make sure the code base is always up-to-date. Let's say we hire a junior developer and she's working on its first pull request. She pushes

---

<sup>61</sup><https://www.oxfordlearnersdictionaries.com/definition/english/rectify?q=rectify>

few classes and we're about to review the code:

```
/**  
 * @var int  
 */  
public $count;
```

Oh, what she didn't notice is that we use PHP 7.4, so `$count` should be a typed property. You're on a break after lunch, so you'd be happy educate her in a review: "We're using PHP 7.4 so just use a type property."

*(10 minutes pass)*

- Junior replies: "What is typed property?"
- You: "You add the type directly to the property, like `public int $count;`."

*(5 minutes pass)*

- Junior: "ah, where can learn about that?"
- You (frustrated, because you're reviewing 3 more PRs on your phone while trying enjoy your coffee): "google it!"

After 20 minutes, you get a notification that the pull request has been updated:

```
/**  
 * @var int  
 */  
public int $count;
```

- You: "Good, why is there a doc block?"

*(7 minutes pass)*

- Junior: "What doc block?"
- You: "The `@var int` one."



- Junior: “Ah, should I remove it?”

*(10 minutes pass)*

- You: “Yes! It’s not needed anymore.”

- Junior: “Ah, we use it in the rest of the code, so I guessed it’s a way to go.”
- You: “Well, that’s some old code that we haven’t updated yet.”
- Junior: “How should I know? The code should be consistent everywhere, this is a mess.”
- You: “I know, but that’s where we are now”

A notification comes in: the new version has been pushed:

```
public int $count;
```

- You: “Good job!”

Then you continue reading further and see that CI failed, because the code is accessing an uninitialized PHP 7.4 typed property...

*The end*

## Who’s to Blame?

Are you a senior developer or CTO? You’d probably be frustrated that you have to comment on all these obvious problems that a junior developer could easily Google, read in the PHP manual, or find more information about on StackOverflow. That’s what you did for the last 10 years and that’s how you became who you are, right?

Are you a junior developer? You’d probably feel dumb because you “should know” all about the new PHP features, what version you use in `composer.json`, which part of the code is to be considered legacy code, etc. The code is confusing for you and your boss seems annoyed by your work.

Both parties are frustrated for obvious reasons, and that was just one property. PHP 7.4 has about 32 new RFCs that both parties have to watch out for. The frustration

usually means one thing. We intuitively know there must be a better way to do it, we just don't see it yet.



So who *is* to blame? The CTO or the junior developer? Neither. It's the poorly set up process that forces people to do work that can easily be automated. Do we point out in code reviews that an extra space is needed before a bracket? No, because it's handled by the coding standard. The same way many architecture conventions and PHP-version related code can be handled by a properly set up process.

It's the poor process that is to blame, not the people working with this process.

[...] rector is not only a tool, it's your private coach guiding you to be a better developer.

– [cavo789](#)<sup>62</sup>

## What Does it Look Like to Work with Rector in CI?

The junior pushes their first branch. The CI fails and Rector creates a pull request to show the diff and also adds a link to [the typed property RFC](#)<sup>63</sup> under the change:

```
-/**  
- * @var int  
- */  
-public $count;  
+public int $count = 0;
```

The junior notices the Rector's PR: "Oh, nice, I didn't know about this" (clicks on the link) "Wow, so that's how it works, now I know where to look when this comes up in code reviews!" The junior accepts the changes as proposed by Rector.

---

<sup>62</sup>[https://www.reddit.com/r/PHP/comments/kld9qy/rector\\_09\\_released/gha7zsc/?context=3](https://www.reddit.com/r/PHP/comments/kld9qy/rector_09_released/gha7zsc/?context=3)

<sup>63</sup>[https://wiki.php.net/rfc/typed\\_properties\\_v2](https://wiki.php.net/rfc/typed_properties_v2)

It's 16:05

You, on a break again, thinking:

- You, thinking “Ah, a new pull request from our junior programmer; hm, CI is passing. Good! Huh, PHP 7.4 typed property? I didn't even know we're on PHP 7.4 already!”
- You: “Good job, I'll merge it.”

It's 16:11

Both parties are at ease and calm since they only had to check crucial things that couldn't be handled by automation. You both feel like a valuable part of the team, and your work gets recognition. Next time the junior creates a pull request she will probably remember the feedback from the previous pull request and add a typed property before even pushing. Just to show her new skill.

The code is consistent, because Rector is watching over the entire code base with the same measures. There are no longer “it's a legacy directory, we have a different rules there” exceptions. The code is readable for everyone and new members can learn from this code much better than from chaos.

## How Rector is Rectifying Itself

Now, we have a theory, but does this work in practice? “Eat your own dog food”

The Rector repository itself is working exactly in the way we described above. Rector watches our back in every pull request that is made on the [GitHub repository](#)<sup>64</sup>. That means every pull request get checked by [13 rule sets](#)<sup>65</sup>, which include a total of 232 rules. That's 232 refactorings we never have to point out in code reviews, do manually, or even think about.

You can find all the core sets in [SetList class](#)<sup>66</sup>.

Which sets are used by Rector to rectify its own code base? The PHP sets are must-haves:

---

<sup>64</sup><https://github.com/rectorphp/rector-src>

<sup>65</sup><https://github.com/rectorphp/rector-src/blob/298c97a3bc72aed972935eea80eae14de2fca35e/rector.php#L29-L42>

<sup>66</sup><https://github.com/rectorphp/rector/blob/9987c84c58d681149363a3e55f5e1c68df16c9f6/packages/Set/ValueObject/SetList.php>

- PHP\_71
- PHP\_72
- PHP\_73
- PHP\_74
- PHP\_80

Then we follow up with sets that improve code consistency and readability:

- CODE\_QUALITY
- PRIVATIZATION

Naming is hard, so we let Rector handle it. In connection with type declarations the code and variable names are consistent and robust:

- TYPE\_DECLARATION
- TYPE\_DECLARATION\_STRICT
- NAMING

As a cherry on the cake, we add knowledge distilled from best PHP community sources:

- EARLY\_RETURN
- CODING\_STYLE
- DEAD\_CODE

This way can forget about so many techniques and best-practise we had already memorized. We can then use the memory we freed up to learn a new skill we really need at the moment.

Developers who contribute to Rector can push their code anytime they want and they always get instant feedback in a matter of minutes. If the feature is approved and CI passes, we are 99% certain that we can merge it.

## Work First, Enjoy Later

The biggest favor you can do to yourself is to work hard today so you can reap its benefits tomorrow.

What if a contributor adds code we don't entirely agree with? Should we give them feedback or let it pass? To continue on the code review story above: we don't put the burden on contributors, nor do we take responsibility for checking all the issues manually on during a code review. That would turn our human-to-human relationship into ping-pong.

Instead we accept the merge request. Then we create an issue for a new Rector rule that will handle the problem we wanted to point out. Then we add a Rector rule to our existing rule set and let it improve the whole code base.

As an example, let's say there is a property that has only one value and never changes. This property should be refactored to a constant:

```
+private const VALUE = 'some_static_value';

public function run()
{
-     $value = 'some_static_value'
    // ...
-     return $value;
+     return self::VALUE;
}
```

Great! Now we run Rector on our code and it will upgrade the code base instantly. As a bonus, this rule will be applied to all future pull request so we will never stumble over this problem again.

You can use a similar strategy for your own projects: what do you constantly talk about in code reviews? What are the aspects of the code that delay the merging of a PR the most? Identify these and create custom Rector rules for them. You won't have to worry about these problems again, and you'll shorten the life span of a pull request so new features can be released sooner.

## Removing Boring Work Opens Your Creativity

You can start with existing rule sets to simplify your code review process, try for instance `CODE_QUALITY`, `CODING_STYLE`, `PRIVATIZATION`, `NAMING`, `DEAD_CODE`, or `DEAD_DOC_BLOCK`. But every project is unique, every team is different and every business uses different techniques to build projects. That's why writing your own rules is a very creative way to push your business to the next level.

For a Nette project it can be a Rector rule for Components:

- When you create a Component, Rector will check if it has a factory.
- If it doesn't, it will create it and register it into services that generates control factories from every Control you create.

For an old Propel project it can be a Rector ruleset:

- When a new Propel active record is found Rector creates a Doctrine repository and a Doctrine entity based on it.
- You can start to use Doctrine while slowly leaving Propel.

Do you have tests full of mocks, that are not useful and only take you time to maintain?

- Make a rule that will convert useless mocks to use your project's DI container and real classes.
- When a new mock is created it will be instantly refactored to dependency injection.
- You don't have to teach your incoming developers the new rules; with your help, Rector handles it for them.

What would you like to automate today? If you can imagine doing it manually in PHP, you can make it happen automatically, with Rector. You already know how to write your own Rector rules. In the next chapter we'll discuss how to really make Rector a member of your team, by making it part of your CI with GitHub Actions.

## Summary

Rector can help us with code reviews. It can help senior developers with teaching juniors basics and conventions of the project. Then senior developers will have more time and energy to talk with junior about more important things that can't be automated, like soft-skills and critical thinking.

When using Rector in CI, developers don't have to wait for feedback. It's instant, 24 hours a day, detailed and with useful links providing opportunities for learning.

Once a common problem has been covered by a Rector rule, you will never have to think about it again, leaving you with more energy for work that matters more to you.

# Rector + GitHub Actions = Next Member of Your Team

## This Chapter Covers:

- Running Rector with GitHub workflows
- Letting Rector report problems issues on pull requests
- Automatically committing fixes for these issues

## Introduction

[GitHub Actions](#)<sup>67</sup> is a CI system from GitHub. How is it different from other systems?

- It's blazing fast
- It's free for open-source
- It has prepared “recipes” for all the common operations you can think of

These recipes are called “GitHub Actions”. They wrap complex functionality and only ask for a few configuration values. Do you need to push to remote SSH servers? Just provide your email address and a public key and that's it. Do you need to run `composer install` with cache for parallel jobs? You only have to change one line in your configuration file.

It's really handy that actions are stored in public repositories on GitHub. If we need to understand any action in depth, or want to learn how to use it, we just open the repository, e.g. [ramsey/composer-install](#)<sup>68</sup> and find out.

---

<sup>67</sup><https://github.com/features/actions>

<sup>68</sup><https://github.com/ramsey/composer-install>



This whole architecture allows us to create combos with various workflows that we stack upon each other: clone repository, generate the website, deploy the website, and tweet about the new post. Boom!

Let's add Rector to your project using GitHub Actions and see what it can do for us.

## What Are The Steps for a Rector Run?

GitHub Actions work with configuration files called “workflows”. There is a convention to keep one workflow per job. These workflows of a project are located in its `.github/workflow` directory.

For setting up a workflow that uses Rector we need the same four steps as we'd have in any local setup:

- Clone the repository
- Set up PHP with the version that Rector uses
- Install Composer dependencies
- Run Rector

In manual steps performed on the command-line, it might look like this:

```
# 1. clone the code
git clone git@github.com:some-organization/some-project.git
cd some-project

# 2. ensure that we have a compatible PHP version
# php -v | PHP 7.1 ... 8.0

# 3. install dependencies
composer install

# 4. run Rector
vendor/bin/rector process /src
```

These are common steps for developers, and they are usually quite simple to do. Now we'll take these four steps and put them into the workflow syntax.

# How to Add Rector to GitHub Actions

First, we'll create a new file `.github/workflow/rector.yaml`. That's where GitHub looks for Actions to run. We'll split the setup into two parts, so it's easier to follow:

- The first part is generic for any PHP operation. You can use it for running PHPUnit, PHPStan, or ECS as well.
- The second part is custom, and required only for Rector.

## 1. Generic PHP Setup

```
name: Rector
```

```
on:
```

```
  # the CI will be triggered on commits to pull requests
  pull_request: null
```

```
jobs:
```

```
  rector:
```

```
    # you can also chose from Ubuntu, Windows and MacOS
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      # clone the repository code to the CI workspace
      - uses: actions/checkout@v2
```

```
      # use PHP 8
```

```
      # (see https://github.com/shivammathur/setup-php)
```

```
      -
```

```
        uses: shivammathur/setup-php@v2
```

```
        with:
```

```
          php-version: 8.0
```

```
      # install Composer dependencies with cache
```

```
# (see https://github.com/ramsey/composer-install)
- uses: "ramsey/composer-install@v1"
```

At this point we have our project cloned, on an Ubuntu server running PHP 8, and with its Composer dependencies installed.

## 2. Rector Setup

All we have to do is prepend one more line:

```
# run Rector
- run: vendor/bin/rector process --ansi --dry-run
```

That's it.



### Why the `--dry-run` Option?

What is this `--dry-run` option doing there? Don't we want Rector to change the code?

Well, if a PHPUnit test fails, the CI reports failure. If PHPStan finds an error, the CI reports failure too. We want it to report failures so we know what we still have to work on. Rector is different because it isn't merely a reporting tool; it actually changes code. Should Rector report changed code as an error? It did the job right, so unless there were any fatal errors it should report success. That's why Rector uses the `--dry-run` [inverted exit code convention](https://github.com/rectorphp/rector/issues/800)<sup>69</sup>. If Rector finds code that needs to change on a dry run, it returns failure. It's the way CI tells us there is some code to refactor.

## What Feedback Do We Get?

To sum up, this is the full workflow we push to our repository. In order for it to work, we need to use pull requests and not directly push to the `main` branch, since the CI triggers only on pull requests:

---

<sup>69</sup><https://github.com/rectorphp/rector/issues/800>

```
name: Rector
```

```
on:
```

```
  # the CI will be triggered on commits to pull requests
  pull_request: null
```

```
jobs:
```

```
  rector:
```

```
    # you can also chose from Ubuntu, Windows and MacOS
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      # clone the repository code to the CI workspace
      - uses: actions/checkout@v2
```

```
      # use PHP 8
```

```
      # (see https://github.com/shivammathur/setup-php)
```

```
      -
```

```
        uses: shivammathur/setup-php@v2
```

```
        with:
```

```
          php-version: 8.0
```

```
      # install Composer dependencies with cache
```

```
      # (see https://github.com/ramsey/composer-install)
```

```
      - uses: "ramsey/composer-install@v1"
```

```
      # run Rector
```

```
      - run: vendor/bin/rector process --ansi --dry-run
```

## How to Work with Rector's Output?

If Rector fails on CI, there are three options for dealing with it:

1. You can fix the changes manually, one by one,
2. You can run Rector locally to do the changes for you,
3. Or you can let Rector do the work for you in a pull request of its own

## 1. Manual Changes

Let's say our pull request fails because Rector wants to make a change but we run it with the `--dry-run` option. It shows a diff for just 1 file with 5 lines of suggested changes in total. We can handle that ourselves! We go to that file, do the change manually as the Rector diff shows, commit, and push. The job is done, Rector is happy, and we can move on to another pull request.

We actually recommend this approach at the start, because it's very similar to traditional coding work. However, when there are 30 changes in 10 files fixing it manually might take a bit longer and start to become annoying... And that's the reason Rector was created. To save you from this daunting, boring work.

## 2. Run Rector Locally

If there are too many changes to handle them manually, we should run Rector locally:

```
vendor/bin/rector process src/
```

Rector is now working for us! Depending on the size of your project and how many rules you have enabled, this might take 30-300 seconds to finish. Even if it takes only 30 seconds, that's 30 seconds we'll never get back. 30 seconds that we have to book attention for. And we're not even finished. We have to add the modified code, commit it, and push it, and wait for feedback from the next CI run.

The CI is running... 2 minutes later... oops, we forgot to run the coding standard tool on the modified code. Okay, well, then apply the coding standard and commit again. We've just wasted 5 minutes on a task that can be fully automated and most importantly, we lost our flow. This method isn't good enough.

How can we let Rector automatically fix everything that's wrong about our pull request and just forget about it?

## 3. Let Rector Code for Us in Pull Requests

You don't tell your colleagues to type in the next line of code, to push Enter now, to fix this PHPStan error, do you? Why would you tell it to Rector? It's the next member of your team. It's here for you.

This is the most effective knowledge about next-level laziness you can get out of this book. If we have taught you to never run Rector yourself locally we succeeded.

## What is The Work We Want to Automate?

Is this hard to imagine? Let's describe what the manual steps are:

- Run Rector locally
- Apply coding standard on modified code
- Stage and commit the modified code
- Push the commit, thereby adding it to the pull request

Now let's transfer these manual steps to a bash script:

```
vendor/bin/rector process src/  
vendor/bin/ecs check --fix  
git add .  
git commit -m "Apply Rector run"  
git push
```

That's the whole process, nothing more.

If you can write it in a Bash Script, it can be in the CI

Do you see a Bash script? Let's turn it to a workflow.

But wait, there are some git operations. We need an SSH key locally and remotely, add a user name, email address, and so on. That will be lot of work with git commands, sudo hacking and low-level devops shenanigans. Well, on any other CI, that would be the case. But on GitHub Actions, we'll just find the right Action. We'll get to that later.

## Make Your Rector Workflow Come Alive

Now we'll upgrade our existing workflow we already have up and running:

```

name: Rector

on:
  pull_request: null

jobs:
  rector:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v2
+       with:
+         token: ${ secrets.ACCESS_TOKEN }

      - uses: shivammathur/setup-php@v2
        with:
          php-version: 8.0

      - uses: "ramsey/composer-install@v1"
-       run: vendor/bin/rector process --ansi --dry-run
+       run: vendor/bin/rector process --ansi
+
+       run: vendor/ecs check --match-git-diff --fix --ansi

```

The token is needed to enable Git commits and pushes from the CI process itself. How do we add it?

- First, [create an access token](https://github.com/settings/tokens/new)<sup>70</sup>.
- Then add it to the repository's "Secrets": which you find under *Settings - Secrets*.

We've also removed the `--dry-run` so Rector will actually change the code. Now we have the actual code changes and the pull request has correct access rights to push those changes too.

---

<sup>70</sup><https://github.com/settings/tokens/new>

Since Rector might change your code in ways that don't necessarily match your coding standard, we also run ECS with the `--fix` option. Your new Rector team member may be software, it still has to respect rules just like any other member.

## Make Rector Push Changes

How do we push the code? As we hinted, the GitHub Actions got us covered. There is [EndBug/add-and-commit](https://github.com/EndBug/add-and-commit)<sup>71</sup> that does exactly what we need.

We only add it our workflow:

```
# see https://github.com/EndBug/add-and-commit
-
  uses: EndBug/add-and-commit@v5.1.0
  with:
    # The arguments for the `git add` command
    # (see the paragraph below for more info)
    add: .
    message: "[ci-review] Apply Rector"
    # this is the convention of the bot we use for
    # CI-made changes, so we know who to blame :)
    author_name: "GitHub Actions"
    author_email: "action@github.com"
  env:
    # to get push access
    GITHUB_TOKEN: ${ secrets.ACCESS_TOKEN }
```

And that's it! This is our final workflow that will replace the old one:

---

<sup>71</sup><https://github.com/EndBug/add-and-commit>



```
name: Rector

on:
  pull_request: null

jobs:
  rector:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v2
        with:
          token: ${ secrets.ACCESS_TOKEN }

      - uses: shivammathur/setup-php@v2
        with:
          php-version: 8.0

      - uses: "ramsey/composer-install@v1"

      - run: vendor/bin/rector process --ansi

      - run: vendor/ecs check --match-git-diff --fix --ansi

      - uses: EndBug/add-and-commit@v5.1.0
        with:
          add: .
          message: "[ci-review] Apply Rector"
          author_name: "GitHub Actions"
          author_email: "action@github.com"
        env:
          GITHUB_TOKEN: ${ secrets.ACCESS_TOKEN }
```

Now you can push it, let Rector work for you and enjoy more flow then ever before.



## Eat Our Own Dog Food

Do you wonder what Rector's own GitHub workflows look like? Check out the [rector.yaml workflow](https://github.com/rectorphp/rector-src/blob/main/.github/workflows/rector.yaml)<sup>72</sup>. We have added neat comments to explain the process step by step.

## Summary

With GitHub Actions it's easy to make Rector your next team member. A standard workflow for Rector will clone the repository, set up PHP, install Composer dependencies, then run Rector with the `--dry-run` option. If Rector wants to change anything about the code in your pull request, this will fail the CI run.

When this happens, there are three options:

1. Make the necessary changes manually, or better:
2. Run Rector locally, or even better:
3. Delegate this repeated work to a GitHub Action.

Option 3 requires an additional ECS run against the modified code and commit and push rights for the CI runner so changes made by Rector and ECS can be automatically added to the pull request itself. This way you can focus on work that's important to you and let Rector handle all the boring repeated steps.

---

<sup>72</sup><https://github.com/rectorphp/rector-src/blob/main/.github/workflows/rector.yaml>

# The Future of Instant Upgrades

## This Chapter Covers:

- The future of legacy code
- The end of BC breaks
- Instant upgrade plugins and services

## Introduction

It takes time to fully grasp the power of new tools. We have to include them in our workflow and see where they help.

It took Composer three years to get 66 millions downloads a month. In 2021, the same number applies to [daily downloads](https://packagist.org/statistics)<sup>73</sup>. That's 3000% more than in 2015. When PHP-Parser was introduced, no one really noticed. 3 years later it had only 14k downloads a day. Now it's [302.000 downloads](https://packagist.org/packages/nikic/php-parser/stats)<sup>74</sup>.

It's not about the tools themselves. It's about the PHP community that finds new ways to use these tools. Whether it's a plugin, a tool built on top of it, or a project that is using only part of it.

This is not a book on how to use Xth version of already established language or framework. You have a unique advantage in your hands. We're on the verge of automated refactoring epoque. Only very few people in the whole PHP community have the power to reshape whole projects in minutes like you do now.

In this chapter we explore a few futuristic paths where building on top of Rector could lead us.

---

<sup>73</sup><https://packagist.org/statistics>

<sup>74</sup><https://packagist.org/packages/nikic/php-parser/stats>

## Removing Legacy

Legacy code is the number one problem in software development. It's so hard to get rid of it that we've learned to live with it like with a little bit of fat on our belly. It's something every project has, unless it was created last week. Legacy code is so popular that we can have hundreds of talks, books, and blog posts about it and there would still be something we didn't talk about.

Some projects are stuck on an old framework, e.g. Symfony 1.4. For other projects the PHP version is the problem. Most often they're stuck on PHP 5.6 because 7.0 had backward compatibility (BC) breaks not only in the language, but in all PHP packages that bumped their version. For yet another group of projects it's the homegrown-proprietary-framework that is slowing them down.

The following will sound really crazy in 2021, but we're sure that in the future all these problems will be solved with a single command-line execution.

## How Can We Be So Confident?

The best predictor of future behaviour is past behaviour.

Look at the trend: today we can already upgrade any PHP code from 5.3 to 8.0 with a single command. This was not possible in 2018. It's only a matter of time until we'll be able to cover more and more package upgrades this way.

The same way artificial intelligence (AI) is learning more and more complex tasks every year, more upgrade paths will be covered by the PHP community. Today we can upgrade Symfony from 3.0 to 5.2 with a single command, and with pretty big confidence. It's only a matter of time before we can do the same for Symfony 2.8, 2.6... or maybe even 1.4.

Proprietary frameworks will always be a challenge, each of them being slightly different. But "slightly different" means they share similar patterns. There is probably some kind of a request and response object. It will probably use controllers, database entities, repositories, etc. Again, it's a matter of time before we can identify those and create a generic migration rule set.

In the future we won't write as much about legacy code and how to get rid of it anymore. We won't have to explain 10 reasons SQL injection is bad and 50 places where to look for it. This is now the biggest waste of developer's time: to read a post or view a talk, understand it, look for it in your own code, and try to change it. Soon you'll quit, pronouncing the code "legacy code" and say we shouldn't touch it.

Instead, we should write rule sets that will do the upgrade work for us. Of course the rules will not be as good as a human expert. But if they fail in some edge cases, we'll improve them. These upgrade sets will be more affordable than a full-time human expert. They will become better over time, and you can use them any time you want.

## No Backward-Compatibility Breaks and No Backward-Compatibility

What happens if we bump the Symfony version in your `composer.json` file from 4 to 5? That's right: lots of fatal errors in our code. But if we go from Symfony 5.2.6 to 5.2.7, we can be sure nothing serious happens and nothing breaks. That's because the Symfony project decided to respect Semantic Versioning ([semver](https://semver.org/)<sup>75</sup>) and promises to only do BC breaks in major version changes (e.g. 4 to 5). But what actually is this promise for?

Let's look at an example. Say we used to have a `setValue()` method in Symfony 5.2.6, but in Symfony 5.2.7 they renamed it to `setItem()`. To keep backward-compatibility, they ensure that both versions still work by forwarding the call to `setValue()` to the new `setItem()` method:

---

<sup>75</sup><https://semver.org/>

```
/**
 * @deprecated, use setItem() instead
 */
public function setValue($value): void
{
    $this->setItem($value);
}

public function setItem($item): void
{
    $this->item = $item;
}
```

Now, are `setValue()` and `setItem()` features that we'll use separately?

```
$value = 100;

$valueObject->setValue($value);
$valueObject->setItem($value);
```

No. We'll use one, or the other. So now we have two ways to do one thing (yay!). It's like saying "we will not change it now, but be ready for the future". We already changed something, but we'll give you some time to upgrade your code; you decide when you do it.

We now have an extremely expensive workflow: for every change in the framework, every developer that uses this framework has to update their code base too. That's very annoying for the developers. But what about the framework developers? They have to think about every change: whether it could break someone's code, and how to prevent it. They also need to document the change, describe how to upgrade... phew, all that energy for a simple method rename. Imagine there are more serious changes like argument positions changing, new arguments being added, public methods being extracted to their own service - that's a real hell.

This is why established frameworks are evolving so slowly and new frameworks are coming to the market with an extremely fast pace of development. One example: Symfony vs Laravel.

In the ideal world, we want to handle a framework upgrade in the same way as we update our packages with Composer:

```
composer update
```

If a method name changes between Symfony 5.2.6 and Symfony 5.2.7, when we run `composer update` and it downloads 5.2.7, for us, the method should be renamed in our whole project:

```
$value = 100;

// does not exist anymore
-$valueObject->setValue($value);

// 1:1 replacement
+$valueObject->setItem($value);
```

That would be a dream come true. How can we achieve it? This one is simpler than you might think. Today, a package release is normally accompanied with a list of upgrade steps (often in Markdown format) that users can read and decide to apply now or later. In order to automate this process, the format of such an upgrade guide would have to become some kind of PHP recipe that contains the change itself - in this case a “method rename” change. On `composer update` a plugin would be able to read those recipes. Then Rector could apply them to the entire codebase. That way we’d install Symfony 5.2.7 and immediately use it with the new syntax. We can do the pull request review and merge. No BC breaks, only fresh new code.

## A Bot That Sends Pull-Requests Daily

Rector can already participate in your project by adding commits to pull requests. All you have to do is [set it up](#). But there is a much bigger potential here. The current workflow still requires a lot of setup from you, trying out different rule sets and keeping track of relevant framework upgrades. For every new Symfony release you still have to check if there are new upgrade sets available in Rector, etc.

Today we can already use tools like [dependabot](https://dependabot.com)<sup>76</sup>. It's a bot that automatically sends pull-requests to our repository. It updates package versions in `composer.json` and `composer.lock` when a new version of one of the packages required in your project is released.

If we can have a bot that updates versions in `composer.json` on daily basis, why not have a bot that would handle pull-requests with automated upgrades? When a new PHP 8.1 is released, such a bot will automatically send you a pull-request bumping your `composer.json` PHP version and upgrading your whole PHP project to PHP 8.1 syntax.

If you like, it will also send you pull-requests for removing dead code or improving code readability. It's up to you if you reject its suggestions or merge them. The bot will remember your decision and send you fewer of the pull-requests that you rejected and create more pull-requests like the one you've merged. Zero setup, and you'll only need to merge or close pull-requests. That's next level laziness, and we can't wait for it.

Fun fact: we worked on similar service for Rector, but instead of spreading our energy too wide, we decided to focus on Rector itself. But we're already aware of 2 such services in private beta - you'll see them coming in 2022/2023.

## Instant Upgrades on Click

A bot that sends you pull-request might seem like a bit out of control and it will take time to get it right. In the meantime there is a younger brother of this project. What will it do? It follows the best practice for pull-requests, to keep the change small and focused:

Do you want to add Symfony PHP 8 attributes? Click a button, and your project now has Symfony PHP 8 attributes. Do you want to use promoted properties in your constructors? Click a button, and your project now has promoted properties.



We're already working on this service and we'll open it to beta testers in the summer of 2021. Would you like to be the first to try? Send us an email (but don't tell anybody), we'll see if we can fit you in: [tomas@getrector.org](mailto:tomas@getrector.org)<sup>77</sup>.

---

<sup>76</sup><https://dependabot.com>

<sup>77</sup><mailto:tomas@getrector.org>



## PHP Stands Out

Last but not least, we're very proud to be part of the PHP community. It's the only community with such an advanced ecosystem. An ecosystem that is built on open-source technologies created and maintained by the smartest programmers on the planet. We don't think it's an exaggeration. It takes a very good programmer to create a beautiful piece of code, but it takes a great programmer to create a piece of code that is beautiful, stable for years, and explained well to the community. The RFC system is amazing, the open-source community, and the framework communities, which have no direct origin in commerce like Java or Javascript have.

We believe that PHP will become a leader in software technology, with the AST and all the automated tools built around it in the past 10 years.

What the future really holds? It's up to you, up to us.

# Epilogue

*By Matthias*

In October 2020 Tomas and I first discussed the idea of writing a book about Rector. Since then we've had various meetings where we talked about topics related to Rector, life as an open source maintainer, the future of legacy projects, and so on. Aside from the technical content that is in this book, I thought some of the other things we've talked about would also be worth sharing. So to finish the book we decided to add an epilogue in the style of an interview.

How much time do you spend on Rector?

Every day I spend about 3 to 5 hours on Rector. This is for dealing with new issues and pull requests on the main repository, or making small improvements. Sometimes more work is needed, for instance when I recently switched from runtime reflection to static reflection. PHPStan also made that transition last summer, and it made sense for Rector to do it too, since Rector heavily relies on PHPStan. However, this wasn't a change I could do in small steps. It had to be started and completed as a single effort. So to make it happen I spent three weeks of 8 hours per day.

How do you combine client work with open source work?

Sometimes Rector just requires my full attention, but normally I combine open source work with client work. I use Rector to help clients upgrade their projects to newer PHP or framework versions, or even to switch between frameworks. Client work always feeds back into the project. Instead of working on Rector itself I have to use Rector and experience how it works for its users. This creates empathy for people who submit issues for the project on GitHub. As the creator of a tool it's easy to reply "Just use it correctly". But things don't always work out as expected, and then I'm wondering as well: "Who wrote this tool?!"

Using Rector for client projects gives me new ideas too: how to make it easier to work with Rector, or what features are still missing. Then I go back to the Rector project itself, improve it, and this again will make my client work more effective.

In a way, by continuously improving Rector, you'll be out of client work eventually

Haha, well, clients thankfully pay me to get the work done. They hire me as an expert in automated refactorings. As for Rector itself: I'm not doing it for the money; I'm doing it because I want the tool to be the best version of itself. It needs to exist, and needs to be affordable, since many teams are out there stuck with old technology. Rector can help them. So when it comes to Rector itself, I work on issues that will be helpful for everyone. As an example, we're planning to offer a downgrade of Rector to PHP 5.6, so teams who are really behind can use it to get back to speed on modern PHP development.

What part of managing the Rector project was unexpected?

To deal with the workload I started hiring developers. That was a really new experience to me. Having to pay people on a regular basis, people who rely on you, is quite stressful. It means we have to steadily earn money as well. Also, I realized that paying or being paid for development comes with different mindsets. At the moment Rector has one awesome developer [Samsonasik](https://github.com/samsonasik)<sup>78</sup> who is doing amazing work with fixing bugs and helping others with their pull-requests.

An upside of working on Rector with other developers is that, by observing the contributions that those developers made, we learned about some of the things that were wrong about Rector. I think it's good to have multiple opinions in a software project. Not just the founders' opinion should matter.

What do you find are currently the major issues preventing adoption of Rector in teams or companies?

In the past it was Rector's own dependencies. There would be conflicts with the packages that were installed in the user's project. Also, PHP lower versions of PHP 7 weren't supported. Now that Rector can be installed with prefixed namespaces, and it has support for PHP 7.1, dependencies are no longer an issue. Nowadays it's mostly the mindset of programmers that gets in the way. They don't believe the technology to help them with boring upgrade even exists. Also, if developers do believe, the decision makers often don't: CTOs, project managers, etc. They often believe that when refactoring you have to make a small change manually, test it, commit it, then make the next small change. This is such an extremely expensive way of refactoring projects in the large. I heard of one company that had a 15-month

---

<sup>78</sup><https://github.com/samsonasik>

refactoring strike from Nette to Symfony and said they didn't need Rector for it: the 5-member team handle it. With Rector similar migration was done in 80 hours with 2-member team. That's 12 000 hours manually vs 80 hours with Rector. That's how expensive convergent mindset is.

#### What needs to be done to change this mindset?

Pre-COVID, going to conferences really helped. Seeing it done, seeing it work, will make it real. I think every tool has this problem, in particular if it changes code for you. PHPStan is easier to adapt because it will just warn you about issues, still leaving the actual work to the developer. You can use PHPStan on level 8 with all errors ignored and be happy about it. But Rector will go in and fix things for you, so it needs to gain more trust. In a year or two end people will feel like: how did we work without these tools?

#### What surprised you about how people use Rector?

The adoption for their own frameworks. CakePHP, Drupal, Typo3: they all started to create their own rulesets. Educating their own community. I learned about a lot of frameworks this way.

Abuse? Some people use it as a PhpStorm (e.g. rename class), yet Rectors' main focus is instant upgrades and automated migrations.

Surprise: PHP community is hungry for fully automated PHP downgrades. Why downgrades? Because with automated downgrades, every package maintainer could **bump min. version to PHP 8 today**<sup>79</sup>, while having automated support for the lower PHP versions. E.g. Rector 0.11 is developed on PHP 8 with promoted properties and downgraded on release to PHP 7.1.

#### What's currently lacking in the PHP ecosystem surrounding Rector?

Sometimes the Rector project has special needs, and fulfilling them brings a new tool into existence. As an example, recently we built a static Doctrine annotation parser. Something powerful that's missing right now is a Rector plugin for Composer. For instance, when you upgrade your Symfony dependencies from version 4 to version 5 with Composer, the plugin could also automatically run the Rector rules that correspond to this migration. This is a really powerful idea.

Right now upgrading a package is a really wasteful activity. Every developer who uses the package has to read the upgrade guide on their own. And everyone has

---

<sup>79</sup><https://getrector.org/blog/how-to-bump-minimal-version-without-leaving-anyone-behind>

to manually make the necessary changes on their own as well. If the package would provide the necessary Rector rules for every new version, the Composer plugin could pick up those rules and run them when the package would actually be updated in the user's project.

Frameworks can stop making `trigger_error()` calls to indicate deprecations. They would be allowed to make breaking changes. The user wouldn't even notice them, because Rector has already upgraded their code base to deal with those changes. Eventually, there's no need for semantic versioning of packages at all.

All the tools are there. We have Rector, Composer, and the ability to build Composer plugins. We also have the `BackwardCompatibilityCheck` package which can warn you about BC-breaking changes you make to your code. Combining them all should be doable, but of course, somebody has to do it.

Once framework leaders will enter this mindset, I'm sure we'll reach another level as PHP community. Even more so because other languages don't have this ability to upgrade and refactor their code in an automated fashion. We can change even faster and adopt more safely! There is no question if teams will be able to keep up with changes in their favorite framework. Their projects will never be hopelessly outdated anymore.

What's your dream for Rector in a couple of years?

Continuing this train of thought, my hope is to remove all discussions about deprecations, BC-breaking changes, etc. Just run `composer update` and everything still works. There should be 0 work for upgrades. But the same should be true for programming concepts. With Rector you could go from singletons to dependency injection and again, it should be 0 work. Being able to try 2 alternative concepts like this with 0 cost, it will be really helpful with learning. You don't need to read an article or a book about it, you can try it out by running Rector, give it a week, and if you don't like it after all, you can switch back without any effort. We could move into the future based on experience instead of quotes and loud opinions.

What do you hope people get from this book?

I hope that developers realize that Rector is there to save you from doing the boring work. I hope they'll use Rector and find their own creative way to save dozens of hours a month. This book should give the reader confidence: they can use Rector themselves, and let it solve their own specific problems. Upgrading PHP is one

example, which comes with the same challenges for everybody. But every project has its own problems too. Those too will become solvable. When developers find the confidence to do it, they will be able to do it, share it with others, and we will all rise as a PHP community.